Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales
Ingeniería en Computación

Redes de Computadoras

Monografía

MQTT y Protocol Buffers

Un stack de comunicación en tiempo real para IoT, robótica y mensajería con servicios en la nube

Equipo
WireGuardians

Viberti, Benjamin[1]	DNI 46224179
Espinoza Sutta, Aaron Alejandro[1]	DNI 96009173
Cleri, Juan Ignacio[1]	DNI 46452662
Pineda, Juan Ignacio[1]	DNI 45591343
Grafión, Atilio Leonel[1]	DNI 43940195
Badenes, Tomás[1]	DNI 44785038
Oviedo, Ignacio Nicolás[1]	DNI 44169239
Mendez, Jorge Nicolas[1]	DNI 41301342

Docente
Oliva Cuneo, Facundo Nicolas

[1]{b.viberti,aaron.espinoza_4500,ignacio.cleri,juan.ignacio.pineda,atilio.grafion,tomasbadenes,
ignacio.oviedo.239,jorge.mendez}@mi.unc.edu.ar

BORRADOR

8 de noviembre de 2026

Índice

Introducción **4**
1. Objetivos 5
2. Alcance 5
   2.1. Tecnologías estudiadas 5
   2.2. Casos de uso 6
   2.3. Mapeo tecnológico 6
   2.4. Parte experimental 6

Marco teórico **8**
1. Modelo publicador/suscriptor 9
2. MQTT 11
3. Protocol Buffers 13
   3.1. Esquema y generación de código 13
   3.2. Codificación binaria 13
   3.3. Compatibilidad entre versiones 14
   3.4. Limitaciones 14
4. Stack MQTT y Protocol Buffers 14
   4.1. Protocol Buffers como carga útil 14
   4.2. Asociación entre tópico y esquema 15
   4.3. Esquema compartido 16
   4.4. Sparkplug B 16
   4.5. Síntesis 17
5. Comparación con alternativas 17
   5.1. MQTT, HTTP y CoAP 18
   5.2. Protocol Buffers y JSON 20

Desarrollo **23**
1. MQTT sobre TCP y TLS 23
   1.1. Conexión persistente y keep-alive 23
   1.2. Cifrado con TLS 23
   1.3. QoS MQTT sobre la confiabilidad de TCP 23
2. Capa de red 23
   2.1. NAT y firewalls 23
   2.2. Ruteo interno 24
   2.3. Ruteo externo 26
   2.4. Convergencia y conexiones de larga duración 26
3. Ruteo por tópicos y ruteo IP 28
4. Casos de uso 28
   4.1. Mensajería instantánea 29
   4.2. Robótica 30
   4.3. Internet de las Cosas industrial 32
   4.4. Internet de las Cosas sobre redes LoRa 33
   4.5. Ventajas y limitaciones observadas 35
5. Diseño experimental 37
   5.1. Plataforma común 37
   5.2. Experimento 1: MQTT contra HTTPS para telemetría 37
   5.3. Experimento 2: sincronización de osciladores acoplados 55
6. Mapeo tecnológico 60

BORRADOR

6.1. Nodos 60
6.2. Conectividad 60
6.3. Broker 61
6.4. Costo del sistema 61
6.5. Escalabilidad 62
Conclusiones 64
Bibliografía 65
Anexos 71
A. Mensajes del experimento de sincronización 71

BORRADOR

Introducción

Gran parte de la Internet comercial se apoya en el modelo de solicitud y respuesta de HTTP: un cliente consulta a un servidor cuando necesita un dato y recibe la respuesta, que suele venir codificada en formatos de texto como HTML, JSON o XML. Este modelo está pensado para intercambios puntuales, que el cliente inicia a demanda, y no para flujos de datos continuos ni para que el servidor tome la iniciativa.

Un número creciente de sistemas no encaja en ese modelo. Los dispositivos del Internet de las Cosas, las flotas de robots y las aplicaciones de mensajería móvil generan flujos continuos de telemetría, eventos y comandos a alta frecuencia y en ambos sentidos, desde equipos de recursos acotados que operan sobre redes celulares o inalámbricas inestables y detrás de firewalls fuera del control de quien administra el servidor. Esta combinación plantea dos limitaciones de fondo para la ingeniería de redes: el costo, en ancho de banda y energía, de consultar periódicamente y de codificar los datos en formatos de texto; y la imposibilidad de que un servidor inicie comunicaciones hacia dispositivos que no admiten conexiones entrantes.

Frente a este panorama, muchos de estos sistemas han recurrido a la mensajería de tipo publicador/suscriptor, que desacopla a los participantes mediante un intermediario. En este campo se inscribe la combinación de MQTT y Protocol Buffers, un stack pensado para la comunicación en tiempo real con ancho de banda limitado. MQTT es un protocolo liviano de transporte de mensajes, organizado en torno a un broker, concebido para entornos restringidos (como los de comunicación máquina a máquina y de Internet de las Cosas) en los que se necesita una huella de código reducida o el ancho de banda es escaso. Protocol Buffers es un mecanismo, independiente del lenguaje y de la plataforma, para serializar datos estructurados y tipados en un formato más compacto y rápido que los textuales, a partir de un esquema compartido.

Su adopción en sistemas reales es amplia y heterogénea, como se analiza en la Sección 4. En la mensajería a gran escala, Facebook recurrió a MQTT en Messenger para mantener una conexión persistente con los teléfonos sin agotar su batería. En la robótica móvil, InOrbit, una empresa que ofrece una plataforma para supervisar y operar flotas de robots, comunica los robots con la nube mediante MQTT y mensajes definidos con Protocol Buffers, una tecnología que, según la propia empresa, gestiona miles de robots en producción y que publicó como código abierto en el proyecto OpenRobOps [1]. En la automatización industrial, la especificación Eclipse Sparkplug define precisamente esta combinación para comunicar los equipos de una planta, y en las redes de radio de largo alcance y bajo consumo, proyectos como Meshtastic transmiten mensajes de Protocol Buffers que llegan a Internet a través de un broker MQTT. Otros sistemas, como los estándares de interoperabilidad de robots VDA 5050 e ISO 21423, combinan MQTT con JSON, lo que muestra que las dos piezas del stack pueden elegirse de forma independiente.

Un rasgo de este esquema es especialmente relevante desde el encaminamiento: el dispositivo abre una conexión persistente hacia el broker alojado en la nube, cuya dirección suele estar habilitada en el firewall, y por esa misma conexión el broker le entrega los mensajes, sin necesidad de conexiones entrantes. El principio rector del stack es, entonces, trasladar la complejidad desde la red hacia el protocolo de aplicación: ningún participante necesita conocer la ubicación de los demás, sino solo la del broker, y el contrato sobre el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. De este modo, la comunicación en tiempo real entre miles de entidades heterogéneas resulta viable sobre redes que no fueron concebidas para ese fin.

BORRADOR

1. Objetivos

Se desea realizar un análisis riguroso del stack de comunicación formado por MQTT y Protocol Buffers, como alternativa para el intercambio de datos en tiempo real, a alta frecuencia y con ancho de banda limitado, en aplicaciones de Internet de las Cosas, robótica y mensajería. El estudio sigue el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye, y evalúa cómo cada capa de la arquitectura contribuye a que ese recorrido sea eficiente y confiable.

Más que catalogar las prestaciones de ambas tecnologías, el trabajo busca explicar por qué resultan adecuadas para el problema planteado: sostener una comunicación continua sobre redes que el diseñador no controla, con enlaces inestables, firewalls de terceros y trayectos que atraviesan múltiples sistemas autónomos. Con ese fin, el análisis se articula en torno a dos ejes centrales de la materia, que pertenecen a capas distintas: la capa de aplicación (capa 7), donde operan MQTT y Protocol Buffers, y la capa de red (capa 3), donde el ruteo determina por qué camino llegan los mensajes al broker y cómo se restablece ese camino cuando la red cambia.

Para alcanzar este propósito se plantean los siguientes objetivos específicos:

- Presentar el modelo publicador/suscriptor y describir cómo lo implementa MQTT (broker, tópicos, niveles de calidad de servicio y sesiones), junto con el mecanismo de serialización de Protocol Buffers, y compararlos con alternativas como HTTP, CoAP y JSON.
- Analizar el funcionamiento de MQTT sobre la capa de transporte (TCP, keep-alive y TLS) y distinguir las garantías de entrega de sus niveles de calidad de servicio de la confiabilidad que ya ofrece TCP.
- Analizar el recorrido de los mensajes en la capa de red (capa 3): NAT y firewalls, y el efecto de la convergencia de los protocolos de ruteo interno (OSPF, con RIP y EIGRP como comparación) y externo (BGP) sobre las conexiones TCP de larga duración en las que se apoya MQTT.
- Contrastar el ruteo por destino IP de la capa de red con el ruteo por tópicos que realiza el broker en la capa de aplicación.
- Examinar casos de uso reales en robótica, mensajería e Internet de las Cosas, como los de InOrbit, para determinar las ventajas y limitaciones del stack en entornos de alta frecuencia y recursos acotados.
- Medir experimentalmente, sobre nodos emulados, el costo en bytes y latencia de MQTT y Protocol Buffers frente a HTTPS y JSON, y su comportamiento ante pérdidas, latencia y cortes de conexión.

2. Alcance

El trabajo se centra en el stack formado por MQTT y Protocol Buffers, considerando distintos transportes para MQTT (TCP, TLS y WebSockets), y en el recorrido de un mensaje desde el dispositivo que lo genera hasta el broker que lo distribuye. A continuación se precisa qué aspectos abarca el análisis y cuáles quedan fuera de él.

2.1. Tecnologías estudiadas

En la capa de aplicación, el estudio considera MQTT en su versión 5.0 y Protocol Buffers en su sintaxis proto3. Del protocolo MQTT se analizan el modelo de broker, los tópicos y sus comodines, los niveles de calidad de servicio, las sesiones, los mensajes retenidos y el *last will*. De Protocol Buffers se analizan el esquema, la codificación binaria y la compatibilidad entre

BORRADOR

versiones. Como puntos de comparación se toman HTTP (y HTTPS) y CoAP como protocolos de transporte de mensajes, y JSON como formato de serialización.

En la capa de transporte, el análisis se limita a TCP y TLS en la medida en que afectan a MQTT, conceptos como la conexión persistente, el keep-alive, el costo del establecimiento de la sesión segura y la relación entre la confiabilidad de TCP y los niveles de calidad de servicio de MQTT. La seguridad se aborda únicamente desde el cifrado del canal y la autenticación de los clientes, sin un análisis integral de amenazas.

En la capa de red, se estudian NAT y los firewalls, y el ruteo interno y externo por el que los mensajes llegan al broker. Para el ruteo interno se toma OSPF como protocolo principal y se usan RIP y EIGRP como comparación. Para el ruteo externo se considera BGP. Estos protocolos se analizan en cuanto a su convergencia y su efecto sobre las conexiones TCP de larga duración, y no en cuanto a su configuración en equipos concretos.

2.2. Casos de uso

Los casos de uso abarcan la mensajería (Facebook Messenger), la robótica móvil (InOrbit, OpenRobOps y los estándares VDA 5050 e ISO 21423), la automatización industrial (Eclipse Sparkplug) y las redes de radio LoRa (Meshtastic y ChirpStack). Se analizan a partir de la documentación, las especificaciones y el código fuente que publican sus responsables, con foco en cómo emplean MQTT y Protocol Buffers y no en los detalles de cada dominio. Las redes LoRa, excluidas de la parte experimental, se consideran únicamente en este análisis, en el punto en que sus datos ingresan a una red IP.

2.3. Mapeo tecnológico

El trabajo incluye además un mapeo tecnológico, desarrollado en la Sección 6, con los equipos reales con los que se implementaría el escenario de telemetría del primer experimento (Sección 5.2) en lugar de los equipos emulados. Se relevan los nodos, el broker, ya sea en un servidor propio o como servicio administrado, y la conectividad, y para cada uno se indica el modelo, el proveedor y un precio de referencia con su fecha y moneda. El objetivo es estimar la viabilidad de una solución considerando la calidad técnica, el plazo y el presupuesto, y no proponer la alternativa de máxima prestación.

2.4. Parte experimental

Los experimentos se realizan sobre placas ESP32 emuladas con QEMU, que se comunican con un broker Mosquitto y un backend, estos cumplen la función de contrastar con mediciones propias lo que el marco teórico y el desarrollo plantean de forma cualitativa. El primero cuantifica la ventaja en bytes y en latencia que la bibliografía atribuye a MQTT y a Protocol Buffers, y permite separar cuánto aporta cada uno y cuánto cuesta el cifrado. El segundo ilustra el comportamiento del stack en una aplicación de tiempo real con intercambio a alta frecuencia, en la que las pérdidas, la latencia y los cortes de conexión analizados en el desarrollo tienen un efecto directo y observable sobre el resultado.

Se plantean dos experimentos en los que se comparan las siguientes combinaciones de protocolo y formato de serialización:

- MQTT con Protocol Buffers
- MQTT con JSON
- HTTP con Protocol Buffers
- HTTP con JSON

BORRADOR

El primer experimento se realiza en un escenario de telemetría y mide por separado el costo en bytes y en latencia del transporte y de la serialización. El segundo compara las mismas combinaciones en la sincronización de osciladores acoplados. En ambos, las condiciones de red, como las pérdidas, la latencia y los cortes de conexión, se introducen de forma artificial, y cada combinación se evalúa con y sin TLS.

La emulación impone limitaciones que acotan el alcance de los resultados. No reproduce el comportamiento de una radio real, por lo que no se mide el consumo de energía de los dispositivos y no se incluyen enlaces inalámbricos restringidos como LoRa, NB-IoT o satelitales. Tampoco reproduce los tiempos de procesamiento de un microcontrolador físico. Los escenarios de ruteo se emulan, en lugar de reproducirse sobre una red con múltiples sistemas autónomos reales. Por último, no se evalúan la escalabilidad del broker con miles de clientes ni las configuraciones en clúster.

BORRADOR

Marco teórico

Las redes de computadoras existen para hacer posibles las aplicaciones de red, es decir, programas que se ejecutan en distintos sistemas terminales y que se comunican entre sí a través de la red. La lógica de estas aplicaciones reside únicamente en los sistemas terminales, mientras que los nodos intermedios operan en la capa de red e inferiores. En consecuencia, quien desarrolla una aplicación no modifica la red, sino que utiliza los servicios que esta ofrece y define cómo se organiza la comunicación entre los extremos. MQTT y Protocol Buffers se ubican en este plano.

La forma predominante de organizar la comunicación a nivel de aplicación es el modelo cliente-servidor [2]. En este modelo, un servidor permanece activo en una dirección conocida a la espera de solicitudes, y un cliente lo contacta cuando requiere un servicio, envía una solicitud y aguarda la respuesta. La Web constituye el ejemplo característico, en el que el navegador solicita un recurso y el servidor lo devuelve. Se trata de un modelo orientado a la demanda, en el que la información se transmite únicamente cuando es solicitada y el intercambio es iniciado siempre por la parte que la requiere.

Este modelo presenta limitaciones para los sistemas considerados en este trabajo. En ellos, los dispositivos generan datos de manera continua y en instantes que no pueden anticiparse, y deben a su vez recibir comandos originados en la nube. Para que el servidor transmita un comando por iniciativa propia, debería actuar como cliente del dispositivo, lo que requiere que este acepte conexiones entrantes. Esta condición rara vez se cumple cuando el dispositivo se encuentra detrás de un NAT o de un firewall administrado por terceros. La alternativa que ofrece el propio modelo es la consulta periódica por parte del dispositivo, cuyo costo en tráfico y en latencia aumenta con la frecuencia de las consultas.

La capa de transporte ofrece una base para superar esta limitación. TCP establece una conexión persistente *full-duplex*, que permite a ambos extremos transmitir datos en cualquier momento con entrega fiable y ordenada. Si es el dispositivo quien abre la conexión hacia un servidor de dirección conocida, no necesita aceptar conexiones entrantes y conserva el rol de cliente. En ese caso, una vez establecida la conexión, el servidor también puede enviarle mensajes al cliente sin esperar que este realice una solicitud, lo que vuelve innecesaria la consulta periódica.

La conexión TCP, sin embargo, solo garantiza la entrega de una secuencia de bytes entre dos procesos. Qué mensajes se intercambian, con qué formato y en qué momento lo determina el protocolo de la capa de aplicación, que especifica los tipos de mensajes, su sintaxis y su semántica, así como las reglas que indican cuándo y cómo se envía y se responde cada uno de ellos [3].

En los sistemas estudiados, el diseño de ese protocolo está condicionado en primer lugar por la forma en que circula la información. Un mismo dato suele interesar a varios consumidores y cada consumidor recibe datos de muchos dispositivos, por lo que resulta conveniente que productores y consumidores puedan comunicarse sin conocerse entre sí ni estar conectados simultáneamente, que es la propuesta del modelo publicador/suscriptor implementado por MQTT. A esto se suma la representación de los datos dentro de cada mensaje, que debe ser interpretada de manera unívoca por equipos heterogéneos y ocupar poco ancho de banda en enlaces que suelen ser limitados, problema del que se ocupa Protocol Buffers.

BORRADOR

1. Modelo publicador/suscriptor

El modelo publicador/suscriptor (*publish/subscribe*) ofrece precisamente la posibilidad de que productores y consumidores de información se comuniquen sin conocerse entre sí, lo que lo hace adecuado para los sistemas descritos. En este esquema de interacción, los suscriptores expresan su interés en un evento o en un patrón de eventos, y son notificados de forma asíncrona de todo evento generado por un publicador que coincida con ese interés [4]. El modelo básico se apoya en un servicio de notificación de eventos, una abstracción que almacena las suscripciones y entrega cada evento publicado a todos los suscriptores interesados. Las suscripciones permanecen en ese servicio y son invisibles a los publicadores.

De esta manera, una de las características principales del modelo y común a todas sus variaciones es el desacoplamiento total de las entidades que se comunican, que puede descomponerse en tres dimensiones [4]:

- El **desacoplamiento en el espacio** (*space decoupling*) implica que las partes no necesitan conocerse, ya que los publicadores no mantienen referencias a los suscriptores ni saben cuántos participan de la interacción, y lo mismo ocurre en sentido inverso.
- El **desacoplamiento en el tiempo** implica que las partes no necesitan participar de la interacción al mismo tiempo, por lo que un publicador puede publicar eventos mientras un suscriptor está desconectado, y un suscriptor puede recibir la notificación de un evento cuyo publicador ya se desconectó.
- El **desacoplamiento de sincronización** implica que los publicadores no se bloquean al producir eventos y que los suscriptores son notificados mientras realizan otras actividades, fuera del flujo principal de control de ambos.

Este desacoplamiento reduce la coordinación necesaria entre los dispositivos participantes de la comunicación y favorece la escalabilidad en entornos asíncronos por naturaleza.

Estas tres dimensiones permiten contrastar el modelo con el cliente/servidor descrito al comienzo del capítulo. En el caso del paradigma cliente/servidor el cliente debe conocer la dirección del servidor, ambos deben estar activos durante el intercambio y el cliente queda a la espera de la respuesta, por lo que la interacción está acoplada en el espacio, en el tiempo y, del lado del cliente, en la sincronización. La misma comparación puede extenderse a otros paradigmas de comunicación distribuida, como resume la Tabla 1. La invocación remota de procedimientos (RPC) presenta el mismo acoplamiento que el modelo cliente-servidor, mientras que sus variantes asíncronas y el patrón *observer* eliminan únicamente el acoplamiento de sincronización. Los espacios de tuplas y las colas de mensajes desacoplan a productores y consumidores en el espacio y en el tiempo, pero obligan a los consumidores a extraer los datos de forma síncrona. El modelo publicador/suscriptor es el único que reúne las tres propiedades.

BORRA-DOR

Tabla 1: Desacoplamiento que ofrece cada paradigma de comunicación [4, p. 121].

Paradigma	**Espacio**	**Tiempo**	**Sincronización**
Pasaje de mensajes	No	No	Solo el productor
RPC/RMI	No	No	Solo el productor
RPC/RMI asíncrona	No	No	Sí
RPC/RMI con futuros	No	No	Sí
Notificaciones (*observer*)	No	No	Sí
Espacios de tuplas	Sí	Sí	Solo el productor
Colas de mensajes (*pull*)	Sí	Sí	Solo el productor
Publicador/suscriptor	Sí	Sí	Sí

Las variantes del modelo publicador/suscriptor se distinguen por la forma en que los suscriptores especifican los eventos de interés. En el esquema basado en tópicos, el más antiguo, los eventos se publican bajo nombres que los sistemas modernos organizan en jerarquías, y la sintaxis de comodines permite suscribirse a todos los tópicos de la jerarquía que coincidan con el patrón dado. Este esquema puede implementarse de manera muy eficiente, aunque puede resultar limitado. El esquema basado en contenido clasifica los eventos según sus propias propiedades, mediante filtros que define el suscriptor, y ofrece mayor expresividad a costa de protocolos más complejos y de un mayor costo de ejecución. El esquema basado en tipos clasifica los eventos según su tipo, lo que permite verificar en tiempo de compilación que productores y consumidores coinciden en la estructura de los datos. MQTT, descrito en la Sección 2, adopta el esquema basado en tópicos.

La implementación del servicio de eventos admite a su vez distintas arquitecturas. En una arquitectura centralizada, los productores envían los mensajes a una entidad que los almacena y los reenvía a los consumidores. En una arquitectura distribuida no existe tal entidad, y los mecanismos de almacenamiento y reenvío residen en los propios procesos de productores y consumidores, mientras que un enfoque intermedio implementa el servicio como una red de servidores dedicados. En la arquitectura centralizada, que es la que adopta MQTT, la entidad central se denomina *broker*. Este suele comunicarse con publicadores y suscriptores mediante canales punto a punto confiables y conservar copias de los eventos en almacenamiento estable, lo que le permite ofrecer persistencia y entrega garantizada. Como contraparte, el uso de un broker agrega un salto a cada mensaje y concentra la carga de todos los participantes en un punto de falla que tiene el potencial de demorar la entrega de los eventos [4].

Sobre una red TCP/IP, la arquitectura centralizada se apoya en interacciones cliente-servidor. Tanto los publicadores como los suscriptores establecen la conexión hacia el broker, por lo que todos ellos actúan como clientes, y el broker es el único participante que debe tener una dirección conocida y aceptar conexiones entrantes. Su función es la de un sistema intermedio que mantiene conexiones TCP con cada extremo y retransmite los mensajes entre ellas, de manera análoga a los intermediarios definidos para HTTP [5]. La diferencia con el modelo cliente-servidor aparece una vez registrada la suscripción, ya que el broker entrega cada evento por la conexión que el suscriptor estableció previamente, sin requerir de una solicitud. De este modo, los datos llegan al suscriptor en el momento en que se producen, sin que este deba consultar periódicamente ni aceptar conexiones entrantes. Esta capacidad de entregar datos por iniciativa del otro extremo es común a cualquier canal bidireccional persistente, como una conexión WebSocket, pero lo que distingue al modelo publicador/suscriptor es que, sobre esos canales, el broker determina a

BORRADOR

qué suscriptores corresponde cada evento de modo que publicadores y suscriptores permanecen desacoplados aun cuando cada uno mantiene una conexión punto a punto con el broker.

2. MQTT

MQTT es un protocolo de transporte de mensajes de tipo publicador/suscriptor, basado en una arquitectura cliente-servidor y diseñado para ser liviano, abierto y fácil de implementar [6]. La primera versión estandarizada por OASIS fue la 3.1.1, adoptada también como norma ISO/IEC 20922, y la versión 5.0, en la que se centra este trabajo, incorpora numerosas funcionalidades nuevas sin alterar el núcleo del protocolo. MQTT adopta la arquitectura centralizada descrita en la sección anterior denominando *broker* al servidor y cliente a todo programa o dispositivo que abre una conexión hacia él para publicar mensajes y/o suscribirse a ellos, y deja a cada implementación la elección de los mecanismos de autenticación, autorización y cifrado. MQTT delega en la capa de transporte la entrega ordenada y sin pérdidas de los datos a la capa de transporte, por lo que funciona sobre TCP, TLS o WebSocket, pero no directamente sobre UDP.

Cada mensaje de aplicación lleva un nombre de tópico, el cual consiste en una cadena de texto que el servidor compara con las suscripciones de los clientes. Los tópicos se organizan en niveles separados por el carácter `/`, y las suscripciones se expresan mediante filtros que se comparan con los tópicos de mensajes entrantes. Las expresiones de suscripción pueden incluir los comodines `+`, que coincide con exactamente un nivel, y `#`, que solo puede ubicarse al final del filtro y coincide con cualquier cantidad de niveles.

De esta manera las expresiones de suscripción permiten, por ejemplo, en una flota de robots que publica en tópicos de la forma `flota/{ID de robot}/{sensor}`, suscribirse con el filtro `flota/+/bateria` que recibe el estado de batería de todos los robots o el filtro `flota/robot1/#`, que recibe todos los mensajes de el robot `robot1`.

Implementar el esquema basado en tópicos, MQTT implementa desacoplamiento en el espacio, ya que el publicador solo conoce el nombre del tópico y es el servidor quien determina qué clientes reciben cada mensaje. La versión 5.0 agrega además las suscripciones compartidas, en las que cada mensaje se entrega a uno solo de los clientes de un grupo, lo que permite repartir el procesamiento entre varios consumidores.

La entrega de los mensajes se rige por tres niveles de calidad de servicio (QoS), cuyos intercambios de paquetes muestra la Figura 1. Con QoS 0, el mensaje se envía sin confirmación ni reintentos y llega una vez o ninguna. Con QoS 1, el receptor confirma cada mensaje con un paquete PUBACK y el emisor lo conserva hasta recibir esa confirmación, lo que asegura que llegue al menos una vez aunque pueden producirse duplicados. Con QoS 2, un intercambio en dos etapas garantiza que el mensaje llegue exactamente una vez, a costa de una mayor sobrecarga.

El nivel de QoS se aplica por separado entre el publicador y el broker y entre el broker y cada suscriptor, y cada suscriptor recibe el mensaje con el menor nivel de QoS entre el usado al publicarlo y el máximo concedido a su suscripción. La relación entre estos niveles y la confiabilidad que ya ofrece TCP se analiza en la Sección 1.3.

BORRADOR

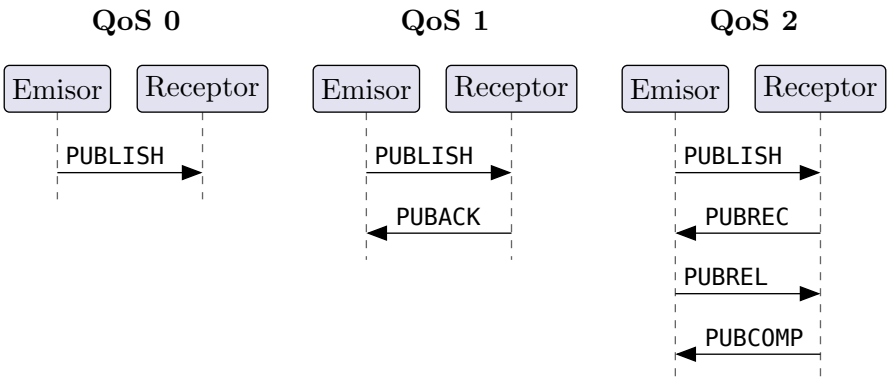


Figura 1: Intercambio de paquetes de control en cada nivel de QoS.

Las confirmaciones de QoS 1 y 2 requieren que cliente y servidor mantengan un estado de sesión. En el servidor, ese estado comprende las suscripciones del cliente y los mensajes destinados a él que aún no fueron enviados o confirmados, y puede conservarse a lo largo de varias conexiones consecutivas. Al conectarse, el cliente indica si inicia una sesión nueva o retoma la anterior, y durante cuánto tiempo debe conservarse el estado una vez cerrada la conexión. Este mecanismo es el que permite el desacoplamiento en el tiempo, ya que el servidor almacena los mensajes dirigidos a un suscriptor desconectado y se los entrega cuando este vuelve a conectarse. Sin embargo, ese desacoplamiento no es incondicional, ya que si la sesión no persiste más allá de la conexión, o si los mensajes se publican con QoS 0 que el servidor no está obligado a almacenar, un suscriptor desconectado nunca recibe los mensajes publicados, por lo que corresponde a la aplicación elegir la combinación adecuada para cada tipo de dato.

Complementando este comportamiento, MQTT suma dos mecanismos más. Un mensaje publicado con el indicador RETAIN se almacena en el servidor como último mensaje retenido de su tópico y se entrega a cada nuevo suscriptor, de modo que un cliente que se suscribe recibe de inmediato el estado más reciente aunque el publicador lo envíe de forma irregular. El mensaje de última voluntad (*Will Message*), por su parte, lo registra el cliente al conectarse y el servidor lo publica a los suscriptores si la conexión se cierra de forma anormal, por ejemplo cuando el cliente deja de comunicarse dentro del intervalo de keep-alive, que es el tiempo máximo que puede transcurrir entre dos paquetes enviados por el cliente. Esto permite notificar a los interesados de la desconexión de un dispositivo.

Por último, toda la comunicación en MQTT se realiza mediante paquetes de control, de los cuales la especificación define quince tipos. Entre ellos se encuentran CONNECT y CONNACK para establecer la conexión, PUBLISH para transportar los mensajes de aplicación, PUBACK y los demás paquetes de confirmación de los niveles de QoS, SUBSCRIBE y UNSUBSCRIBE para gestionar las suscripciones, y PINGREQ y PINGRESP para el keep-alive. Cada paquete se compone de un encabezado fijo, presente en todos ellos, seguido de un encabezado variable y de una carga útil que solo aparecen en algunos tipos. El primer byte del encabezado fijo indica el tipo de paquete y algunos indicadores, como el nivel de QoS y RETAIN en el caso de PUBLISH, y a continuación se codifica la longitud restante del paquete con un entero de longitud variable que ocupa un solo byte para valores de hasta 127. De este modo, el encabezado fijo ocupa apenas dos bytes en los paquetes pequeños, que es también el tamaño total de los paquetes de keep-alive. El uso de este tipo de encabezados resulta en una economización del costo de transporte, la cual se evalúa en el experimento de la Sección 5.2.

El contenido de los mensajes queda fuera del protocolo MQTT, que transporta la carga útil sin interpretarla. La versión 5.0 permite declarar su formato mediante las propiedades de indicador

BORRADOR

de formato y tipo de contenido, que el servidor reenvía sin modificar, pero no impone ninguno, aspecto que se retoma en la Sección 4. La representación de los datos queda así a cargo de la aplicación, que es el problema que aborda Protocol Buffers, descrito en la Sección 3.

3. Protocol Buffers

Una conexión TCP entrega una secuencia de bytes, y un broker MQTT reenvía el contenido de cada mensaje sin interpretarlo. Para que un dato producido por un microcontrolador sea leído correctamente por un servicio escrito en otro lenguaje y ejecutado en otra arquitectura, ambos extremos deben acordar cómo se traduce una estructura de datos a bytes y viceversa, incluyendo el orden de los bytes, el tamaño de los enteros y la forma de delimitar cada campo. En el modelo OSI esta tarea corresponde a la capa de presentación [5], pero la arquitectura TCP/IP no la incluye, por lo que cada aplicación debe resolverla por su cuenta [2].

Protocol Buffers, también conocido como Protobuf, es un mecanismo para serializar datos estructurados que es neutral respecto del lenguaje de programación y de la plataforma [7]. Fue desarrollado por Google, que lo publicó como software libre y mantiene implementaciones oficiales para una variedad de lenguajes [8]. Su propuesta es comparable a la de JSON, pero con mensajes más pequeños, un procesamiento más rápido y la generación de código nativo para cada lenguaje. Según su documentación, ofrece un formato de serialización para paquetes de datos estructurados y tipados de hasta unos pocos megabytes, adecuado tanto para el tráfico de red efímero como para el almacenamiento de datos a largo plazo, y que puede extenderse con nueva información sin invalidar los datos existentes ni obligar a actualizar el código [7]. Estas propiedades explican su amplia adopción por parte de grandes empresas de internet y en arquitecturas de microservicios [9].

3.1. Esquema y generación de código

La estructura de los datos se define una única vez en un archivo de esquema con extensión `.proto`, en el que cada mensaje se describe como un conjunto de campos con un tipo, un nombre y un número que lo identifica. Por ejemplo, la lectura de un sensor ambiental podría definirse así:

```
syntax = "proto3";

message Lectura {
  float temperatura = 1;
  float humedad     = 2;
  uint32 bateria    = 3;
}
```

A partir de este archivo, el compilador `protoc` genera el código fuente que permite construir, serializar y deserializar el mensaje en cada lenguaje de destino. De este modo, el `.proto` cumple a la vez la función de documentación y la de contrato explícito entre los equipos o servicios que intercambian los datos, y un mismo esquema produce implementaciones compatibles en todos los lenguajes soportados.

3.2. Codificación binaria

En la codificación binaria de Protobuf, cada campo se transmite precedido de una etiqueta (*tag*) que combina el número de campo definido en el esquema con un tipo de codificación (*wire type*), que indica cómo leer el valor a continuación, como un entero de longitud variable, un valor de

BORRADOR

32 o de 64 bits de longitud fija, o un bloque precedido de su longitud, que se usa para cadenas, bytes y mensajes anidados [10]. Se trata, en esencia, de una codificación de tipo-longitud-valor (TLV), como la de BER.

Los enteros se codifican como *varints*, en los que cada byte aporta siete bits del valor y utiliza el bit más significativo para indicar si el número continúa en el byte siguiente. Así, los valores pequeños, que son los más frecuentes, ocupan un solo byte. Por ejemplo, un campo número 1 con el valor 150 se codifica en tres bytes, `08 96 01`: el primero es la etiqueta y los dos restantes el valor [10]. Para los enteros con signo existe además la codificación *zigzag*, que intercala positivos y negativos para que los valores negativos pequeños también resulten cortos. El mensaje no incluye los nombres de los campos, que solo existen en el esquema, y en proto3 los campos que tienen su valor por defecto directamente no se transmiten [11]. Esta combinación es la que hace que un mensaje Protobuf pese varias veces menos que su equivalente en JSON y que se procese más rápido.

3.3. Compatibilidad entre versiones

Dado que los campos se identifican por número y no por posición ni por nombre, el esquema puede evolucionar sin romper a quienes usan versiones anteriores. Un programa que recibe un campo cuyo número no conoce simplemente lo ignora, de modo que los clientes viejos pueden leer mensajes nuevos (compatibilidad hacia adelante). En sentido inverso, un programa nuevo que lee un mensaje viejo encuentra los valores por defecto en los campos ausentes (compatibilidad hacia atrás). La condición para que esto funcione es no reutilizar números de campo, para lo cual el lenguaje ofrece la palabra clave `reserved`, que impide volver a asignar los números o nombres de campos eliminados [11]. Esta compatibilidad en ambos sentidos es la principal fortaleza de Protobuf frente a otros formatos binarios.

3.4. Limitaciones

Las mismas decisiones que reducen el tamaño de los mensajes tienen costos. En primer lugar, el formato no es legible por personas, por lo que la depuración requiere herramientas como `protoc --decode` o conversores a JSON. En segundo lugar, no es autodescriptivo: sin el esquema no es posible interpretar los datos, ya que el binario no indica qué mensaje contiene ni qué significa cada campo. En tercer lugar, la codificación no delimita los mensajes, por lo que el protocolo que los transporta debe indicar dónde termina cada uno. Por último, agrega al proyecto un paso de compilación y de generación de código, y resulta menos práctico que JSON para APIs públicas simples o para su uso directo desde navegadores, aunque para ese caso existen alternativas como gRPC-Web.

4. Stack MQTT y Protocol Buffers

Las secciones anteriores describieron MQTT y Protocol Buffers por separado. Esta sección analiza cómo se combinan: qué lugar ocupa cada uno en un mensaje, cómo sabe el receptor qué tipo de mensaje recibe, qué implica compartir el esquema entre los participantes y qué requisitos de los sistemas estudiados cubre cada pieza.

4.1. Protocol Buffers como carga útil

MQTT y Protocol Buffers resuelven problemas distintos y complementarios:

BORRADOR

- MQTT define cómo se distribuyen los mensajes entre los participantes, pero no impone ningún formato para su contenido, ya que la carga útil de cada paquete PUBLISH es una secuencia de bytes cuyo contenido y formato son específicos de la aplicación [6].
- Protocol Buffers define cómo representar datos estructurados en una secuencia de bytes compacta, pero no cómo transportarla, como se vio en la Sección 3.

La combinación consiste, entonces, en serializar cada mensaje con Protocol Buffers y publicarlo como carga útil de un paquete PUBLISH, que el broker reenvía a los suscriptores sin interpretarlo. A modo de analogía, el mensaje de Protocol Buffers es la carta y el paquete PUBLISH, el sobre que la transporta: el broker lee la dirección escrita en el sobre, que es el tópico, pero nunca lo abre.

La división de tareas se complementa también en la delimitación de los mensajes. Como se señaló en la Sección 3, la codificación de Protocol Buffers no indica dónde termina un mensaje, por lo que, si se envían varios por un mismo flujo, la aplicación debe anteponer el tamaño de cada uno [12]. Sobre una conexión TCP, que solo entrega una secuencia continua de bytes, esa tarea quedaría a cargo de la aplicación. MQTT, en cambio, transporta cada mensaje de aplicación en un único paquete PUBLISH, cuya longitud se obtiene de su encabezado fijo [6], de modo que cada paquete contiene exactamente un mensaje de Protocol Buffers.

4.2. Asociación entre tópico y esquema

Un mensaje de Protocol Buffers tampoco indica a qué tipo pertenece, y sin su esquema no puede interpretarse [7], por lo que el receptor debe saber de antemano cuál de los mensajes definidos en el archivo `.proto`, descrito en la Sección 3, contiene cada carga útil. La práctica habitual es asociar cada tópico, o cada patrón de tópicos, a un único tipo de mensaje. De este modo, el tópico cumple una doble función:

- Determina a qué suscriptores entrega el broker cada mensaje, como en cualquier sistema basado en tópicos.
- Identifica el esquema con el que se interpreta el contenido del mensaje.

Por ejemplo, en la flota de robots que publica en tópicos de la forma `flota/{ID de robot}/{sensor}`, el tópico `flota/robot1/bateria` puede transportar siempre un mensaje `EstadoBateria`, y un suscriptor con el filtro `flota/+/bateria` sabe que todo lo que recibe se interpreta con ese mismo esquema.

Además de esta convención, la versión 5.0 de MQTT incorpora propiedades opcionales que el publicador puede agregar al paquete PUBLISH y que resultan útiles para la combinación. El broker reenvía estas propiedades a cada suscriptor junto con el mensaje, sin modificarlas, con excepción del alias de tópico, que solo tiene validez dentro de cada conexión [6]:

- El **indicador de formato de la carga útil** (*Payload Format Indicator*) distingue entre bytes sin especificar, como un mensaje de Protocol Buffers, y texto codificado en UTF-8, como un documento JSON.
- El **tipo de contenido** (*Content Type*) es una cadena que puede nombrar el tipo de mensaje transportado. La especificación no fija ningún valor para Protocol Buffers, por lo que el emisor y el receptor deben acordarlo. Los sistemas que todavía usan MQTT 3.1.1, como los de robótica analizados en la Sección 4.2, no disponen de esta propiedad y deducen el tipo de cada mensaje a partir del tópico.
- Las **propiedades de usuario** (*User Property*) admiten pares nombre-valor definidos por la aplicación, como la versión del esquema.

BORRADOR

- El **tópico de respuesta** (*Response Topic*) y los **datos de correlación** (*Correlation Data*) permiten implementar solicitudes y respuestas sobre MQTT (como un comando enviado a un robot y su confirmación), ambos codificados con Protocol Buffers.
- El **alias de tópico** (*Topic Alias*) reemplaza un nombre de tópico largo y repetido por un entero de dos bytes, lo que complementa en el encabezado la reducción de tamaño que Protocol Buffers logra en el contenido.

4.3. Esquema compartido

El archivo `.proto` constituye así un contrato entre los participantes, ya que el formato de los datos se fija una única vez en un esquema distribuido entre los clientes. A partir de ese archivo se genera el código de cada participante, ya sea el firmware de un microcontrolador o un servicio en la nube [7]. En el experimento de la Sección 5.2, por ejemplo, un mismo esquema genera el código de las placas ESP32 y el del backend que recibe sus datos.

Ese contrato introduce un acoplamiento en el formato de los datos que el modelo publicador/suscriptor no elimina. Como publicadores y suscriptores no se conocen ni se actualizan al mismo tiempo, un cambio en un mensaje puede llegar a unos participantes antes que a otros. El desacoplamiento en el tiempo acentúa el problema, ya que un mensaje retenido, o almacenado en la sesión de un suscriptor desconectado, puede leerse mucho después de su publicación con una versión más nueva del esquema. Las reglas de compatibilidad descritas en la Sección 3 vuelven manejable esta situación, siempre que no se modifiquen ni se reutilicen los números de campo existentes [11]:

- Un publicador puede agregar campos sin afectar a los suscriptores que usan el esquema anterior, que simplemente los ignoran.
- Un suscriptor con el esquema nuevo puede leer los mensajes de los publicadores que no se actualizaron, ya que los campos ausentes toman su valor por defecto.

La combinación tiene, a su vez, costos propios que se suman a las limitaciones de cada pieza:

- El broker solo puede encaminar los mensajes según su tópico, ya que no interpreta su contenido, lo que corresponde a la menor expresividad del esquema basado en tópicos frente al basado en contenido [4].
- Todo componente de la infraestructura que necesite leer los datos debe contar con el esquema. Esto alcanza también a los servicios que reciben los mensajes en la nube. Por ejemplo, AWS IoT Core, el servicio de Amazon Web Services que actúa como broker MQTT para dispositivos, puede decodificar mensajes de Protocol Buffers en sus reglas de procesamiento, pero solo si se le proporciona un archivo descriptor generado a partir del `.proto` [13].
- Las herramientas genéricas de MQTT permiten verificar que los mensajes circulan, pero muestran su contenido como bytes ilegibles, lo que dificulta la depuración [14].

4.4. Sparkplug B

La madurez de esta combinación se refleja en su estandarización. La especificación Eclipse Sparkplug, adoptada como norma ISO/IEC 20237 [15], define para los sistemas industriales precisamente lo que MQTT deja abierto, y en su versión B codifica la carga útil con Protocol Buffers [16]. Entre otras cosas, fija el espacio de tópicos, y con él la asociación entre tópico y esquema, y la forma de informar el estado de conexión de cada nodo mediante el mensaje de última voluntad. Su funcionamiento se analiza en la Sección 4.3.

BORRADOR

4.5. Síntesis

Cada pieza del stack cubre una parte de los requisitos planteados al comienzo de la sección, como resume la Tabla 2. Cuánto aporta cada una al tamaño y a la latencia de los mensajes se mide en el experimento de la Sección 5.2.

Tabla 2: Requisitos de los sistemas estudiados y pieza del stack que cubre cada uno.

Requisito	**Pieza del stack que lo cubre**
Enviar datos al dispositivo sin que acepte conexiones entrantes	Conexión TCP abierta por el dispositivo y entrega por parte del broker
Comunicar productores y consumidores que no se conocen	MQTT: broker y tópicos
Tolerar desconexiones	MQTT: sesiones persistentes, mensajes retenidos y mensaje de última voluntad
Delimitar cada mensaje	MQTT: un paquete PUBLISH por mensaje
Identificar el tipo de cada mensaje	Asociación entre tópico y esquema; en MQTT 5.0, tipo de contenido
Interpretar los datos de forma unívoca entre equipos heterogéneos	Protocol Buffers: esquema compartido y código generado para cada lenguaje
Reducir el uso de ancho de banda	MQTT: encabezado fijo mínimo y alias de tópico; Protocol Buffers: codificación binaria
Modificar los mensajes sin afectar a los demás participantes	Protocol Buffers: reglas de compatibilidad del esquema

Algunas empresas y proyectos que aplican este stack, analizados en la Sección 4, son InOrbit y OpenRobOps, en robótica [1], [17]; Meshtastic, en redes LoRa [14]; y Amazon Web Services, en la nube, con su servicio AWS IoT Core [13].

5. Comparación con alternativas

Para intercambiar datos entre un dispositivo (ya sea un sensor, un celular o un robot) y la nube deben tomarse dos decisiones:

1. Qué protocolo de capa de aplicación implementar (define cómo se intercambian los mensajes).
2. Qué mecanismo de serialización se utiliza para representar los datos dentro de cada mensaje (con qué formato viaja la información).

Las secciones anteriores describieron la combinación de MQTT y Protocol Buffers como respuesta a ambas decisiones para el intercambio de datos en tiempo real con ancho de banda limitado, pero no es la única opción, ya que existen otras alternativas consolidadas. Esta sección analiza esas alternativas para identificar en qué aspectos MQTT y Protocol Buffers aventajan a cada una de ellas y en cuáles resultan inferiores. En particular, se compara MQTT con otros dos protocolos de capa de aplicación: HTTP, sobre el que se apoya gran parte de la Internet comercial, y CoAP, diseñado específicamente para dispositivos restringidos. Del mismo modo, se compara Protocol Buffers con JSON, un mecanismo de serialización basado en texto y de uso extendido.

La comparación de protocolos considera el modelo de interacción, el protocolo de transporte, el tamaño de los encabezados, la confiabilidad y el paso por NAT y firewalls, mientras que la de mecanismos de serialización considera el tamaño de los mensajes, el costo de procesamiento y la necesidad de un esquema compartido. El análisis se apoya en las especificaciones de cada tecnología y en estudios publicados.

BORRADOR

5.1. MQTT, HTTP y CoAP

Se definirán brevemente los protocolos HTTP y CoAP, para posteriormente adentrarse en sus diferencias con MQTT.

5.1.1. HTTP

Se trata de un protocolo de aplicación **sin estado** de **solicitud y respuesta**. Cada solicitud se interpreta de manera independiente, por lo que el servidor no conserva información sobre los clientes entre una solicitud y la siguiente [3], [5, p. 780]. Es el protocolo de la Web y el ejemplo característico del modelo cliente-servidor: el cliente envía una solicitud que indica un método (como GET o POST) y el recurso al que se refiere (identificado por una URI), y el servidor devuelve una respuesta con un código de estado y, eventualmente, el contenido solicitado [18].

HTTP utiliza TCP como protocolo de transporte, del que obtiene la entrega fiable y ordenada de los datos [3]. En el modelo de solicitud y respuesta, el servidor no puede iniciar una conexión con el cliente ni enviarle una respuesta que no haya sido solicitada. Por eso, un cliente que necesita enterarse de eventos nuevos debe consultar periódicamente al servidor, técnica conocida como *polling* [19].

5.1.2. CoAP

CoAP (*Constrained Application Protocol*) es un protocolo de transferencia web estandarizado por el IETF (*Internet Engineering Task Force*) para nodos y redes restringidos, como microcontroladores con poca memoria que operan sobre redes de bajo consumo y con pérdidas. Está orientado a aplicaciones máquina a máquina (M2M) [20].

Adopta el mismo modelo de solicitud y respuesta que HTTP, con recursos identificados por URI, métodos como GET, POST, PUT y DELETE, y códigos de respuesta. Sin embargo, no busca comprimir HTTP, sino implementar el subconjunto de esa arquitectura que necesitan las aplicaciones M2M, y está diseñado para traducirse fácilmente a HTTP mediante *proxies*, lo que facilita su integración con la Web [20].

CoAP funciona sobre UDP, con DTLS como mecanismo de seguridad, y codifica sus mensajes en un formato binario compacto [20]. Como UDP no garantiza la entrega, el propio protocolo incorpora una confiabilidad liviana: los mensajes confirmables (CON) se retransmiten hasta que el receptor responde con un acuse de recibo (ACK), mientras que los no confirmables (NON) se envían sin confirmación, lo que resulta adecuado, por ejemplo, para cada lectura de un flujo de datos de un sensor [20].

Por último, la extensión Observe permite que un cliente se registre en un recurso para que el servidor le envíe una nueva representación cada vez que ese recurso cambia, lo que evita el *polling* [21].

5.1.3. Comparación

De las descripciones anteriores y de los estudios que comparan estos protocolos se desprenden las siguientes diferencias:

1. **Modelo de interacción.** HTTP y CoAP siguen el modelo de solicitud y respuesta, en el que el cliente aplica métodos sobre recursos y recibe códigos de respuesta. En ese modelo, el dato viaja cuando el cliente lo pide, por lo que para enterarse de eventos nuevos debe recurrir al *polling*, cuyo costo crece con la frecuencia de las consultas, o al *long polling*, en el que cada notificación sigue siendo un mensaje HTTP completo, con todos sus encabezados [19]. La extensión Observe acerca CoAP a un modelo de notificaciones, pero sin un broker

BORRADOR

que desacople a las partes, y su propia especificación aclara que no pretende reemplazar a los sistemas publicador/suscriptor [21]. MQTT, en cambio, implementa el modelo publicador/suscriptor: el broker entrega cada mensaje en el momento en que se publica, mediante la conexión que el suscriptor estableció previamente, y puede conservar los mensajes destinados a un cliente desconectado mediante sesiones persistentes y mensajes retenidos [6], algo que HTTP y CoAP no prevén.

2. **Protocolo de transporte.** HTTP y MQTT funcionan sobre TCP, del que heredan la entrega fiable y ordenada, pero deben establecer una conexión antes de intercambiar datos. MQTT la establece una sola vez y la mantiene abierta, mientras que HTTP puede abrir una conexión por cada solicitud o reutilizarla para varias [3]. CoAP, en cambio, funciona sobre UDP y no necesita establecer una conexión, de modo que una transacción se resuelve con dos datagramas, uno en cada sentido [22].
3. **Tamaño de los encabezados.** HTTP utiliza encabezados de texto de longitud variable, mientras que los de CoAP y MQTT son binarios: el encabezado fijo de CoAP ocupa cuatro bytes [20] y el de MQTT, dos bytes como mínimo [6]. Sin embargo, la conexión TCP que requiere MQTT aumenta su sobrecarga total. Por eso, en las comparaciones cualitativas, HTTP resulta el más pesado en tamaño de mensaje, consumo de energía, ancho de banda y latencia, CoAP el más liviano y MQTT ocupa una posición intermedia (esas comparaciones no consideran las retransmisiones) [22].
4. **Confiabilidad.** HTTP depende exclusivamente de la confiabilidad de TCP. MQTT le agrega tres niveles de calidad de servicio y el mensaje de última voluntad, mientras que CoAP ofrece, mediante sus mensajes no confirmables y confirmables, dos niveles aproximadamente equivalentes a QoS 0 y QoS 1 [22]. Ante pérdidas, MQTT presenta menor retardo que CoAP cuando las pérdidas son bajas y mayor cuando son elevadas, y con mensajes pequeños y pérdidas de hasta el 25% CoAP genera menos tráfico adicional para garantizar la entrega [23]. En un estudio sobre la comunicación entre vehículos y la nube, CoAP obtuvo las menores latencias y la menor sobrecarga, pero no logró entregar todos los mensajes con pérdidas elevadas, ni siquiera con mensajes confirmables, mientras que los protocolos basados en TCP, entre ellos MQTT, los entregaron todos [24].
5. **Paso por NAT y firewalls.** Muchos firewalls están configurados para bloquear el tráfico UDP [3]. Entre el 2% y el 4% de las redes de acceso terrestres lo bloquean por completo, y las restricciones se concentran en las redes empresariales; además, los NAT descartan las asociaciones UDP mucho antes que las TCP (en promedio, a los 160 segundos frente a los 386 minutos), lo que obliga a enviar mensajes de mantenimiento con mayor frecuencia [25]. Esto favorece a HTTP y a MQTT frente a CoAP, que para estos casos cuenta con una variante sobre TCP, TLS y WebSocket, a costa de un mayor tamaño de código, más memoria y más intercambios [25].
6. **Seguridad.** HTTP y MQTT se protegen con TLS sobre TCP, mientras que CoAP utiliza DTLS sobre UDP [22]. MQTT, además, deja a cada implementación la elección de los mecanismos de autenticación y autorización [6].
7. **Interoperabilidad y formato de los datos.** HTTP es el protocolo más interoperable, ya que basta con disponer de una pila HTTP en cada extremo [22], y CoAP se integra con la Web mediante *proxies* que traducen a HTTP [20]. MQTT, en cambio, requiere un broker y no indica en sus mensajes el formato del contenido, por lo que todos los clientes deben conocerlo de antemano [22]; la versión 5.0 agrega propiedades opcionales para declararlo, pero no impone ninguno [6]. Ninguno de los tres protocolos impone un mecanismo de serialización, aspecto que se analiza en la siguiente subsección.

BORRADOR

5.1.4. Ventajas y desventajas

Frente a HTTP, la principal ventaja de MQTT es que el broker puede entregarle datos al dispositivo sin que este los solicite, lo que evita el *polling*. A esto se suman encabezados mínimos, una única conexión que se mantiene abierta y sirve en ambos sentidos, niveles de calidad de servicio propios y la posibilidad de conservar los mensajes de un cliente desconectado. Como contrapartida, MQTT es menos interoperable que HTTP, requiere un broker, que agrega un salto a cada mensaje y concentra la carga de todos los participantes, como se señaló al describir el modelo publicador/suscriptor, y obliga a que los clientes acuerden de antemano el formato de los datos.

Frente a CoAP, la ventaja de MQTT es la confiabilidad: al funcionar sobre TCP, en los estudios citados entregó todos los mensajes aun con pérdidas elevadas, y atraviesa con menos dificultad los NAT y firewalls, y su broker ofrece un desacoplamiento completo entre publicadores y suscriptores, que las notificaciones de mejor esfuerzo de Observe no alcanzan. Como contrapartida, CoAP es más liviano: genera menos sobrecarga, no requiere la pila TCP y admite solicitudes *multicast* [20], para las que MQTT no está diseñado [22]. Para redes en las que TCP no es viable, como algunas redes inalámbricas de sensores, existe una variante, MQTT-SN, que funciona sobre un servicio de datagramas y se conecta al broker mediante un *gateway* [26].

En síntesis, MQTT resulta preferible cuando se necesita una entrega confiable de datos en ambos sentidos sobre redes que no se controlan, con dispositivos detrás de NAT o firewalls; CoAP conviene para dispositivos muy restringidos que envían actualizaciones pequeñas sin necesidad de entrega garantizada [24]; y HTTP, para intercambios puntuales iniciados por el cliente o cuando la integración con la Web es prioritaria. La Tabla 3 resume la comparación.

Tabla 3: Comparación entre HTTP, CoAP y MQTT.

Criterio	**HTTP**	**CoAP**	**MQTT**
Modelo de interacción	Solicitud y respuesta	Solicitud y respuesta (con Observe, notificaciones de mejor esfuerzo)	Publicador/suscriptor
Entrega sin solicitud previa	No (requiere *polling*)	Solo con Observe	Sí, a través del broker
Clientes desconectados	Sin entrega posterior	Sin entrega posterior	Sesiones persistentes y mensajes retenidos
Transporte	TCP	UDP	TCP
Encabezado	Texto, de longitud variable	Binario, 4 bytes fijos	Binario, 2 bytes fijos como mínimo
Confiabilidad	La de TCP	Mensajes confirmables y no confirmables	La de TCP y QoS 0, 1 y 2
Paso por NAT y firewalls	Favorable (TCP)	Desfavorable (UDP)	Favorable (TCP)
Seguridad	TLS	DTLS	TLS
Interoperabilidad	La más alta	Alta, mediante *proxies* a HTTP	Requiere un broker y un formato acordado

5.2. Protocol Buffers y JSON

Se definirá brevemente JSON, para posteriormente contrastarlo con Protocol Buffers.

BORRADOR

5.2.1. JSON

JSON (*JavaScript Object Notation*) es un formato de intercambio de datos liviano, basado en texto e independiente del lenguaje de programación, derivado del estándar del lenguaje ECMAScript [27]. Representa los datos mediante objetos, que son conjuntos de pares nombre-valor, y arreglos, cuyos elementos pueden ser cadenas de texto, números, valores booleanos, el valor nulo u otros objetos y arreglos. Fuera de un ecosistema cerrado, los textos JSON deben codificarse en UTF-8 [27]. Por ejemplo, una lectura de un sensor con su temperatura y su nivel de batería puede representarse así:

```
{"temperatura":23.5,"bateria":80}
```

Como cada mensaje incluye los nombres de sus campos, JSON es autodescriptivo: puede leerse e interpretarse sin conocer de antemano la estructura de los datos, del mismo modo que los mensajes de HTTP/1.1, escritos en texto legible [3]. Esa característica determina su principal ventaja y su principal desventaja:

- **Ventaja:** cualquier persona o programa puede leer el mensaje sin un acuerdo previo sobre su estructura, lo que facilita la depuración y la interoperabilidad.
- **Desventaja:** los nombres de los campos viajan en cada mensaje y los números se representan como texto decimal. El ejemplo anterior ocupa 33 bytes, de los cuales 22 corresponden a los nombres de los campos y sus comillas. Además, la especificación recomienda no esperar más precisión que la de un número de punto flotante de doble precisión, por lo que solo garantiza la interoperabilidad de los enteros de hasta $2^{53} - 1$ en valor absoluto [27].

5.2.2. Contraste con Protocol Buffers

De la descripción anterior, de la presentada para Protocol Buffers y de los estudios que comparan ambos mecanismos se desprenden las siguientes diferencias:

1. **Representación.** JSON codifica los datos como texto, mientras que Protocol Buffers utiliza un formato binario en el que cada campo se codifica como un registro de tipo, longitud y valor, identificado por un número de campo en lugar de su nombre [10].
2. **Esquema compartido.** JSON no requiere un esquema previo. Protocol Buffers, en cambio, requiere que emisor y receptor compartan el archivo `.proto` que define la estructura de los mensajes, ya que sin él no es posible interpretar los datos recibidos. A cambio, ese esquema permite generar código para cada lenguaje y fijar el tipo de cada campo [7].
3. **Tamaño de los mensajes.** Al reemplazar los nombres por números de campo y codificar los valores en binario, Protocol Buffers produce mensajes considerablemente más pequeños. En un relevamiento sobre 27 documentos JSON de distintos ámbitos, Protocol Buffers redujo el tamaño con una mediana del 70,6% [28]; con cargas reales tomadas de mensajes MQTT públicos, el tamaño medio pasó de 111 bytes en JSON a 40 bytes [29]; y con mensajes de producción de vehículos conectados, de 12.187 a 1.157 bytes en promedio [24]. La reducción, sin embargo, no es universal: en uno de los 27 documentos, formado por arreglos muy anidados y casi sin nombres de campo, Protocol Buffers resultó más grande que JSON [28].
4. **Costo de procesamiento.** Aunque la documentación presenta a Protocol Buffers como más rápido que JSON [7], en microcontroladores la diferencia no es clara: en algunas arquitecturas, la biblioteca de JSON evaluada serializó más rápido. Aun así, con radios lentas transmitir un byte consume unas diez mil veces más energía que un ciclo de CPU, por lo que Protocol Buffers resultó la opción más eficiente en energía, a costa de un mayor uso de memoria RAM [29].

BORRADOR

5. **Tipado y evolución.** JSON ofrece solo tipos genéricos y no impone una estructura, por lo que nada garantiza que emisor y receptor interpreten los datos de la misma manera. Protocol Buffers asigna un tipo concreto a cada campo y controla la evolución de los mensajes: el código nuevo puede leer mensajes antiguos, y los campos ausentes toman un valor por defecto [7].
6. **Estandarización y alcance.** JSON es un estándar del IETF [27], mientras que Protocol Buffers no es un estándar formal de ningún organismo, lo que puede ser una limitación en entornos que exigen basarse en estándares. Además, Protocol Buffers supone que cada mensaje entra completo en memoria, por lo que no resulta adecuado para datos de más de unos pocos megabytes [7].

5.2.3. Ventajas y desventajas

La principal ventaja de Protocol Buffers frente a JSON es el tamaño de los mensajes, que en los estudios citados resulta de casi tres a más de diez veces menor. Esta diferencia es especialmente relevante en los sistemas considerados, que envían mensajes pequeños a alta frecuencia por enlaces limitados: de poco sirve que MQTT o CoAP reduzcan sus encabezados a unos pocos bytes si la carga útil se codifica en texto. A esto se suman el tipado de los campos y la evolución controlada del esquema. Como contrapartida, Protocol Buffers sacrifica la legibilidad, obliga a distribuir y mantener el esquema entre todos los participantes, lo que introduce un acoplamiento en el formato de los datos que el modelo publicador/suscriptor no elimina, y no es un estándar formal. JSON, por su parte, conviene cuando la legibilidad, la depuración o la integración con sistemas existentes, como las aplicaciones web, pesan más que el tamaño de los mensajes.

En síntesis, Protocol Buffers resulta preferible cuando el tamaño de los mensajes y el costo de transmitirlos son críticos y es posible distribuir un esquema común entre los participantes, mientras que JSON conviene cuando se priorizan la legibilidad, la flexibilidad y la interoperabilidad sin acuerdos previos. La Tabla 4 resume la comparación.

Tabla 4: Comparación entre JSON y Protocol Buffers.

Criterio	**JSON**	**Protocol Buffers**
Representación	Texto (UTF-8)	Binaria
Esquema	No requiere	Requiere un archivo `.proto` compartido
Autodescriptivo	Sí: incluye los nombres de los campos	No: incluye números de campo
Legibilidad	Legible por personas	Requiere herramientas y el esquema
Números	Texto decimal	Binarios, de longitud variable o fija
Tamaño de los mensajes	Mayor	De casi tres a más de diez veces menor
Tipado	Genérico	Un tipo concreto por campo
Evolución	Sin control	Controlada por números de campo y valores por defecto
Estandarización	Estándar del IETF	Sin estandarización formal

BORRADOR

Desarrollo

1. MQTT sobre TCP y TLS

1.1. Conexión persistente y keep-alive

1.2. Cifrado con TLS

1.3. QoS MQTT sobre la confiabilidad de TCP

2. Capa de red

2.1. NAT y firewalls

La mayoría de los dispositivos considerados en este trabajo no tiene una dirección IP pública propia. Un robot en un depósito, un sensor en una planta industrial o un teléfono en una red celular se encuentran detrás de un traductor de direcciones (NAT), que reemplaza la dirección y el puerto de origen de cada datagrama saliente por los de su interfaz pública y registra la correspondencia en una tabla, para poder traducir en sentido inverso las respuestas que llegan [3]. Ante el agotamiento de las direcciones IPv4, muchos proveedores de acceso agregan además su propio NAT de gran escala, de modo que el tráfico del dispositivo puede atravesar dos traducciones antes de llegar a Internet [30].

La tabla de traducción solo incorpora una entrada cuando un host interno inicia una conexión. Un segmento que llega desde Internet sin una entrada que le corresponda no tiene destinatario y se descarta, de manera que un dispositivo detrás de un NAT no puede actuar como servidor sin una configuración explícita en el router, como el reenvío de puertos. Los firewalls con estado producen el mismo efecto, por diseño admiten el tráfico de respuesta de las conexiones iniciadas desde la red interna y bloquean las conexiones entrantes no solicitadas [3]. En ambos casos, quien administra el broker no controla esos equipos, que pertenecen al cliente o a su proveedor de acceso.

El modelo de MQTT se ajusta a esta restricción, ya que, como se vio en la Sección 2, todos los participantes son clientes y es el dispositivo quien abre la conexión TCP hacia el broker. Esa conexión saliente crea la entrada en el NAT y en el firewall, y a partir de ella el broker puede entregarle mensajes al dispositivo, incluidos los comandos que se originan en la nube, como segmentos de una conexión ya establecida. El único requisito sobre la red del cliente es que permita las conexiones salientes hacia la dirección y el puerto del broker. IANA tiene registrados para MQTT los puertos TCP 1883, sin cifrado, y 8883, sobre TLS, y, cuando una red restrictiva solo deja pasar el tráfico web, MQTT puede transportarse sobre WebSocket [6], a cambio de la sobrecarga de ese protocolo adicional.

La contracara de este esquema es que las entradas de la tabla no son permanentes. El NAT no tiene forma de saber si una conexión inactiva sigue en uso, por lo que la elimina tras un tiempo sin tráfico. Para las conexiones TCP establecidas, el IETF (Internet Engineering Task Force), una organización internacional abierta de normalización que tiene como objetivos el contribuir a la ingeniería de internet, exige que ese tiempo no sea menor que dos horas y cuatro minutos [31]. En las mediciones sobre gateways domésticos que cita el RFC 8323, el promedio para TCP fue seis horas y veintiseis minutos [25], pero se trata de un mínimo recomendado y no de una garantía, para un equipo que no lo cumple, o un firewall intermedio configurado con un plazo menor, descarta la entrada mucho antes. Si la entrada desaparece mientras la

BORRADOR

conexión está inactiva, ninguno de los extremos se entera. El siguiente segmento del broker hacia el dispositivo se descarta en el NAT, y el siguiente del dispositivo genera una entrada nueva, con otro puerto público, que el broker no reconoce como parte de ninguna conexión. El resultado es una conexión que figura como abierta en ambos extremos pero que ya no transporta datos, situación especialmente grave para un suscriptor que espera comandos y no transmite por iniciativa propia.

El keep-alive de MQTT, definido en la Sección 2, resuelve también este problema. Al obligar al cliente a enviar algún paquete de control dentro de cada intervalo, aunque sea un PINGREQ, garantiza que la conexión nunca permanezca inactiva más que ese intervalo, y con ello refresca la entrada en cada NAT y firewall del trayecto, siempre que el intervalo sea menor que su tiempo de expiración. TCP ofrece un mecanismo análogo, pero es opcional y su intervalo predeterminado no debe ser menor que dos horas [32], un valor demasiado largo frente a los plazos de muchos equipos intermedios. Por eso el mantenimiento de la conexión se resuelve en la capa de aplicación, donde el intervalo es configurable por cada cliente. La especificación de MQTT indica que su valor depende de la aplicación y que suele ser de algunos minutos [6], y en los nodos del Experimento 1 (Sección 5.2) se usó un intervalo de 60 segundos. Elegirlo implica un compromiso, ya que un intervalo corto mantiene vivas las entradas y detecta antes una conexión rota, pero cada PINGREQ y su respuesta consumen ancho de banda y, en los dispositivos móviles, obligan a activar la radio, con el consiguiente gasto de energía.

2.2. Ruteo interno

Entre el dispositivo y el broker, cada datagrama atraviesa la red local del cliente, la de su proveedor de acceso, eventualmente las de uno o más proveedores de tránsito y la del proveedor de nube que aloja el broker, es decir, redes administradas por organizaciones distintas. Internet organiza esas redes en sistemas autónomos (AS), conjuntos de routers bajo una misma administración, y divide el problema del encaminamiento en dos niveles. Dentro de cada AS, un protocolo de ruteo interno (IGP) calcula los caminos entre sus routers, mientras que entre AS es BGP el que intercambia la información de alcanzabilidad [3]. Para MQTT, ambos niveles son transparentes mientras funcionan. Su comportamiento importa cuando un enlace o un router falla y la red debe encontrar un camino alternativo, porque durante ese lapso los segmentos de la conexión se pierden. Esta subsección y la siguiente describen cómo y en cuánto tiempo se recupera cada protocolo.

OSPF es un protocolo de estado de enlace. Cada router descubre a sus vecinos mediante paquetes Hello y describe sus enlaces, con el costo de cada uno, en anuncios de estado de enlace (LSA) que se inundan a todos los routers del área. Así, todos construyen la misma base de datos con la topología completa, y cada uno calcula, con el algoritmo de Dijkstra, el árbol de caminos más cortos con raíz en sí mismo [33]. Ante una falla, el router que la detecta inunda un LSA actualizado y cada router recalcula sus rutas de forma independiente, sin esperar a que lo hagan sus vecinos, lo que le da a OSPF una convergencia rápida. El factor que suele dominar es la detección. Si la falla deja la interfaz caída, el router la advierte de inmediato; si no, como cuando el enlace pasa por un switch intermedio, recién la advierte cuando deja de recibir paquetes Hello durante el RouterDeadInterval, que con los valores habituales de 10 segundos entre Hellos es de 40 segundos [33]. Mecanismos como BFD reducen esa detección a fracciones de segundo [34]. OSPF admite además dividir el AS en áreas, de modo que la inundación y el recálculo se limitan al área afectada [3].

BORRADOR

RIP, en cambio, es un protocolo de vector distancia basado en el algoritmo de Bellman-Ford. Cada router conoce solo las distancias, medidas en saltos, que le anuncian sus vecinos, y les envía su tabla completa cada 30 segundos [35]. La métrica admite como máximo 15 saltos, ya que el valor 16 representa un destino inalcanzable, lo que limita el tamaño de las redes en las que puede usarse. Su convergencia es lenta: una ruta se da por perdida recién después de 180 segundos sin anuncios que la confirmen, y la noticia de la falla avanza de a un router por intercambio. Además, como los vecinos pueden seguir anunciando caminos que pasaban por el enlace caído, la métrica de la ruta puede crecer de a poco hasta llegar a 16, el problema conocido como conteo hasta el infinito [3]. El horizonte dividido con reversa envenenada y las actualizaciones disparadas atenúan el problema, pero no lo eliminan en los bucles de tres o más routers [35].

EIGRP, desarrollado por Cisco y publicado como RFC informativo, es un protocolo de vector distancia avanzado que evita estas limitaciones mediante el algoritmo DUAL [36]. Usa una métrica compuesta, que por defecto combina ancho de banda y retardo, y envía actualizaciones solo cuando hay cambios. Para cada destino registra, además del sucesor (el vecino por el que pasa la mejor ruta), los sucesores factibles: vecinos cuya distancia anunciada es menor que la mejor distancia que el router conoce hacia ese destino, condición que garantiza que el camino a través de ellos no forma un bucle. Si el sucesor falla y existe un sucesor factible, el router conmuta a él de forma local e inmediata; solo si no lo hay pasa la ruta a estado activo y consulta a sus vecinos, lo que demora la convergencia [36]. Para detectar vecinos caídos, en los enlaces de alta velocidad envía Hellos cada 5 segundos con un tiempo de espera de 15 segundos [36]. La Tabla 5 resume las diferencias entre los tres protocolos.

Tabla 5: Protocolos de ruteo interno, con los temporizadores predeterminados de sus especificaciones [33], [35], [36].

	RIP	**OSPF**	**EIGRP**
Tipo	Vector distancia	Estado de enlace	Vector distancia avanzado
Algoritmo	Bellman-Ford	Dijkstra	DUAL
Métrica	Saltos (máximo 15)	Costo por enlace	Compuesta (ancho de banda y retardo)
Actualizaciones	Tabla completa cada 30 s	LSA ante cambios y refresco cada 30 min	Solo ante cambios
Detección de una falla no visible en la interfaz	180 s sin anuncios	40 s sin Hellos	15 s sin Hellos
Recuperación	Propagación de a un salto; posible conteo hasta el infinito	Recálculo independiente en cada router del área	Inmediata con sucesor factible; si no, consulta a los vecinos

Para una conexión MQTT, estas diferencias se traducen en el tiempo durante el cual sus segmentos se pierden: desde fracciones de segundo, cuando la falla se detecta en la interfaz o con BFD, hasta decenas de segundos con los temporizadores predeterminados de OSPF y EIGRP, y varios minutos en RIP. Qué ocurre con la conexión según la duración de ese lapso se analiza en la Sección 2.4.

BORRADOR

2.3. Ruteo externo

El broker no suele estar en el mismo AS que el dispositivo. Un robot conectado a la red de un cliente llega a Internet a través del AS de su proveedor de acceso, el broker se aloja en el AS de un proveedor de nube y entre ambos puede haber uno o más proveedores de tránsito. El camino entre AS lo determina BGP, el protocolo de ruteo externo de Internet [37].

BGP es un protocolo de vector de caminos. Los routers de borde de AS vecinos establecen sesiones eBGP sobre conexiones TCP y se anuncian prefijos acompañados de atributos, entre ellos el AS-PATH, la lista de AS que atravesó el anuncio, que permite descartar las rutas que contienen al propio AS y evitar así los bucles. Dentro de cada AS, las sesiones iBGP distribuyen esas rutas a los demás routers [3], [37]. A diferencia de un IGP, BGP no busca el camino más corto. Cada AS elige entre las rutas disponibles según sus políticas, que reflejan relaciones comerciales, como la de no ofrecer tránsito gratuito a otro proveedor, y solo después considera la longitud del AS-PATH y otros criterios de desempate [3]. En consecuencia, el trayecto entre un dispositivo y el broker no es necesariamente el más corto ni el de menor latencia, y puede cambiar por decisiones administrativas que no controlan ni el dispositivo ni el operador del broker.

La convergencia de BGP es mucho más lenta que la de un IGP. Un router detecta la caída de un vecino cuando expira el tiempo de espera de la sesión, para el que la especificación sugiere 90 segundos, con mensajes KEEPALIVE cada 30 segundos. Además, para limitar la cantidad de actualizaciones, un router espera un intervalo mínimo entre anuncios sucesivos hacia un mismo vecino, con un valor sugerido de 30 segundos entre vecinos externos [37]. Cuando un prefijo deja de ser alcanzable por un camino, los routers no descubren de inmediato cuál es la alternativa válida, sino que van probando caminos más largos que todavía figuran en sus tablas, cada uno demorado por ese intervalo. En mediciones sobre Internet, Labovitz et al. observaron que la conmutación a un camino alternativo entre dominios demoraba en promedio tres minutos, y que algunas recuperaciones provocaban oscilaciones de las tablas de ruteo de hasta 15 minutos [38].

Para el stack estudiado, esto tiene dos consecuencias. La primera es que una falla entre AS puede dejar al dispositivo sin camino hacia el broker durante un tiempo comparable o mayor que el intervalo de keep-alive de MQTT, lo que fuerza el cierre de la conexión y la reconexión del cliente. La segunda es que la mitigación está, en su mayor parte, fuera del alcance de quien diseña el sistema. El único factor que controla es la ubicación del broker: alojarlo en una región del proveedor de nube cercana a los dispositivos reduce la cantidad de AS del trayecto y, con ella, la exposición a la convergencia de BGP.

2.4. Convergencia y conexiones de larga duración

Las subsecciones anteriores describieron cuánto tarda la red en encontrar un camino alternativo después de una falla. Mientras tanto, el trayecto entre el dispositivo y el broker se comporta como un agujero negro: los routers que todavía apuntan al enlace caído descartan los segmentos, y durante la reconvergencia pueden formarse bucles transitorios, en los que los segmentos circulan hasta que se agota su TTL [39]. Ni TCP ni MQTT reciben aviso de lo que ocurre, porque el ruteo opera en la capa de red y las direcciones de los extremos no cambian. Lo único que observa el emisor es que sus segmentos dejan de ser confirmados. Lo que le pasa a la conexión depende, entonces, de cómo se compara la duración del corte con dos temporizadores de los extremos: el de retransmisión de TCP y el keep-alive de MQTT.

TCP reacciona a la falta de confirmaciones con su temporizador de retransmisión (RTO). Cuando este vence, retransmite el segmento más antiguo sin confirmar y duplica el RTO, de

BORRADOR

modo que los intentos sucesivos quedan cada vez más espaciados [40]. Con un RTO inicial de 1 s, los reintentos ocurren aproximadamente a 1, 3, 7, 15, 31 y 63 s del primer envío. Esto tiene una consecuencia poco intuitiva, ya que la conexión no se recupera en el instante en que la red converge, sino en el reintento siguiente. Si el camino se restablece a los 20 s, por ejemplo, el próximo intento sale recién a los 31 s, y la demora total resulta un 50 % mayor que el propio corte; en el peor caso, puede llegar casi al doble. Por otra parte, TCP tolera cortes largos antes de abandonar la conexión: la especificación exige que siga reintentando durante al menos 100 s [32], y con los valores predeterminados de Linux, el sistema operativo del servidor de los experimentos, el plazo es de unos 15 minutos [41].

En la práctica, quien suele cerrar primero la conexión es el keep-alive de MQTT. Si el broker no recibe ningún paquete de control del cliente durante una vez y media el intervalo de keep-alive, debe cerrar la conexión, y el cliente que no recibe respuesta a un PINGREQ en un tiempo razonable también debería cerrarla [6]. Con el intervalo de 60 s del Experimento 1, el broker da por caído a un cliente tras 90 s sin recibir nada de él. El umbral efectivo es incluso menor, porque durante el corte el PINGREQ queda detrás de los segmentos pendientes y depende de los mismos reintentos. Si los reintentos caen a los 63 y a los 127 s, un corte de algo más de un minuto ya alcanza para que el broker cierre la conexión, aunque la red haya convergido antes de los 90 s. Así, según la duración del corte se distinguen tres regímenes, que resume la Tabla 6.

Tabla 6: Efecto de un corte del camino sobre una conexión MQTT según su duración. Con el keep-alive de 60 s del Experimento 1, el umbral es de 90 s, o algo menos por el espaciamiento de los reintentos de TCP.

Duración del corte	**Ejemplos**	**Conexión**	**Efecto sobre los mensajes**
Unos pocos segundos	IGP con detección en la interfaz o con BFD; EIGRP con sucesor factible	Se mantiene; TCP retransmite una o dos veces	Llegan todos, con más latencia; QoS 1 puede duplicarlos
Menor que 1,5 veces el keep-alive	OSPF o EIGRP con detección por ausencia de Hellos	Se mantiene, con reintentos cada vez más espaciados	Llegan todos, juntos y con una demora del orden del corte
Mayor que 1,5 veces el keep-alive	RIP con detección por tiempo; BGP ante el retiro de una ruta	El broker la cierra y publica la última voluntad; el cliente se reconecta	Se pierden los de QoS 0 en tránsito; los de QoS 1 y 2 se reenvían si se retoma la sesión

Cortes de hasta unos pocos segundos. Son los que produce un IGP cuando la falla se detecta en la interfaz o mediante BFD, o EIGRP cuando cuenta con un sucesor factible. TCP los absorbe con uno o dos reintentos y la única consecuencia es una demora adicional del orden del RTO. Es el único régimen que cubrió el Experimento 1, y de forma indirecta, ya que inyectó pérdidas aleatorias e independientes y no cortes. Con 5 % de pérdida, la entrega fue del 100 % en todos los transportes y no hubo desconexiones de MQTT, pero el percentil 99 de la latencia de MQTT QoS 0 subió a unos 3 s, por pérdidas repetidas del mismo segmento con el RTO duplicado en cada intento (Tabla 14). En este régimen, QoS 1 no mejora la entrega y agrega duplicados, porque el cliente reenvía el PUBLISH mientras TCP todavía está recuperando el original, como se discute en la Sección 1.3.

Cortes menores que una vez y media el keep-alive. Corresponden, por ejemplo, a una falla que OSPF o EIGRP detectan por la ausencia de Hellos con sus temporizadores predeterminados (Tabla 5). La conexión sobrevive, pero las lecturas generadas durante el corte se acumulan sin

BORRADOR

confirmar en el dispositivo y llegan juntas cuando el camino se restablece, con una demora del orden del corte más la espera hasta el reintento siguiente. Ningún mensaje se pierde, cualquiera sea el nivel de QoS, pero su valor puede haber caducado: una lectura de telemetría que llega con decenas de segundos de atraso, o un comando que el robot recibe tarde, pueden resultar tan inútiles como uno perdido. La memoria del dispositivo impone además un límite, ya que un microcontrolador no puede acumular indefinidamente las lecturas pendientes.

Cortes mayores que una vez y media el keep-alive. Son los de RIP ante una falla que no se ve en la interfaz, o los de BGP cuando se retira una ruta, que suelen durar minutos (Sección 2.3). El broker cierra la conexión y, como el cierre es anormal, publica el mensaje de última voluntad del cliente, con lo que los demás participantes se enteran de la caída (Sección 2). Cuando el camino se restablece, el siguiente segmento del dispositivo llega a un broker que ya no reconoce la conexión y que responde con un *reset*, de modo que el cliente también la da por cerrada y se reconecta. La reconexión tiene un costo: un nuevo handshake TCP y TLS, que en el Experimento 1 constituye el costo fijo por conexión (Tabla 19) y que al ESP32 le insume unos 3 s de cómputo. Lo que se pierde depende del nivel de QoS y de la sesión. Los mensajes de QoS 0 que estaban en tránsito se pierden sin aviso. Los de QoS 1 y 2 que no habían sido confirmados se reenvían al reconectarse, siempre que el cliente retome la sesión anterior y el broker la haya conservado [6], y en ese caso el broker conserva también las suscripciones y los mensajes que llegaron para el cliente durante el corte. Es en este régimen donde los niveles de QoS y las sesiones persistentes aportan lo que TCP no puede dar, porque la confiabilidad de TCP termina con la conexión.

Hay un caso en el que la duración del corte deja de importar: cuando el camino nuevo atraviesa un NAT distinto del original, como cuando un proveedor conmuta a otro equipo de NAT de gran escala, o cuando el dispositivo cambia de red, por ejemplo de Wi-Fi a la red celular. La traducción de la conexión no existe en el nuevo NAT (Sección 2.1), por lo que la conexión se pierde aunque la red converja de inmediato, con las mismas consecuencias que en el tercer régimen.

El Experimento 1 no incluyó cortes de conexión, por lo que los dos últimos regímenes no se midieron (Sección 5.2). Un ensayo que los cubra podría reutilizar la misma plataforma: aplicar con `tc netem` una pérdida del 100 % sobre la interfaz de un nodo durante un intervalo fijo, con duraciones por debajo y por encima del umbral del keep-alive, y medir para cada nivel de QoS, con sesión persistente y sin ella, las lecturas perdidas, duplicadas y demoradas, y el tiempo hasta que el nodo vuelve a publicar.

3. Ruteo por tópicos y ruteo IP

4. Casos de uso

Las secciones anteriores analizaron la combinación de MQTT y Protocol Buffers a partir de las capas y protocolos que atraviesa un mensaje. Esta sección examina cómo la emplean sistemas reales de distintos dominios, con el fin de identificar qué propiedades de cada tecnología aprovechan y qué ventajas y limitaciones aparecen en la práctica. En cada caso se consideran, en la medida en que las fuentes lo permiten, el problema de comunicación que enfrenta, la combinación de protocolo y formato que adopta, los mecanismos de MQTT que utiliza, los aspectos de red involucrados y sus limitaciones. Los mecanismos analizados, como los niveles de QoS o el mensaje de última voluntad, existen tanto en la versión 3.1.1 de MQTT como en

BORRADOR

la 5.0, por lo que la versión solo se aclara cuando un caso depende de una función que agregó la 5.0.

No todos los casos combinan MQTT y Protocol Buffers. Algunos emplean MQTT con otro formato, y su inclusión permite distinguir el aporte de cada pieza, ya que, como se planteó en la Sección 5, el protocolo de aplicación y el mecanismo de serialización constituyen decisiones independientes. Esa independencia se observa también en sistemas que no se analizan aquí. Zigbee2MQTT, por ejemplo, un programa que conecta a MQTT lámparas, sensores y otros dispositivos domésticos inalámbricos, utiliza MQTT con JSON [42], mientras que la aplicación de mensajería Signal utiliza Protocol Buffers sobre WebSocket, sin MQTT [43].

4.1. Mensajería instantánea

La mensajería en teléfonos móviles ofrece uno de los ejemplos más conocidos de uso masivo de MQTT. Al desarrollar su aplicación de mensajería Messenger, Facebook (hoy Meta) necesitaba entregar los mensajes a los teléfonos con baja latencia y sin agotar su batería, y recurrió a MQTT por tratarse de un protocolo diseñado para hacer un uso moderado del ancho de banda y de la energía. Sobre él construyó un mecanismo que mantiene una conexión persistente entre cada teléfono y los servidores de la empresa, con el que logró entregar en muchos casos los mensajes de un teléfono a otro en cientos de milisegundos en lugar de varios segundos [44]. Se trata del esquema planteado en la Sección 1, en el que el teléfono abre la conexión hacia el servidor y recibe por ella los datos, sin aceptar conexiones entrantes ni consultar periódicamente. La documentación pública no detalla la estructura de tópicos ni los niveles de QoS empleados, por lo que el análisis de este caso se limita al modelo de conexión, al formato de los mensajes y a los límites que la propia empresa encontró en el protocolo.

La evolución posterior de Messenger muestra cuánto influye el formato de los mensajes sobre esa conexión persistente. En su diseño original, la aplicación recibía primero un aviso breve de que había datos nuevos, y luego debía pedirlos mediante una consulta HTTPS, es decir, HTTP protegido con TLS, cuya respuesta en JSON era de gran tamaño. Cada mensaje nuevo exigía así un intercambio adicional y una transferencia voluminosa. Una reforma de 2014 cambió ese esquema. La aplicación pasó a obtener el estado completo de las conversaciones una sola vez, al iniciarse, y a recibir después por MQTT solo los cambios, a medida que se producían. Esa misma reforma reemplazó JSON por Thrift, una herramienta desarrollada originalmente en Facebook que, de forma análoga a Protocol Buffers, define los mensajes mediante un esquema y los codifica por lo general en formato binario [45]. El cambio de formato redujo aproximadamente a la mitad el tamaño de los mensajes [46]. Aunque el formato no es Protocol Buffers, la reducción es coherente con la ventaja de un formato con esquema frente a JSON descrita en la Sección 5.2.

Un artículo que la empresa presentó en 2021 muestra, en cambio, un límite de MQTT a gran escala. Allí se describe el sistema que mantiene actualizada la información que el usuario tiene en pantalla, como los comentarios nuevos en una entrada, los contactos que están en línea o el aviso de que alguien está escribiendo. Cada uno de esos elementos constituye un flujo de datos, es decir, una secuencia de actualizaciones que el teléfono recibe mientras esa información está a la vista, y un mismo teléfono puede tener muchos flujos abiertos a la vez. En MQTT, todos ellos pertenecerían a una única sesión, atendida por un único servidor. Si ese servidor se sobrecarga o falla, no es posible trasladar solo algunos flujos a otro servidor, sino que debe trasladarse la sesión completa. Por eso la empresa diseñó un protocolo propio que viaja por la misma conexión del teléfono, pero permite que cada flujo sea atendido por un servidor distinto y se recupere por separado [47].

BORRADOR

4.2. Robótica

En la mensajería, la mayor parte del tráfico va del servidor al teléfono. En la robótica móvil, en cambio, el tráfico es intenso en ambos sentidos. Los robots móviles autónomos (AMR) y guiados (AGV) que se emplean en la industria y la logística, pero también en otras tareas como la limpieza, trabajan en flotas, es decir, en conjuntos de robots que comparten una misma instalación, y deben enviar de forma continua su posición y su estado, y al mismo tiempo recibir órdenes. Además, operan en depósitos o fábricas cuya red administra la empresa que los utiliza, y no siempre quien provee el software que los supervisa, por lo que suelen quedar detrás de NAT y firewalls que no aceptan conexiones entrantes. En este dominio se analizan tanto un servicio comercial que combina MQTT con Protocol Buffers como estándares que adoptan MQTT con JSON para que robots y sistemas de distintos fabricantes puedan comunicarse entre sí.

4.2.1. InOrbit y OpenRobOps

InOrbit es una empresa que ofrece una plataforma primariamente basada en la nube para supervisar y operar flotas heterogéneas de robots móvils y dispositivos de distintos fabricantes. La empresa publica como código abierto un kit de desarrollo o SDK que desarrolladores pueden utilizar para incorporar dispositivos con la plataforma.

Con el SDK, cada robot abre su propia conexión MQTT 3.1.1 hacia el broker de la plataforma, cifrada con TLS. Antes de conectarse, obtiene la dirección del broker y el usuario y la contraseña con los que debe identificarse mediante una consulta HTTPS, de modo que HTTP se usa una sola vez, al iniciar, y MQTT para la conexión que permanece abierta. Cada tópico indica el robot y el tipo de dato que transporta, y por la misma conexión viajan en un sentido los datos del robot y en el otro las órdenes de la plataforma. La mayor parte de los datos se codifica con Protocol Buffers, con un tipo de mensaje para cada clase de dato, como la posición, el mapa del lugar o las imágenes de una cámara. A la reducción de tamaño que aporta la codificación se suman decisiones del propio SDK, como limitar la frecuencia con que se publican ciertos datos o eliminar puntos intermedios de los recorridos antes de enviarlos [17].

Los mecanismos de MQTT se eligen según la naturaleza de cada dato. La posición y los demás datos que cambian continuamente se publican con QoS 0, ya que si una muestra se pierde, la siguiente la reemplaza. El mapa, por el contrario, cambia rara vez, y como su pérdida dejaría a la plataforma sin él durante mucho tiempo, se publica con QoS 1 y como mensaje retenido, de modo que quien empieza a recibir datos del robot lo obtiene de inmediato. Algo similar ocurre con el estado de conexión. Al conectarse, el robot publica que está en línea mediante un mensaje retenido y registra en el mismo tópico un mensaje de última voluntad, también retenido. Si la conexión se interrumpe, el broker publica la última voluntad, que reemplaza al mensaje anterior, y quien se suscribe después recibe así el estado vigente.

InOrbit publicó además, bajo el nombre OpenRobOps, una versión de código abierto de la plataforma, que permite examinar también el lado del servidor, incluida la definición de los mensajes en archivos `.proto`. Como broker utiliza Mosquitto, un broker MQTT de código abierto, configurado para que cada robot tenga sus propias credenciales y solo pueda acceder a sus tópicos, de modo que un robot comprometido no pueda leer los datos de otros ni hacerse pasar por ellos. Su esquema ilustra la evolución descrita en la Sección 3.3, ya que los campos que dejan de usarse permanecen con una marca que indica que no deben emplearse, y la información nueva se agrega con números de campo no utilizados, de modo que los programas antiguos siguen siendo comprendidos [1].

BORRADOR

Tanto el SDK de InOrbit como OpenRobOps utilizan MQTT 3.1.1, y esa elección explica parte de sus limitaciones. Esta versión no permite indicar en el mensaje qué formato tiene su contenido, por lo que el receptor deduce el tipo de cada mensaje a partir del tópico en el que llega. Esa dependencia es frágil, porque junto con los mensajes de Protocol Buffers conviven formatos más antiguos, como texto plano o documentos JSON guardados dentro de un campo de texto de un mensaje de Protocol Buffers, y como ningún mensaje declara su formato, cada programa que los recibe debe saber de antemano qué formato corresponde a cada tópico. La versión 3.1.1 tampoco ofrece una forma de asociar una respuesta con la solicitud que la originó. Para confirmar que una orden llegó, el robot devuelve una copia de cada mensaje que recibe, y la plataforma identifica a qué orden corresponde mediante un número que la propia orden incluía. La versión 5.0 resuelve ambas necesidades con propiedades del protocolo. Una indica el tipo de contenido del mensaje, y otras dos permiten que quien solicita indique en qué tópico espera la respuesta y agregue un identificador que quien responde devuelve sin cambios, de modo que el solicitante puede asociar cada respuesta con su solicitud [6].

Otras limitaciones provienen de la arquitectura y no de la versión del protocolo. Abrir una conexión por robot, incluso cuando un mismo programa representa a varios, permite que el broker limite cada conexión a los tópicos de su robot y que el mensaje de última voluntad, que se registra una vez por conexión, anuncie la desconexión de un robot concreto. A cambio, cada robot exige su propia negociación TLS y sus propios paquetes de keep-alive, un costo que se multiplica cuando un solo equipo representa a muchos robots. Además, OpenRobOps ejecuta un único broker, ya que Mosquitto no permite que varias copias del broker funcionen como un solo sistema, repartiéndose los robots o reemplazando a una copia que falla, porque cada una guarda por separado sus mensajes retenidos y sus sesiones. Si ese broker se detiene, todos los robots quedan incomunicados, que es el punto de falla que la Sección 1 atribuye a la arquitectura centralizada.

4.2.2. VDA 5050 e ISO 21423

Sin un estándar común, cada fabricante de robots ofrece su propio sistema de control de flota, es decir, el software que asigna tareas a los robots y coordina sus movimientos. Para operar en conjunto robots de distintas marcas, una planta depende entonces de una plataforma como las de la Sección 4.2.1, que debe adaptarse a la interfaz particular de cada fabricante. VDA 5050 es una especificación, desarrollada por las asociaciones alemanas de la industria automotriz (VDA) y de fabricantes de maquinaria (VDMA), que define cómo se comunican los robots con un sistema de control de flota de cualquier fabricante. A diferencia del protocolo de la Sección 4.2.1, que una sola empresa define para su propia plataforma, se trata de un estándar abierto, que adopta MQTT con JSON, y que también es adoptado por InOrbit y OpenRobOps.

La especificación exige como mínimo la versión 3.1.1 de MQTT y elige JSON porque permite agregar parámetros nuevos sin afectar a los equipos existentes y porque cada mensaje puede verificarse contra un esquema JSON, un documento aparte que describe los campos y los tipos que debe contener. A diferencia de lo que ocurre con Protocol Buffers, ese esquema no hace falta para leer el mensaje, sino solo para comprobar que es válido, de modo que cualquier fabricante puede interpretar los mensajes sin disponer de él. Los tópicos indican el nombre del protocolo, su versión, el fabricante, el número de serie del robot y el tipo de mensaje [48]. Como la versión forma parte del tópico, robots que usan versiones incompatibles pueden convivir en un mismo broker, ya que un sistema suscrito a los tópicos de una versión nunca recibe los mensajes de otra. Es una estrategia de evolución distinta de la de Protocol Buffers, que mantiene la

BORRADOR

compatibilidad dentro de un mismo esquema mediante los números de campo, como se describió en la Sección 3.3.

Con el objetivo declarado de reducir la sobrecarga de comunicación, la especificación establece QoS 0 para todos los mensajes, salvo los del tópico que informa si el robot está conectado, que usan QoS 1 y se publican como retenidos. En ese tópico, el robot registra además un mensaje de última voluntad que avisa la pérdida de la conexión. Sin embargo, la reducción de sobrecarga que se busca con el nivel de QoS no se extiende al contenido de los mensajes. La confirmación de QoS 1 agrega apenas unos pocos bytes, mientras que el mensaje con el que el robot informa periódicamente su posición, su batería y la tarea en curso repite en JSON, en cada envío, los nombres de todos sus campos, que es precisamente el costo que Protocol Buffers elimina, como se mostró en la Sección 5.2.

La norma ISO 21423, publicada en 2026, sigue un enfoque análogo. Especifica la comunicación entre robots móviles industriales, sistemas de control de flota y otros equipos del edificio que los robots necesitan accionar para desplazarse, como puertas automáticas o ascensores [49], y adopta también MQTT y JSON [50]. Las dos plataformas de la Sección 4.2.1 admiten estos estándares, mostrando que una misma plataforma puede ofrecer los dos formatos sobre MQTT, con Protocol Buffers en su protocolo propio y JSON en los estándares que comparte con otros fabricantes.

4.3. Internet de las Cosas industrial

En una planta industrial, donde máquinas y sensores conectados a la red envían datos de forma continua, un sistema de supervisión debe conocer en todo momento el estado de numerosos equipos que a menudo provienen de distintos fabricantes. Al igual que VDA 5050 e ISO 21423 para los robots móviles, este ámbito cuenta con un estándar abierto de interoperabilidad sobre MQTT, que cumple una función similar para los equipos de planta. A diferencia de aquellos, sin embargo, este estándar adopta Protocol Buffers en lugar de JSON. Eclipse Sparkplug, desarrollada por la Fundación Eclipse, una organización sin fines de lucro que mantiene proyectos de software abierto, define cómo se comunican sobre MQTT tres tipos de participantes. Los dispositivos son los equipos que generan los datos, como sensores, motores o las computadoras industriales que comandan las máquinas. Los nodos del *edge* o de borde, llamados así porque se ubican en el límite de la red, junto a las máquinas, reúnen los datos de esos dispositivos, que a menudo no pueden comunicarse por MQTT, y los publican en el broker. Las aplicaciones de supervisión, por último, se suscriben a esos datos para mostrarlos, almacenarlos o enviar órdenes. La especificación completa lo que MQTT deja abierto de manera deliberada, como la estructura de los tópicos, el formato de los mensajes y la forma de conocer qué equipos están en línea [51]. Su versión 3.0 fue adoptada como norma ISO/IEC 20237:2023 [15].

Cada tópico indica la versión de la especificación, un grupo de nodos, el tipo de mensaje, el nodo y el dispositivo. El tipo distingue los datos, las órdenes y los mensajes que anuncian la conexión y la desconexión. El contenido de los mensajes se codifica con Protocol Buffers, mediante un esquema común que representa los datos como una lista de mediciones, llamadas métricas, cada una con su tipo y su momento de registro. Como ese esquema es genérico y sirve para cualquier equipo, el nombre de cada métrica viaja como texto, lo que reintroduce dentro del mensaje binario el costo que Protocol Buffers evita al identificar los campos por número. Para reducirlo, la especificación permite identificar cada métrica con un número corto, llamado alias. El nodo informa la correspondencia entre nombres y alias una sola vez, en el mensaje que publica al conectarse y que se describe a continuación, y los mensajes de datos posteriores llevan solo el

BORRADOR

número. Con el mismo propósito, recomienda publicar un valor solo cuando cambia, y no de forma periódica.

El aspecto más elaborado de Sparkplug es la forma en que la aplicación de supervisión sabe qué nodos están en línea. Para ello se apoya en el mensaje de última voluntad y el keep-alive descritos en la Sección 2. Al conectarse, cada nodo publica un mensaje que anuncia su presencia y enumera todas las métricas que reportará, que la especificación llama certificado de nacimiento y que no tiene relación con los certificados digitales de TLS. Antes de publicarlo, registra como mensaje de última voluntad un certificado de muerte, que el broker publica si la conexión se interrumpe sin aviso. Al recibirlo, la aplicación de supervisión marca como desactualizados los datos de ese nodo, para que nadie tome los últimos valores recibidos como vigentes. La rapidez con que se detecta la caída depende, por lo tanto, del intervalo de keep-alive analizado en la Sección 1.1.

Este esquema presenta, sin embargo, un problema con el orden de llegada de los certificados. Como el broker solo detecta la caída de una conexión cuando vence el keep-alive, el nodo puede volver a conectarse antes de que eso ocurra. En ese caso, el certificado de muerte de la conexión anterior puede publicarse después del nuevo certificado de nacimiento, y la aplicación daría por desconectado a un nodo que está en línea. Para evitarlo, ambos certificados llevan un número que cambia en cada conexión, y la aplicación solo considera caído al nodo cuando el certificado de muerte corresponde al último nacimiento recibido.

Un problema distinto es la pérdida de datos. Como se publican con QoS 0, los mensajes que están en tránsito cuando se corta una conexión se pierden, aunque TCP sea confiable mientras la conexión existe. Para detectarlo, cada mensaje lleva un número de secuencia, y cuando la aplicación advierte que falta uno, le pide al nodo un nuevo certificado de nacimiento para reconstruir su estado completo. Se trata de un mecanismo de la aplicación que suple la confirmación que QoS 0 no ofrece, en línea con lo analizado en la Sección 1.3.

Las aplicaciones de supervisión anuncian también su propio estado, con un mensaje retenido y un mensaje de última voluntad. Así, los nodos saben si hay alguien recibiendo sus datos y pueden esperar a que la aplicación designada como principal, encargada de registrarlos, esté en línea antes de publicar, para no enviar datos que nadie almacenaría. La especificación exige además que cada conexión comience sin sesión persistente, de modo que el broker no acumula mensajes viejos para los clientes desconectados. Quien se desconecta pierde los datos publicados mientras tanto, pero recupera el estado actual, porque cada nodo vuelve a publicar su estado completo al reconectarse. Cuando se emplea la versión 5.0, este comportamiento se fija con propiedades exclusivas de esa versión, como la duración de la sesión una vez cerrada la conexión.

Estas decisiones tienen también costos. La especificación prohíbe publicar el certificado de nacimiento como mensaje retenido, por lo que una aplicación que se suscribe más tarde debe pedirle al nodo uno nuevo antes de poder interpretar los mensajes que usan alias, salvo que el broker implemente una extensión opcional que lo conserve. Además, como las métricas viajan dentro del contenido de los mensajes, el broker solo puede distribuirlos según el nodo o el dispositivo que los origina. Una aplicación interesada en una única métrica debe recibir y decodificar todos los mensajes del nodo, una limitación del ruteo por tópicos ya señalada en la Sección 3.

4.4. Internet de las Cosas sobre redes LoRa

Los casos anteriores suponen dispositivos conectados directamente a una red IP. LoRa es, en cambio, una tecnología de radio que permite comunicar dispositivos a varios kilómetros de

BORRADOR

distancia con muy bajo consumo de energía, a costa de una velocidad de transmisión muy reducida, y cuyos enlaces no transportan IP. Por eso, en las redes LoRa no puede emplearse MQTT directamente. Su interés para este trabajo radica en dos aspectos. Por un lado, muestran a Protocol Buffers en el enlace más restringido de todos los casos analizados. Por otro, muestran el papel de MQTT en el punto en que los datos de la radio ingresan a una red IP, donde un equipo intermediario debe hacerlos llegar a un servidor. Los dos proyectos que se describen a continuación, Meshtastic y ChirpStack, ilustran respectivamente el primer y el segundo aspecto.

4.4.1. Meshtastic

Meshtastic es un software de código abierto que se instala en pequeñas radios LoRa de bajo costo y que, junto con una aplicación para teléfonos, permite intercambiar mensajes de texto y datos de posición sin depender de redes celulares ni de Internet. Las radios forman una red mallada, en la que cada una retransmite los mensajes que recibe para que lleguen a radios fuera del alcance del emisor. Por radio, cada mensaje viaja con un encabezado binario fijo y un contenido codificado con Protocol Buffers, que no puede superar los 239 bytes [52], un margen en el que un formato textual como JSON resultaría difícil de sostener.

Una radio que además dispone de conexión a Internet puede actuar como puente entre la malla y un broker MQTT. Publica en el broker los mensajes que recibe de la malla y se suscribe a él para retransmitir en su malla los mensajes que otros puentes publican desde mallas lejanas, de modo que el broker une mallas que no tienen alcance de radio entre sí. Los usuarios de Meshtastic pueden organizar sus conversaciones en grupos, que el proyecto llama canales, y cada mensaje se publica encapsulado en otro mensaje de Protocol Buffers que agrega su canal y el identificador del puente [53]. Como los mensajes de la malla ya están codificados con Protocol Buffers, el puente los reenvía sin traducirlos [14].

Los mensajes de cada canal se cifran con una clave que comparten las radios de ese canal, de modo que solo ellas pueden leerlos. Al publicarlos en el broker, el puente puede conservar ese cifrado o enviarlos descifrados, y la configuración por defecto opta por lo segundo [54]. La diferencia es relevante porque el broker, como intermediario del modelo publicador/suscriptor, recibe el contenido de todos los mensajes. TLS, que también es opcional en la conexión con el broker, protege solo el tramo entre el puente y el broker, por lo que el broker y sus suscriptores siguen pudiendo leer los mensajes. Si el mensaje conserva el cifrado del canal, en cambio, solo pueden leerlo quienes tienen la clave.

La presencia de varios puentes en una misma malla muestra la diferencia entre dos formas de hacer llegar los mensajes, que se vinculan con la comparación entre ruteo por tópicos y ruteo IP de la Sección 3. La malla difunde cada mensaje a todas sus radios, de modo que varios puentes pueden recibirlo y publicarlo. El broker, que entrega cada mensaje según su tópico sin interpretar su contenido, reenvía cada copia a todos los suscriptores, y son estos quienes deben descartar los duplicados a partir del número que la radio de origen asigna a cada mensaje. En el sentido inverso aparece otro problema. Los desarrolladores de Meshtastic mantienen un broker público, al que las radios se conectan por defecto y en el que confluye el tráfico de muchas mallas. Si ese tráfico se retransmitiera por cada malla, saturaría enlaces de radio de muy baja capacidad, por lo que los mensajes que llegan desde ese broker se entregan solo a las radios al alcance directo del puente.

4.4.2. ChirpStack

LoRaWAN es un protocolo que organiza las redes LoRa de manera distinta a Meshtastic. En lugar de una malla, los dispositivos transmiten por radio a uno o más equipos fijos, llamados

BORRADOR

gateways, que reenvían por Internet lo que reciben a un servidor central, llamado servidor de red, que verifica cada mensaje, descarta las copias repetidas y lo entrega al programa del usuario que utiliza esos datos [55]. ChirpStack es una implementación de código abierto de ese servidor, y permite seguir un dato desde la radio hasta ese programa a través de tramos con protocolos y formatos distintos.

Los *gateways* suelen reenviar los datos mediante un protocolo sobre UDP, definido por Semtech, el fabricante de los circuitos de radio LoRa, que representa la información en JSON. Según su propia especificación, se trata de un protocolo deliberadamente básico, que no verifica la identidad de los extremos ni retransmite los datagramas perdidos y que, por eso, está pensado para demostraciones o para redes privadas y confiables. El protocolo contempla además que el *gateway* se encuentre detrás de un NAT, por lo que el *gateway* envía periódicamente un mensaje para que el NAT mantenga abierta la asociación por la que le llegan los datos que el servidor envía hacia los dispositivos [56]. El mantenimiento de esas asociaciones se analizó en la Sección 2.1.

ChirpStack incluye un componente, llamado Gateway Bridge, que traduce ese protocolo a MQTT, con Protocol Buffers como codificación recomendada y JSON reservado para la depuración [57]. La documentación recomienda ejecutarlo en el propio *gateway*, de modo que el tramo que atraviesa Internet aproveche la autenticación y el cifrado TLS de la conexión MQTT [58]. Se reemplaza así un protocolo sin confiabilidad ni seguridad por una conexión TCP persistente lo más cerca posible del origen de los datos. Los mensajes que el *gateway* publica incluyen, como campos de Protocol Buffers, información sobre cómo se recibió cada transmisión, como la frecuencia y la intensidad y la calidad de la señal [59], valores que en JSON repetirían sus nombres en cada mensaje.

En cuanto a los mecanismos de MQTT, el Gateway Bridge publica el estado de conexión del *gateway* como mensaje retenido y, cuando se ejecuta en el propio *gateway*, registra además un mensaje de última voluntad que avisa su desconexión [60]. Si se ejecutara en un servidor, su conexión MQTT seguiría abierta aunque el *gateway* dejara de funcionar, y la última voluntad no reflejaría esa caída. El servidor de red, por su parte, emplea la versión 5.0 y, si el broker lo admite, se suscribe a los mensajes de todos los *gateways* mediante una suscripción compartida, que reparte los mensajes entre varias copias del servidor de red que se ejecutan en paralelo [61].

Como en Meshtastic, una misma transmisión de radio puede ser recibida por varios *gateways*, pero en LoRaWAN es el servidor de red el que elimina los duplicados, para lo cual espera un breve intervalo a que lleguen todas las copias. Esa espera, sumada a la latencia del tramo MQTT, demora la respuesta que el servidor envía al dispositivo. Como el dispositivo, para ahorrar energía, solo enciende su receptor durante un breve lapso después de cada transmisión, una respuesta que llega tarde se pierde.

Hacia el programa del usuario, ChirpStack publica los datos en JSON por defecto, con Protocol Buffers como opción [62]. El formato se elige así según el tramo, con la codificación binaria donde se concentra el volumen de mensajes y JSON donde prima la facilidad de integración con otros sistemas.

4.5. Ventajas y limitaciones observadas

Los casos analizados, pese a pertenecer a dominios distintos, comparten un mismo patrón de red. En todos ellos el dispositivo, o el equipo intermediario que lo representa, abre una conexión persistente hacia un broker cuya dirección conoce u obtiene al iniciar, y recibe por ella los mensajes que le están destinados, entre ellos las órdenes. No necesita, por lo tanto, aceptar

BORRADOR

conexiones entrantes, lo que le permite operar detrás de NAT y firewalls, como se analizó en la Sección 2.1. La mayoría informa además si está en línea mediante los mecanismos de MQTT, aunque con variantes. InOrbit, VDA 5050 y ChirpStack combinan un mensaje retenido con el mensaje de última voluntad, de modo que cualquier suscriptor conoce el estado vigente. Sparkplug aplica ese patrón solo a las aplicaciones de supervisión, mientras que los nodos vuelven a anunciar su estado con un nuevo certificado de nacimiento al reconectarse.

La elección del formato sigue también un criterio común. Protocol Buffers aparece en los tramos que concentran el volumen de mensajes, como los datos de los robots, los mensajes de los *gateways* LoRaWAN o las mediciones de los equipos industriales, y en los enlaces de radio más restringidos. JSON, por el contrario, aparece en los datos que se entregan a programas de otras empresas y en estándares compartidos entre fabricantes, como VDA 5050 o ISO 21423, en los que importa que cualquiera pueda leer los mensajes sin disponer de un esquema, aunque Sparkplug muestra que un estándar abierto también puede adoptar Protocol Buffers cuando el volumen de datos lo justifica.

En los datos que se publican con alta frecuencia predomina QoS 0, ya que cada muestra queda reemplazada por la siguiente. Los casos se apoyan en la entrega fiable de TCP y, cuando la necesitan, agregan mecanismos propios, como los números de secuencia de Sparkplug. QoS 1 y los mensajes retenidos quedan reservados para el estado de conexión y para los datos que cambian con poca frecuencia, como el mapa de un robot. La reducción del volumen tampoco depende solo de la codificación, ya que Protocol Buffers se combina con decisiones de la aplicación, como limitar la frecuencia de publicación o publicar solo los cambios. Esas optimizaciones pueden tener, sin embargo, un costo, como muestra Sparkplug, donde una aplicación que se suscribe tarde debe pedir un nuevo certificado de nacimiento antes de poder interpretar los datos que usan alias. En cuanto a la magnitud del beneficio, la única medición realizada sobre un sistema en funcionamiento real proviene de Messenger, que redujo a la mitad el tamaño de los mensajes al reemplazar JSON por Thrift, un formato con esquema análogo a Protocol Buffers.

Los casos también exponen los límites del stack. Del lado de MQTT, el broker conserva estado, y cuando la implementación no permite repartirlo entre múltiples réplicas, como ocurre con Mosquitto, se convierte en el punto de falla de todo el sistema. A gran escala, la sesión resulta una unidad demasiado grande para repartir la carga entre servidores y recuperarse ante fallas, y cuando cada dispositivo mantiene su propia conexión, cada uno asume el costo de su negociación TLS y de su keep-alive. El ruteo por tópicos, por su parte, no permite filtrar según el contenido de los mensajes. Del lado de Protocol Buffers, todo programa que recibe los datos debe disponer del esquema, y la convivencia con formatos más antiguos debilita el acuerdo que establece el esquema.

Por último, varias implementaciones permanecen en MQTT 3.1.1 y resuelven en la aplicación o mediante el tópico funciones que la versión 5.0 incorpora, como indicar el formato de los mensajes o asociar cada respuesta con su solicitud, según se vio en la Sección 4.2. Entre los casos analizados, el servidor de red de ChirpStack es el que más aprovecha las funciones exclusivas de esa versión, mientras que Sparkplug solo las emplea para fijar el comportamiento de la sesión. Ninguno de los casos cuya implementación pudo examinarse utiliza las propiedades que indican el formato o asocian respuestas con solicitudes, lo que sugiere que, al menos en la muestra analizada, las ventajas de la versión 5.0 todavía no se reflejan plenamente en la práctica. En conjunto, los casos indican que el stack aporta sus mayores ventajas en los tramos de alta

BORRADOR

frecuencia y en los dispositivos que operan detrás de redes ajenas, mientras que sus limitaciones se concentran en el broker y en la distribución del esquema.

5. Diseño experimental

5.1. Plataforma común

5.2. Experimento 1: MQTT contra HTTPS para telemetría

Este experimento es el caso base de la parte experimental. Mide cuánto cuesta, en bytes y en latencia, enviar una lectura de un sensor por MQTT y por HTTP, y separa ese costo en sus componentes: la conexión, el protocolo de aplicación, el cifrado y la serialización. Sus resultados sirven de referencia para el segundo experimento (Sección 5.3), que compara las mismas combinaciones en la sincronización de osciladores acoplados, una aplicación de tiempo real con intercambio de mensajes a alta frecuencia.

5.2.1. Conceptos previos

Además de lo presentado en el marco teórico, el análisis de este experimento se apoya en dos mecanismos de TCP y en el protocolo con el que se sincronizan los relojes de los nodos. Se resumen a continuación porque explican buena parte de los resultados.

Algoritmo de Nagle. Una aplicación entrega datos a TCP con llamadas a `send()` sobre un *socket*, y TCP decide cómo repartirlos en segmentos. Si cada llamada saliera de inmediato en su propio segmento, una aplicación que escribe de a pocos bytes llenaría la red de segmentos casi vacíos, en los que los 40 bytes de encabezados TCP/IP pesan más que los datos. El algoritmo de Nagle evita eso: mientras haya datos enviados sin confirmar, TCP no envía un segmento más chico que el tamaño máximo, sino que acumula los datos nuevos hasta recibir el ACK pendiente [63]. Si no hay nada sin confirmar, el segmento sale enseguida. Así, una escritura aislada no se demora, pero la segunda de dos escrituras seguidas espera el ACK de la primera. La especificación vigente de TCP recomienda implementar Nagle, pero exige que cada aplicación pueda desactivarlo en una conexión [64], opción que en la interfaz de *sockets* se llama `TCP_NODELAY`.

ACK retardado. Del lado del receptor existe una optimización equivalente. En lugar de confirmar cada segmento apenas llega, el receptor puede esperar, hasta 500 ms según la especificación, para enviar el ACK junto con los datos de su respuesta y ahorrar un paquete [32]. Si el temporizador vence antes de que haya una respuesta, el ACK sale solo. Cada mecanismo tiene sentido por separado, pero combinados pueden bloquearse entre sí como sucedió durante el experimento.

Sincronización de relojes con SNTP. Para medir la latencia extremo a extremo, el nodo y el servidor tienen que compartir la hora. NTP (*Network Time Protocol*) sincroniza el reloj de un cliente con el de un servidor: el cliente envía una consulta, el servidor responde con su hora y, con las marcas de tiempo de envío y de recepción de ambos mensajes, el cliente estima el tiempo de viaje y lo descuenta [65]. SNTP (*Simple Network Time Protocol*) es una versión reducida de NTP para dispositivos con pocos recursos, que omite los algoritmos con los que NTP filtra las mediciones y elige entre varios servidores [65].

5.2.2. Planteo

Ocho placas ESP32 emuladas envían telemetría periódica a un servidor por dos caminos: MQTT, a través de un broker, y HTTP, con un POST por lectura. El contenido de cada lectura es

POR A - DOR

idéntico en ambos caminos y se serializa en JSON o en Protocol Buffers, lo que permite separar el efecto del transporte del de la serialización.

A diferencia del experimento de sincronización, el caso de uso es neutral: enviar lecturas de los nodos a un servidor es algo que ambas arquitecturas resuelven de forma razonable, sin que el modelo publicador/suscriptor le dé de antemano una ventaja a MQTT.

Motivación

Este experimento corresponde al último de los objetivos específicos (Sección 1): medir el costo en bytes y en latencia de MQTT y Protocol Buffers frente a HTTPS y JSON, y su comportamiento ante pérdidas y latencia. También aporta datos al objetivo que analiza MQTT sobre la capa de transporte, en lo que respecta a TCP y TLS. En el marco teórico se planteó que MQTT envía mensajes más livianos y con menor latencia que HTTP, por su encabezado fijo de dos bytes y su conexión persistente, y que Protocol Buffers produce mensajes más chicos que JSON, por su codificación binaria. Esas ventajas surgen de comparaciones cualitativas y de estudios con otros mensajes y plataformas. El experimento las mide sobre un mismo sistema. Para atribuir cada parte de la diferencia a su causa, incluye dos variantes de HTTP, corridas con y sin TLS y las dos serializaciones.

Fundamento

La diferencia principal entre ambos caminos no está en el contenido de la lectura, sino en cómo cada protocolo usa la conexión TCP (Tabla 7).

Tabla 7: Diferencias entre MQTT y HTTP en el uso de la conexión.

Aspecto	**MQTT**	**HTTP**
Conexión	Una conexión TCP persistente por nodo	Una conexión por lectura, o una reutilizada para varias
Handshake TLS	Una vez, al conectar	En cada conexión nueva
Encabezado por mensaje	Binario: 2 bytes fijos, más el tópico	Línea de solicitud y headers en texto
Respuesta	Ninguna con QoS 0; PUBACK con QoS 1	Siempre una respuesta HTTP
Garantía de entrega	QoS 0, 1 o 2, definidos por el protocolo	La de TCP, más los reintentos que implemente la aplicación
Iniciativa	El servidor puede enviar al nodo por la misma conexión	Solo el cliente inicia

Por eso se prueban dos variantes de HTTP: una conexión nueva por lectura y una conexión reutilizada. La segunda es la comparación justa con MQTT; la primera representa el peor caso, el de un dispositivo que duerme entre envíos y abre una conexión cada vez.

Preguntas de investigación

1. ¿Cuántos bytes cuesta enviar una lectura por MQTT y por HTTPS, contando cabeceras y handshakes?
2. ¿Cómo se comparan la latencia extremo a extremo y la tasa de entrega de ambos?
3. ¿Cómo se degradan ambos protocolos con pérdida y latencia en la red?
4. ¿Qué parte de la diferencia viene de la conexión persistente y qué parte de TLS?
5. ¿Cuánto cambia el costo al pasar de JSON a Protocol Buffers en cada transporte?

Hipótesis

Antes de las corridas se formularon las siguientes hipótesis, que se contrastan con los resultados al final del análisis (Tabla 22):

BORRADOR

- MQTT envía menos bytes por lectura que HTTPS, y la diferencia crece cuando el período de envío es corto.
- HTTPS con conexión reutilizada reduce gran parte de la diferencia, pero los headers HTTP siguen pesando más que la cabecera MQTT.
- HTTPS con una conexión por request es el caso más costoso, sobre todo con TLS y latencia alta, porque cada handshake suma idas y vueltas.
- Con pérdida, MQTT QoS 1 mantiene la entrega a costa de duplicados, mientras que MQTT QoS 0 pierde lecturas.
- Protobuf reduce el payload en ambos caminos, pero su efecto sobre el total es mayor en MQTT, donde el payload es la mayor parte del mensaje. En HTTPS, los headers diluyen el ahorro.

5.2.3. Implementación

Arquitectura

Cada nodo es un ESP32 emulado con el QEMU de Espressif que ejecuta un firmware real, escrito en C con ESP-IDF. Por eso los bytes que se miden son los que producen las bibliotecas que usaría una placa física, sin modificaciones: la pila TCP/IP lwIP, mbedTLS para TLS, esp-mqtt como cliente MQTT y `esp_http_client` como cliente HTTP. Como QEMU no emula WiFi, cada nodo usa una placa Ethernet emulada.

Cada instancia de QEMU se conecta a una interfaz TAP propia, y todas las interfaces se unen en un puente (*bridge*) de Linux en la computadora anfitriona, que tiene la dirección del servidor (`10.10.0.1`); los nodos usan las direcciones `10.10.0.11` en adelante. Se eligió este esquema, y no la red de usuario de QEMU, por dos motivos. La red de usuario termina las conexiones TCP del nodo dentro del propio emulador, de modo que la red no vería los paquetes tal como los arma el ESP32. Además, una interfaz por nodo permite degradar cada enlace por separado. Sobre cada interfaz se aplica `tc netem` en ambos sentidos para inyectar pérdida y latencia, y `dumpcap` captura en el puente todo el tráfico de la corrida (Figura 2).

Del lado del servidor hay dos procesos: el broker Mosquitto y un backend escrito en Go. El backend atiende `POST /tel/{esp}` por HTTP (puerto 80) y por HTTPS (puerto 443) y responde `204 No Content`. Además, está suscripto al broker en el tópico `exp/tel/+` por una conexión local que no pasa por el puente y, por lo tanto, no entra en la captura. Las lecturas de ambos caminos terminan en la misma función de registro, que anota la hora de llegada, de modo que la latencia se calcula igual para los dos. El backend también funciona como servidor SNTP, para que los nodos sincronicen su reloj con el de la computadora anfitriona.

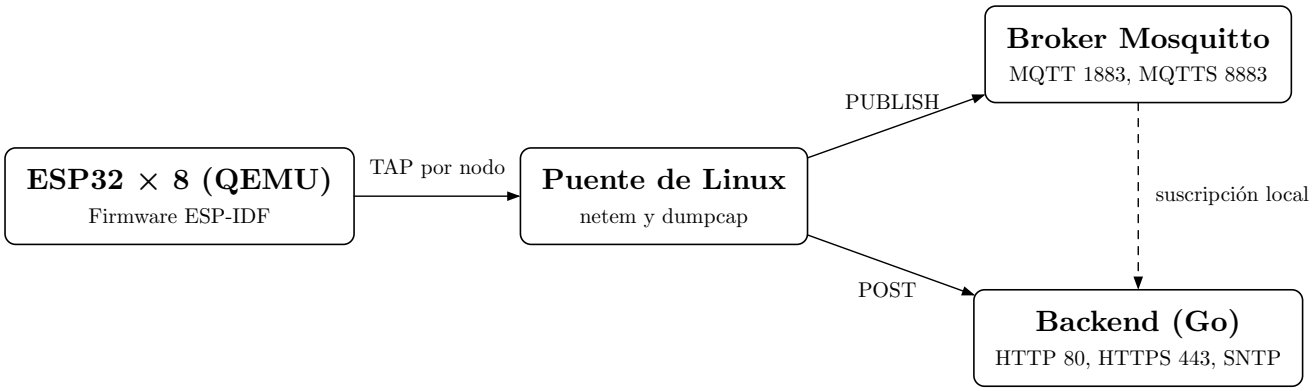


Figura 2: Arquitectura del experimento. La captura se hace en el puente, por donde pasa todo el tráfico de los nodos; la suscripción del backend al broker queda fuera de ella.

BORRADOR

Mensaje y caminos

Todas las lecturas usan el mismo mensaje `Telemetry`, definido en un archivo `.proto` compartido por el firmware, que lo codifica con nanopb, y por el backend:

```
message Telemetry {
  uint32 esp_id = 1;
  uint32 seq    = 2;
  uint64 t_us   = 3;  // hora de creación en el nodo (µs)
  float  value  = 4;
  uint32 status = 5;
}
```

Codificado con Protocol Buffers, el mensaje ocupa entre 16 y 19 bytes (18 en promedio), porque `seq` es un entero de longitud variable. En JSON, con los mismos nombres de campo y sin espacios, ocupa unos 72 bytes, cuatro veces más:

```
{"esp_id":3,"seq":142,"t_us":1793041765379595,"value":23.4567,"status":0}
```

La lectura se envía por uno de los cuatro caminos de la Tabla 8. En HTTP, el formato se indica en el header `Content-Type` (`application/x-protobuf` o `application/json`).

Tabla 8: Transportes comparados.

Transporte	**Qué hace el nodo**
MQTT QoS 0	PUBLISH a `exp/tel/{esp}` con QoS 0, sobre una conexión persistente abierta antes de empezar a medir
MQTT QoS 1	Igual, con QoS 1: el broker confirma cada mensaje con un PUBACK
HTTP con conexión reutilizada	POST a `/tel/{esp}` sobre una única conexión que se mantiene abierta (conexión persistente de HTTP/1.1)
HTTP con una conexión por lectura	Abre una conexión por cada lectura, envía el POST con `Connection: close` y la cierra

Los cuatro caminos se corrieron con y sin TLS. Con TLS se usó la misma configuración en el broker y en el backend: TLS 1.2 [66] con la suite `ECDHE-ECDSA-AES128-GCM-SHA256`, el mismo certificado ECDSA P-256 firmado por una autoridad de certificación propia, que el nodo verifica, y sin reanudación de sesión. El cliente MQTT de ESP-IDF implementa MQTT 3.1.1 [67], y no la versión 5.0. Para este experimento la diferencia es menor, porque el encabezado fijo es el mismo en ambas versiones y la 5.0 solo agrega a cada PUBLISH un byte con la longitud de sus propiedades.

Firmware

En cada nodo, una tarea productora crea una lectura por período con la hora sincronizada (`t_us`) y la encola, y otra tarea, la emisora, la saca de la cola, la serializa y la envía. Separar la creación del envío hace que el tiempo que una lectura espera en la cola, por ejemplo mientras el emisor está bloqueado en un handshake o en una retransmisión, quede incluido en la latencia medida.

Se dejaron los valores por defecto de las bibliotecas, que son los que tendría un ESP32 sin ajustes. En MQTT, el keep-alive es de 60 s y un PUBLISH de QoS 1 se reenvía si su PUBACK no llega en 1 s. En HTTP, cada envío tiene hasta tres intentos con un tiempo de espera de 5

BORRADOR

s, y el algoritmo de Nagle (Sección 5.2.1) está activo. El cliente MQTT abre la conexión antes de que empiece la medición, mientras que el cliente HTTP la abre recién al enviar la primera lectura.

La configuración de cada nodo (transporte, TLS, período y serialización) se lee de una partición de la memoria *flash*, de modo que el firmware se compila una sola vez para todas las corridas.

Diseño de las corridas

Se tomó una configuración base: 8 nodos, período de envío de 1 s, TLS activado, sin pérdida ni latencia agregadas y serialización con Protocol Buffers. A partir de ella se varió un factor a la vez (Tabla 9) para cada uno de los cuatro transportes. Cada configuración se repitió cinco veces, con corridas de 120 s y en orden aleatorio con semilla fija. En total fueron 42 configuraciones y 210 corridas, que llevaron 7 h 40 min.

Tabla 9: Factores del experimento.

Factor	**Valores (base en negrita)**	**Pregunta que responde**
Nodos	2, 4, **8**	¿Se satura el broker o el backend?
Período de envío	100 ms, **1 s**, 10 s	¿Cuándo pesa el handshake?
TLS	**Con**, sin	¿Qué parte del costo es cifrado?
Pérdida	**0 %**, 1 %, 5 %	¿Cómo se degrada la entrega?
RTT agregado	**0 ms**, 50 ms, 200 ms	¿Cómo afecta la latencia de la red a cada camino?
Serialización	**Protocol Buffers**, JSON	¿Cuánto ahorra Protocol Buffers en cada camino?

La pérdida se aplica en cada sentido por separado, de forma aleatoria e independiente por paquete. La latencia se expresa como RTT agregado: la mitad se agrega en cada sentido, como un retardo fijo.

HTTP con una conexión por lectura se corrió con un período base de 10 s en lugar de 1 s. Con criptografía emulada, cada handshake TLS tarda entre 3 y 5 s, de modo que con un período de 1 s la cola del nodo crece sin límite y solo se entrega cerca del 27 % de las lecturas. Por eso, cuando se comparan los cuatro caminos en la base, ese camino aparece con 10 s, y se lo compara además con las corridas de 10 s de los otros tres.

Además, para verificar la explicación de la latencia de HTTP, se corrió una matriz complementaria con el algoritmo de Nagle activo y desactivado, que se presenta en el análisis (Tabla 21).

Métricas

Los bytes se miden en la red y no en la aplicación, para incluir todo lo que agregan TCP, TLS y cada protocolo (Tabla 10).

BORRADOR

Tabla 10: Métricas del experimento.

Métrica	**Cómo se calcula**
Bytes por lectura	Bytes de los paquetes IP del puerto del servidor en la captura, divididos por las lecturas entregadas. Incluyen encabezados TCP/IP y TLS, ACK, handshakes, cierres y retransmisiones
Paquetes por lectura	Paquetes de la captura divididos por las lecturas entregadas
Bytes de handshake	Bytes de cada conexión antes del primer dato de aplicación
Latencia extremo a extremo	Hora de llegada al servidor menos `t_us`, de la primera llegada de cada lectura (mediana y percentiles 95 y 99)
Tasa de entrega	Lecturas distintas recibidas sobre lecturas producidas
Duplicados	Lecturas recibidas más de una vez
Retransmisiones	Segmentos TCP retransmitidos en la captura
Factor de tiempo real	Avance del reloj del nodo sobre avance del reloj de la computadora anfitriona

Entorno y validez

La Tabla 11 resume el entorno en el que se corrió el experimento.

Tabla 11: Entorno del experimento.

Componente	**Versión**
Computadora anfitriona	AMD Ryzen 7 5700U (8 núcleos, 16 hilos), 14 GB de RAM, Linux
Emulador	QEMU de Espressif 9.2.2
Firmware	ESP-IDF 5.5.6 (lwIP, mbedTLS, esp-mqtt, `esp_http_client`) y nanopb 0.4.9.1
Broker	Mosquitto 2.1.2
Backend y análisis	Go 1.27.3 y TShark 4.7.4

Las 210 corridas resultaron válidas, el factor de tiempo real mínimo fue 1,000 en todas, ningún nodo se atrasó al producir lecturas, no quedaron lecturas sin enviar ni se descartaron lecturas de la cola, y no hubo desconexiones de MQTT. Dos corridas fallaron antes de empezar a medir, porque un nodo no logró sincronizar su reloj con 5 % de pérdida. Se amplió el tiempo de espera de la sincronización de 30 s a 90 s y se repitieron. Ese cambio solo afecta el arranque del nodo, antes de que empiece la medición.

Para calcular la latencia, a la hora de llegada, tomada del reloj del servidor, se le resta la hora de creación de la lectura, tomada del reloj del nodo. Ambos relojes se sincronizan por SNTP (Sección 5.2.1), pero el cliente SNTP de lwIP no descuenta el tiempo de viaje de la respuesta del servidor, así que el reloj del nodo queda atrasado medio RTT y cada latencia medida resulta medio RTT más larga. Sin latencia agregada, ese error es de alrededor de 1 ms. Con latencia agregada, el error crece a la mitad del RTT agregado, pero como netem agrega un retardo fijo se lo puede descontar: a cada latencia medida se le restó RTT/2.

5.2.4. Resultados

Se presentan la media y el desvío estándar de las cinco repeticiones de cada configuración. Salvo que se indique otra cosa, los valores corresponden a la configuración base con TLS. La entrega fue del 100 % en las 210 corridas.

BORRADOR

Bytes y paquetes por lectura

La Tabla 12 y la Figura 3 muestran el costo de una lectura en cada camino. Todas las diferencias entre transportes resultaron significativas (prueba de Welch, $p < 10^{-4}$), y los intervalos de confianza del 95 % son estrechos, por ejemplo 165,2 ± 0,1 B en MQTT QoS 0, salvo en HTTP con una conexión por lectura (2596 ± 298B).

Tabla 12: Bytes IP y paquetes por lectura entregada. HTTP con una conexión por lectura usa un período de 10 s; el resto, 1 s.

Transporte	Bytes sin TLS	Bytes con TLS	Paquetes (TLS)	Subida / bajada (TLS)
MQTT QoS 0	118,7	165,2 ± 0,1	2,25	109 / 56 B
MQTT QoS 1	162,1 ± 0,2	237,7 ± 0,2	3,19	150 / 88 B
HTTP, conexión reutilizada	457,7 ± 0,2	555,0 ± 0,2	6,08	334 / 221 B
HTTP, conexión por lectura	741,8	2596 ± 298	22,5	1005 / 1591 B

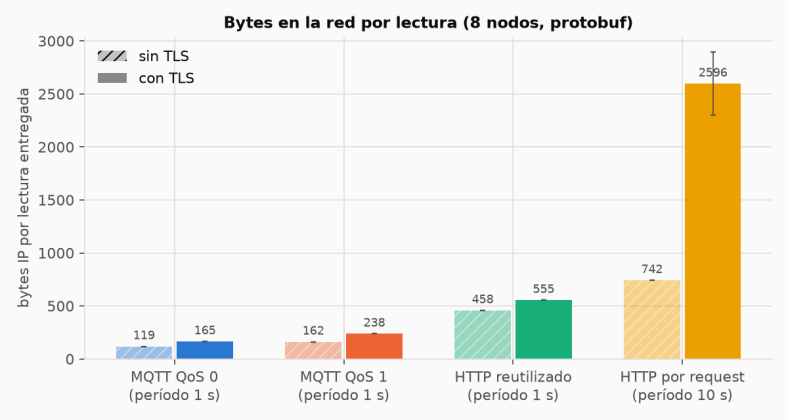


Figura 3: Bytes IP por lectura con y sin TLS (8 nodos, Protocol Buffers).

Con TLS, una lectura por MQTT QoS 0 ocupó 3,4 veces menos bytes que por HTTP con conexión reutilizada y entre 14 y 16 veces menos que por HTTP con una conexión por lectura. HTTP con una conexión por lectura fue además la única configuración con un desvío grande entre repeticiones.

Latencia en la configuración base

Tabla 13: Latencia extremo a extremo en la configuración base.

Transporte	p50 (ms)	p95 (ms)	p99 (ms)
MQTT QoS 0	2,9	5,5	11,9
MQTT QoS 1	3,0	6,3	9,1
HTTP, conexión reutilizada	44,8	48,5	2737
HTTP, conexión por lectura (10 s)	3383	3924	4129

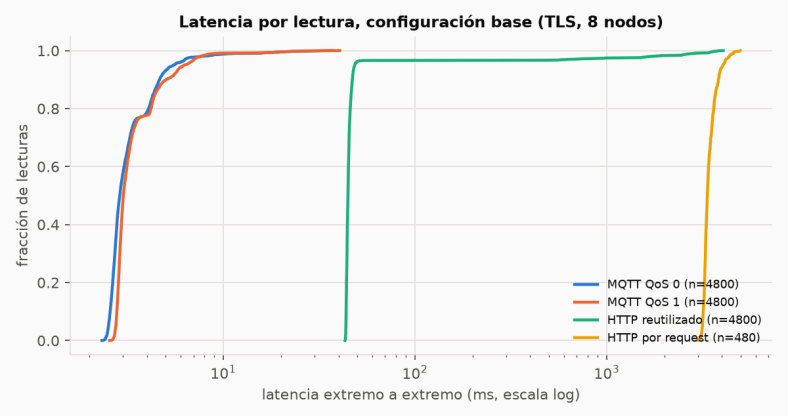


Figura 4: Distribución acumulada de la latencia en la configuración base (escala logarítmica).

La Tabla 13 y la Figura 4 muestran tres grupos separados por un orden de magnitud:

- MQTT tuvo una mediana de unos 3 ms con QoS 0 y con QoS 1, y la diferencia entre ambos no resultó significativa ($p = 0{,}06$).
- HTTP con conexión reutilizada tuvo una mediana de 44,8 ms y una cola larga en el percentil 99. Esa cola se concentra en las primeras lecturas de cada nodo: la primera tardó 3,6 s en promedio, contra 21 ms en MQTT, y desde la décima lectura en adelante el p99 bajó a 48 ms.
- HTTP con una conexión por lectura tuvo una mediana de 3,4 s con TLS y de 10 ms sin TLS.

Pérdida de paquetes

Con 1 % y 5 % de pérdida la entrega siguió siendo del 100 % en todos los caminos. Lo que cambió fueron las retransmisiones, los duplicados y la cola de la latencia (Tabla 14 y Figura 5). Con 5 % de pérdida, el 37 % de las lecturas de HTTP con conexión reutilizada tardó más de 50 ms, contra el 16 % en MQTT QoS 0 y el 10 % en QoS 1. QoS 1 recibió duplicados en el 5,4 % de las lecturas. HTTP con conexión reutilizada tuvo además 1,6 errores por corrida (tiempo de espera de 5 s y reintento de la aplicación), que dejaron 1,4 duplicados.

Tabla 14: Efecto de la pérdida de paquetes. La entrega fue del 100 % en todos los casos. Δ bytes es el aumento de los bytes por lectura respecto de la base.

Transporte	Pérdida	Retransm. por corrida	Duplicados por corrida	p95 (ms)	p99 (ms)	Δ bytes
MQTT QoS 0	1 %	12,8	0	6,7	941	+0,9 %
	5 %	59,2	0	1304	3040	+4,1 %
MQTT QoS 1	1 %	25,4	9,0	6,9	617	+1,5 %
	5 %	124,0	52,0	1017	1749	+7,1 %
HTTP, conexión reutilizada	1 %	30,8	0	1238	3164	+0,7 %
	5 %	169,4	1,4	3810	6638	+4,3 %
HTTP, conexión por lectura	1 %	83,2	0	4491	4784	+9,7 %
	5 %	126,2	0	6287	8492	+12,8 %

BORRADOR

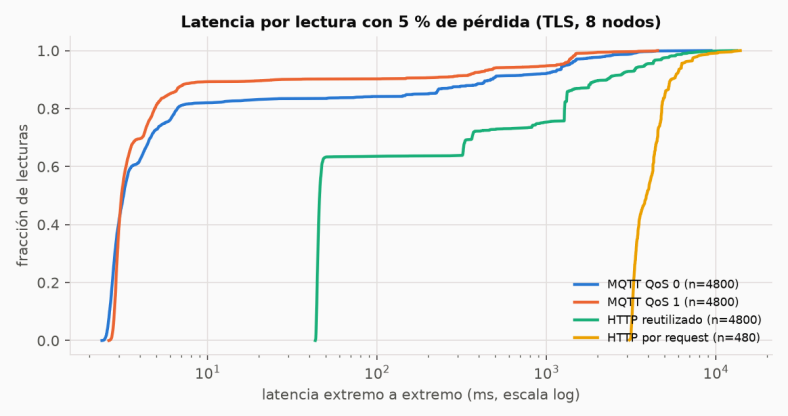


Figura 5: Distribución acumulada de la latencia con 5 % de pérdida (escala logarítmica).

Latencia de la red

La Tabla 15 muestra la mediana de la latencia, ya corregida por el sesgo del reloj, al agregar 50 y 200 ms de RTT. La última columna expresa el aumento con 200 ms en cantidad de RTT. El tiempo de conexión, desde el SYN hasta el fin del handshake, subió entre 340 y 570 ms con 200 ms de RTT. Los bytes por lectura de los caminos con conexión persistente no cambiaron. Los de HTTP con una conexión por lectura bajaron de 2596 a 2297 B.

Tabla 15: Mediana de la latencia, corregida en −RTT/2, según el RTT agregado (TLS).

Transporte	**RTT 0**	**RTT 50 ms**	**RTT 200 ms**	**Δ con 200 ms**	**Δ en RTT**
MQTT QoS 0	2,9 ms	27,8 ms	102,7 ms	+99,8 ms	**0,5**
MQTT QoS 1	3,0 ms	27,9 ms	102,9 ms	+99,9 ms	**0,5**
HTTP, conexión reutilizada	44,8 ms	119,5 ms	344,6 ms	+299,8 ms	**1,5**
HTTP, conexión por lectura	3383 ms	3728 ms	4434 ms	+1051 ms	**5,3**

Efecto de TLS

La Tabla 16 compara cada camino con y sin TLS. En las conexiones persistentes, TLS sumó entre 46 y 97 B por lectura y menos de 0,6 ms de latencia. En HTTP con una conexión por lectura triplicó los bytes y sumó 3,4 s de latencia.

Tabla 16: Lo que agrega TLS a cada camino. En HTTP con una conexión por lectura se usan las corridas sin retransmisiones innecesarias (ver el análisis).

Transporte	**Δ bytes por lectura**	**Δ relativo**	**Δ latencia p50**	**Handshake por conexión**
MQTT QoS 0	+46,5 B	+39 %	+0,39 ms	128 → 1966 B
MQTT QoS 1	+75,6 B	+47 %	+0,38 ms	128 → 1967 B
HTTP, conexión reutilizada	+97,3 B	+21 %	+0,54 ms	128 → 1525 B
HTTP, conexión por lectura	+1555 B	× 3,1	+3373 ms	128 → 1525 B

BORRADOR

Serialización

Tabla 17: Protocol Buffers contra JSON (TLS). En HTTP con una conexión por lectura se usan las corridas sin retransmisiones innecesarias.

Transporte	Payload Protobuf	Payload JSON	Bytes por lectura (Protobuf)	Bytes por lectura (JSON)	Δ
MQTT QoS 0	18,0 B	72,1 B	165,2	219,4	+54,2 B (+32,8 %)
MQTT QoS 1	18,0 B	72,1 B	237,7	291,9	+54,2 B (+22,8 %)
HTTP, conexión reutilizada	18,0 B	72,1 B	555,0	603,2	+48,2 B (+8,7 %)
HTTP, conexión por lectura	17,8 B	71,2 B	2296,6	2357,7	+61 B (+2,7 %)

JSON resultó cuatro veces más grande que Protocol Buffers para este mensaje y agregó unos 54 B por lectura en todos los caminos (Tabla 17). La latencia no cambió.

Cantidad de nodos y período de envío

Con 2, 4 y 8 nodos, los bytes, los paquetes y la latencia por lectura no cambiaron (por ejemplo, 165,1, 165,2 y 165,2 B en MQTT QoS 0). Lo único que creció fue la duración de los handshakes: de 3,01 a 3,32 s en HTTP con una conexión por lectura y de 3,14 a 3,76 s en MQTT QoS 0.

Al acortar el período, los bytes por lectura de los caminos con conexión persistente bajaron. En MQTT QoS 0 fueron 392, 165 y 143 B con períodos de 10 s, 1 s y 100 ms; en HTTP con conexión reutilizada, 673, 555 y 515 B. Con el mismo período de 10 s que HTTP con una conexión por lectura, MQTT QoS 0 usó 392 B, MQTT QoS 1 450 B y HTTP con conexión reutilizada 673 B, contra 2596 B. La latencia de MQTT no cambió con el período. En HTTP con conexión reutilizada y período de 100 ms, la mediana se mantuvo en 44,5 ms, pero el p95 subió a 705 ms.

5.2.5. Análisis de resultados

Anatomía de una lectura

Las capturas permiten ver de qué está hecha cada lectura, paquete por paquete. La Tabla 18 muestra el régimen estable con TLS de los tres caminos con conexión persistente.

BORRADOR

Tabla 18: Paquetes de una lectura en régimen estable, con TLS.

Sentido	Paquete	Bytes IP	Composición
MQTT QoS 0: 2 paquetes, 140 B por lectura			
Nodo → servidor	PUBLISH	100	31 B de MQTT, 29 B de TLS y 40 B de TCP/IP
Servidor → nodo	ACK	40	Solo encabezados TCP/IP
MQTT QoS 1: 3 paquetes, unos 215 B por lectura			
Nodo → servidor	PUBLISH	102	33 B de MQTT (2 B más por el identificador de paquete)
Servidor → nodo	PUBACK	73	4 B de MQTT, 29 B de TLS y 40 B de TCP/IP; incluye el ACK de TCP
Nodo → servidor	ACK	40	Sale solo, unos 83 ms después (ACK retardado de lwIP)
HTTP con conexión reutilizada: 6 paquetes, unos 543 B por lectura			
Nodo → servidor	POST y headers	203	134 B de HTTP
Servidor → nodo	ACK	40	Llega 40 ms después (ACK retardado de Linux)
Nodo → servidor	Cuerpo	87	18 B de Protocol Buffers, retenidos por Nagle hasta el ACK anterior
Servidor → nodo	ACK	40	Solo encabezados TCP/IP
Servidor → nodo	`204 No Content`	133	64 B de HTTP
Nodo → servidor	ACK	40	Sale solo, unos 230 ms después (ACK retardado de lwIP)

HTTP con una conexión por lectura abre y cierra una conexión completa en cada lectura, con unos 22 paquetes: el handshake TCP (3 paquetes, 128 B), el handshake TLS 1.2 (unos 1525 B, de los cuales 655 B son el certificado y los demás mensajes del servidor), el POST con su cuerpo, la respuesta, el cierre de TLS y el cierre de TCP. Sin TLS son 12 paquetes y 742 B.

El encabezado de MQTT es tan chico como plantea el marco teórico. El PUBLISH ocupa 31 bytes: 2 del encabezado fijo, 2 de la longitud del tópico, 9 del tópico `exp/tel/3` y 18 de la lectura. La sobrecarga de MQTT por mensaje es entonces de 13 bytes, contra 134 bytes de la línea de solicitud y los headers de HTTP (153 con una conexión por lectura, que agrega `Connection: close`), unas diez veces menos. Además, MQTT QoS 0 no tiene respuesta de aplicación, mientras que HTTP siempre responde (64 B).

Sin embargo, en una lectura de MQTT los encabezados de capa de aplicación no son lo que más pesa. Cada registro TLS agrega 29 bytes (5 de cabecera, 8 del *nonce* explícito y 16 de la etiqueta de autenticación de AES-GCM) y cada paquete, 40 bytes de TCP/IP. En MQTT QoS 0 con TLS, el payload es solo el 11 % de los bytes de la lectura, y en HTTP con una conexión por lectura, menos del 1 % (Figura 6).

BORRADOR

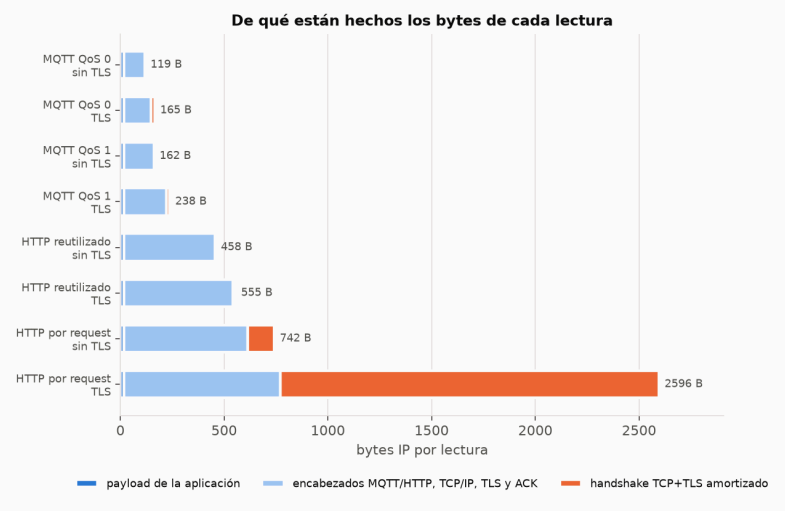


Figura 6: Composición de los bytes de cada lectura: payload, encabezados (MQTT o HTTP, TCP/IP, TLS y ACK) y handshake amortizado.

Los 29 bytes por registro explican casi todo lo que agrega TLS en las conexiones persistentes (Tabla 16). MQTT QoS 0 usa un registro por lectura (29 B), al que se suman 16 B del handshake repartido entre las lecturas, 45 B en total, y se midieron 46,5 B. MQTT QoS 1 usa dos registros, el PUBLISH y el PUBACK (74 B con el handshake; se midieron 75,6 B), y HTTP con conexión reutilizada usa tres, los headers, el cuerpo y la respuesta (100 B; se midieron 97 B). El cifrado simétrico casi no suma latencia: el costo de TLS en tiempo es el handshake, que solo se paga en cada lectura cuando no se reutiliza la conexión.

Costo fijo por conexión y costo por lectura

En los caminos con conexión persistente, lo que cambia con el período es cuántas lecturas viajan por la misma conexión. Si n es la cantidad de lecturas por conexión, los bytes por lectura se ajustan a un modelo de dos términos:

$$b(n) = v + \frac{C}{n} \tag{1}$$

donde v es el costo de cada lectura y C el costo fijo de abrir y cerrar la conexión, que se reparte entre las n lecturas. El ajuste sobre las corridas con distintos períodos, con TLS, dio un R^2 entre 0,97 y 1,00 (Tabla 19 y Figura 7).

Tabla 19: Ajuste del modelo de la Ecuación 1 con TLS.

Transporte	v **(B por lectura)**	C **(B por conexión)**	R^2
MQTT QoS 0	**140**	3020	1,000
MQTT QoS 1	202	2983	0,994
HTTP, conexión reutilizada	526	**1772**	0,973

BORRADOR

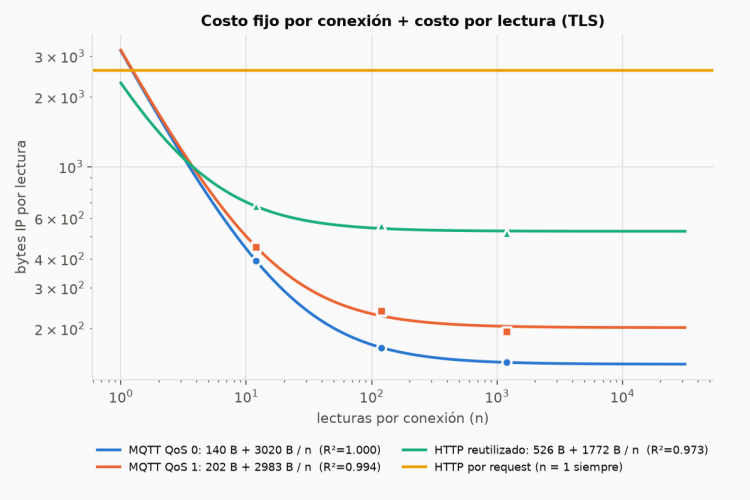


Figura 7: Modelo de costo fijo por conexión y costo por lectura. Los puntos son las mediciones con períodos de 10 s, 1 s y 100 ms; las curvas, el ajuste.

El valor de v coincide con la anatomía de una lectura: los 140 B de MQTT QoS 0 son exactamente el PUBLISH (100 B) y su ACK (40 B). El valor de C corresponde al handshake TCP y TLS, más el inicio y el cierre de la sesión. El C de MQTT es unos 1250 B mayor que el de HTTP por dos razones: Mosquitto envía, junto con el certificado del servidor, el de la autoridad de certificación (856 B de certificados contra 445 B del backend en Go), y MQTT agrega el intercambio de CONNECT y CONNACK al abrir la sesión.

En consecuencia, MQTT tiene el costo por lectura más bajo pero el costo fijo más alto, y su ventaja depende de cuánto dure la conexión. Igualando los modelos, MQTT QoS 0 envía menos bytes que HTTP con conexión reutilizada mientras la conexión lleve más de tres o cuatro lecturas. Con una sola lectura por conexión, el modelo da unos 3160 B para MQTT, más que los 2296 B de un POST por HTTPS, aunque eso es una extrapolación fuera del rango medido (entre 12 y 1200 lecturas por conexión). Para un nodo que duerme y se reconecta en cada envío, la ventaja de MQTT desaparece, salvo que se use reanudación de sesión TLS, que en este experimento estuvo desactivada en ambos caminos.

Qué explica la diferencia

Para separar las causas, la Tabla 20 pasa de HTTP con una conexión por lectura a MQTT QoS 0 cambiando una cosa por vez.

Tabla 20: Descomposición de la diferencia entre HTTP con una conexión por lectura y MQTT QoS 0. Los porcentajes son la parte de la diferencia total que explica cada paso.

Paso	**Con TLS (B por lectura)**	**Sin TLS (B por lectura)**
HTTP con una conexión por lectura	2296	742
→ HTTP con conexión reutilizada (**conexión persistente**)	555 (−1741, **82 %**)	458 (−284, **46 %**)
→ MQTT QoS 1 (**protocolo de aplicación**)	238 (−317, 15 %)	162 (−296, 47 %)
→ MQTT QoS 0 (**sin confirmación de aplicación**)	165 (−73, 3 %)	119 (−43, 7 %)

Con TLS, la conexión persistente explica cuatro quintos de la diferencia, porque evita un handshake de 1525 B en cada lectura. El protocolo de aplicación (headers, respuesta obligatoria y ACK adicionales) explica el 18 % restante. Sin TLS, el handshake TCP pesa mucho menos y

BORRADOR

la diferencia se reparte casi por mitades entre la conexión y el protocolo. Es decir, el factor más pesado no es el protocolo de aplicación en sí, sino la decisión de mantener la conexión abierta. A eso se suma una parte que no es del protocolo ni de la conexión, sino de cómo el cliente HTTP usa TCP, que se analiza a continuación.

Nagle y ACK retardado

Los dos mecanismos de TCP presentados en la Sección 5.2.1, el algoritmo de Nagle y el ACK retardado, explican buena parte de la latencia de HTTP y varios efectos secundarios observados en el experimento.

Bloqueo en HTTP con conexión reutilizada. `esp_http_client` escribe cada solicitud con dos llamadas a `send()`: primero la línea de solicitud con los headers y después el cuerpo, y cada llamada sale como un segmento TCP distinto. La Figura 8 muestra la secuencia medida en una lectura:

1. Los headers salen y quedan sin confirmar, así que Nagle retiene el cuerpo (18 B) en el ESP32 hasta recibir su ACK.
2. El servidor Linux detecta una conexión de ida y vuelta, en la que contesta cada solicitud, y demora el ACK hasta 40 ms para enviarlo junto con la respuesta. Pero no puede responder, porque todavía no recibió el cuerpo.
3. Cada lado espera al otro, y el bloqueo se rompe recién cuando vence el temporizador de 40 ms del servidor: llega el ACK, Nagle libera el cuerpo y el servidor responde.

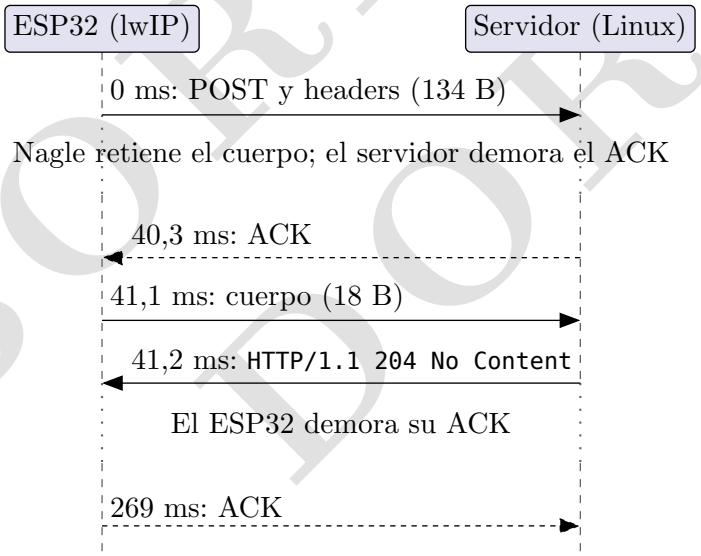


Figura 8: Una lectura de HTTP con conexión reutilizada, sin TLS, tomada de una captura. Las flechas punteadas son ACK sin datos.

La espera medida entre los headers y el cuerpo fue de 41,5 ms sin TLS y de 41,7 ms con TLS. Con TLS ocurre lo mismo porque mbedTLS cifra cada `send()` en su propio registro, de modo que también salen dos segmentos. Esa espera explica casi toda la mediana de 44,8 ms de HTTP con conexión reutilizada (Tabla 13).

El mismo bloqueo dentro del handshake TLS. HTTP con una conexión por lectura sin TLS no sufre la espera: Linux confirma de inmediato los primeros segmentos de una conexión nueva (modo *quickack*), y en la captura el ACK a los headers llega en 44 µs. Con TLS, en cambio, los segmentos del handshake agotan ese modo y la espera aparece dos veces por conexión. La primera es dentro del handshake, porque el ESP32 envía el *Client Key Exchange* y luego el *Change Cipher Spec* con el *Finished* en dos escrituras. La segunda es entre los headers y el

BORRADOR

cuerpo del POST. Esos 80 ms quedan ocultos detrás de los 3,4 s del handshake con criptografía emulada.

Prueba con Nagle desactivado. Para verificar la explicación se corrió una matriz complementaria de 2 × 2 (Nagle activo o desactivado, con y sin TLS) para los dos caminos HTTP, con la configuración base y tres repeticiones. Con Nagle desactivado, el firmware pone `TCP_NODELAY` en el *socket* apenas se abre la conexión. La Tabla 21 resume los resultados:

- En HTTP con conexión reutilizada, la latencia bajó unas diez veces, de 44 ms a entre 4 y 5 ms, con y sin TLS. Los 40 ms que desaparecen son el ACK retardado del servidor.
- Los bytes bajaron unos 40 B por lectura y hubo un paquete menos: sin la espera, el ACK del servidor a los headers viaja junto con la respuesta. El cuerpo siguió en su propio segmento, porque siguen siendo dos `send()`.
- HTTP con una conexión por lectura sin TLS no cambió (741,8 B en los dos casos), porque no sufría la espera.
- HTTP con una conexión por lectura con TLS ahorró un paquete y unos 35 B por conexión, pero la espera dentro del handshake no desapareció. El cliente HTTP avisa que la conexión está abierta recién cuando terminó el handshake, y `esp_http_client` no ofrece un punto anterior para configurar el *socket* sin modificar la biblioteca de TLS. La diferencia de latencia, de unos 120 ms, quedó dentro del ruido del handshake emulado.

Tabla 21: Prueba con el algoritmo de Nagle activo y desactivado (`TCP_NODELAY`): 8 nodos, 120 s, tres repeticiones, período de 1 s (10 s con una conexión por lectura). La entrega fue del 100 % en todos los casos.

Transporte	**TLS**	**Nagle**	**p50 (ms)**	**p95 (ms)**	**Bytes por lectura**	**Paquetes por lectura**
HTTP, conexión reutilizada	No	Activo	44,29 ± 0,07	46,65 ± 0,04	457,9	6,03
	No	**Desactivado**	**4,19 ± 0,05**	**7,17 ± 0,23**	**418,3**	**5,04**
	Sí	Activo	44,74 ± 0,10	48,48 ± 0,62	555,2 ± 0,2	6,08
	Sí	**Desactivado**	**5,02 ± 0,05**	**8,85 ± 0,49**	**515,0 ± 0,5**	**5,08**
HTTP, conexión por lectura	No	Activo	9,77 ± 0,15	59,37 ± 1,73	741,8	12,00
	No	Desactivado	10,13 ± 0,22	62,11 ± 3,34	741,8	12,00
	Sí	Activo	3422 ± 146	4064 ± 278	2927,7 ± 8,9	22,92
	Sí	Desactivado	3304 ± 76	3765 ± 198	2892,6 ± 0,6	21,93

Latencia de la red: la penalidad de Nagle crece con el RTT. MQTT pagó medio RTT por lectura (Tabla 15), el mínimo posible: el PUBLISH sale y llega sin esperar nada. En HTTP, en cambio, el ACK de los headers tarda un RTT completo en volver, y el cuerpo espera ese tiempo además de los 40 ms del ACK retardado. Por eso HTTP con conexión reutilizada pagó 1,5 RTT por lectura: medio RTT del viaje de ida y un RTT de la espera de Nagle. HTTP con una conexión por lectura con TLS pagó unos 5,5 RTT antes de que el cuerpo llegara al servidor: uno del handshake TCP, dos del handshake TLS 1.2, uno de la espera de Nagle dentro del handshake, medio de los headers y uno de la espera entre headers y cuerpo. Se midieron 5,3 RTT con 200 ms de RTT agregado; con 50 ms dio 6,9 RTT, pero su desvío, de 100 a 160 ms, es del orden de lo que se mide. Sin Nagle serían 3,5 RTT, y con TLS 1.3, que ahorra un RTT en el handshake [68], 2,5. El tiempo de conexión, en cambio, subió solo entre dos y tres RTT, poco frente a los 3,3 s de la criptografía emulada.

BORRADOR

El ACK retardado del ESP32 y las retransmisiones innecesarias. El ESP32 tampoco confirma enseguida: lwIP demora el ACK hasta su temporizador rápido, de unos 250 ms. Como en ese momento el nodo no tiene más datos para enviar, el ACK sale solo y suma un paquete por lectura en HTTP con conexión reutilizada y en MQTT QoS 1 (Tabla 18). Con un período de 100 ms, ese ACK viaja dentro del mensaje siguiente, y los paquetes por lectura bajan de 6,08 a 5,25 en HTTP y de 3,19 a 2,43 en QoS 1. Esto también explica que el ajuste de la Ecuación 1 sea algo peor en esos dos caminos.

Esa demora tiene un segundo efecto. Linux retransmite un segmento si no recibe su ACK dentro del tiempo de retransmisión (RTO), que calcula a partir del RTT medido y que nunca baja de 200 ms, aunque la especificación recomienda un mínimo de 1 s [40]. Si el ACK del ESP32 tarda más que ese RTO, el servidor retransmite datos que no se perdieron. Ocurrió en dos casos:

- En la primera respuesta de cada conexión de HTTP con conexión reutilizada. El servidor solo conoce el RTT del handshake, de pocos milisegundos, así que usa el mínimo de 200 ms, menor que los 230 ms que tarda el ACK del ESP32. Después ajusta su estimación y no se repite.
- En el handshake TLS de HTTP con una conexión por lectura. El ESP32 recibe el certificado y los demás mensajes del servidor (692 B) y pasa unos 3 s calculando su parte del handshake. Su ACK sale con el temporizador de lwIP, y si el RTO del servidor es menor, el servidor retransmite ese bloque completo.

El segundo caso explica toda la variabilidad de HTTP con una conexión por lectura (2596 ± 298 B por lectura): los bytes de cada corrida siguen exactamente la cantidad de retransmisiones (correlación de 1,000), a razón de 692 B por retransmisión. Por corrida hubo 0, unas 43 o unas 87 retransmisiones, es decir, el fenómeno ocurrió en casi todas las conexiones de una corrida o en ninguna. La causa más probable es que Linux guarda métricas TCP por destino, como el RTT y su variación, entre conexiones: un RTT alto heredado de una corrida anterior eleva el RTO inicial por encima de la demora del ESP32 y evita la retransmisión. Eso explica que con 50 y 200 ms de RTT agregado nunca hubo retransmisiones (2296 ± 1 B), mientras que sin latencia agregada el resultado dependió de qué corrida se había ejecutado antes. Esta explicación no se aisló con un experimento aparte. Por eso, en el análisis se toma como valor de referencia de HTTP con una conexión por lectura el de 2296 B, que no incluye retransmisiones innecesarias.

Por qué MQTT no sufre el bloqueo. esp-mqtt arma cada PUBLISH completo en un *buffer* y lo escribe con un solo `send()`, así que cada lectura es un único segmento y Nagle no tiene nada que retener. Además, con QoS 0 el broker nunca responde, no detecta una conexión de ida y vuelta y confirma enseguida; con QoS 1, el ACK viaja dentro del PUBACK.

Nagle con pérdida: por qué QoS 1 tuvo mejor cola que QoS 0. MQTT sí se ve afectado por Nagle cuando se pierde un ACK. Con QoS 0, el broker solo envía ACK sin datos. Si uno se pierde, el ESP32 no lo sabe hasta que vence su propio RTO, de al menos 1 s, y mientras tanto Nagle retiene la lectura siguiente, porque hay datos sin confirmar. En las capturas con 5 % de pérdida, el ESP32 reenvió en promedio 54 segmentos por corrida que ya habían llegado al servidor con QoS 0, contra 4,6 con QoS 1, lo que coincide con perder el 5 % de los 960 ACK que bajan en una corrida. Con QoS 1, el ACK viaja dentro del PUBACK, que lleva datos: si se pierde, lo retransmite Linux con su RTO de unos 200 ms, mucho antes que lwIP. Por eso, con 5 % de pérdida, el p99 de QoS 1 fue de 1,7 s contra 3,0 s de QoS 0, aunque la diferencia en el p95 no resultó significativa ($p = 0{,}16$).

BORRADOR

En síntesis, parte de la diferencia entre HTTP y MQTT no se debe al protocolo de aplicación, sino a cómo cada cliente usa TCP. El mismo HTTP cambia su latencia en un orden de magnitud según cómo el cliente reparte los datos en llamadas a `send()`: con Nagle desactivado, HTTP con conexión reutilizada baja a 5 ms, del mismo orden que MQTT, aunque sigue enviando 3,1 veces más bytes (515 B contra 165 B). En el experimento principal se mantuvo el comportamiento por defecto, con Nagle activo, porque es el que tendría un ESP32 sin ajustes.

Pérdida de paquetes

Con hasta 5 % de pérdida no se perdió ninguna lectura, porque TCP retransmitió cada segmento perdido y ninguna conexión se cortó. MQTT QoS 0 solo perdería lecturas si se cortara la conexión, por ejemplo al vencer el keep-alive, por un *reset* o porque el nodo se reconecta a la red, y eso no ocurrió con pérdidas aleatorias en corridas de 120 s. En estas condiciones, QoS 1 no mejoró la entrega y solo agregó duplicados: esp-mqtt reenvía el PUBLISH si el PUBACK no llega en 1 s, aunque TCP todavía esté recuperando el segmento original. Es lo que se plantea en la Sección 1.3: mientras la conexión se mantiene, la confiabilidad la aporta TCP, y la ventaja de QoS 1 aparece cuando la conexión se corta.

Lo que se degradó fue la cola de la latencia, y la Figura 5 muestra escalones en valores fijos:

- Entre 1 y 1,5 s, el RTO de lwIP: un segmento perdido se recupera recién cuando vence el temporizador de retransmisión, que lwIP maneja con un reloj de 500 ms. No hay retransmisión rápida, porque requiere tres ACK duplicados y con una lectura por segundo nunca hay tantos segmentos en vuelo.
- Entre 0,05 y 0,5 s, el bloqueo de cabeza de línea: la lectura que sigue a una perdida sale 1 s después y queda retenida hasta que la anterior se recupera. Por eso las lecturas lentas vienen de a pares.
- Por encima de 2 s, pérdidas repetidas del mismo segmento, con el RTO duplicado en cada intento [40].

HTTP salió peor por su esquema de solicitud y respuesta en serie. Cualquier paquete perdido, en cualquier sentido, frena el envío hasta que se recupera, y las lecturas siguientes esperan en la cola. Además, cada lectura usa 6 paquetes (22 con una conexión por lectura), contra 2 o 3 de MQTT, así que tiene más chances de perder alguno. La diferencia entre QoS 0 y QoS 1 se debe a la combinación de Nagle con los ACK perdidos, descrita en el apartado anterior.

El costo en bytes de la pérdida fue bajo, entre 1 % y 13 %. El mayor fue el de HTTP con una conexión por lectura, al que se suman las retransmisiones innecesarias del handshake.

Serialización

El ahorro de Protocol Buffers fue de unos 54 B por lectura en todos los caminos, porque el mensaje es el mismo. En HTTP fue 6 B menor, porque `application/json` es más corto que `application/x-protobuf` en el header `Content-Type`. La reducción del payload, del 75 %, está en línea con la de los estudios citados en el marco teórico [28], [29].

En términos relativos, en cambio, el efecto depende de cuánto pesa el resto de la lectura. JSON aumentó un 33 % los bytes de MQTT QoS 0, un 9 % los de HTTP con conexión reutilizada y un 3 % los de HTTP con una conexión por lectura. Esto confirma lo anticipado en el marco teórico: una vez que el transporte reduce sus encabezados al mínimo, la serialización pasa a tener un peso importante. Aun así, la elección del transporte pesa más: pasar de HTTP con conexión reutilizada a MQTT QoS 0 ahorra 390 B por lectura, más de siete veces lo que ahorra Protocol Buffers.

BORRADOR

Contraste de hipótesis

La Tabla 22 contrasta las hipótesis del planteo con los resultados.

Tabla 22: Contraste de las hipótesis con los resultados.

Hipótesis	**Resultado**
MQTT envía menos bytes por lectura que HTTPS, y la diferencia crece cuando el período de envío es corto.	**Se confirma, con un matiz.** MQTT envió menos bytes en todos los casos medidos (12 lecturas por conexión o más), y su ventaja frente a HTTP con conexión reutilizada creció al acortar el período: 1,7 veces a 10 s, 3,4 a 1 s y 3,6 a 100 ms. Pero si la conexión dura menos de tres o cuatro lecturas, la relación se invierte (Tabla 19).
HTTPS con conexión reutilizada reduce gran parte de la diferencia, pero los headers HTTP siguen pesando más que la cabecera MQTT.	**Se confirma.** Reutilizar la conexión bajó HTTPS de 2296 a 555 B por lectura (−76 %), pero siguió costando 3,4 veces MQTT QoS 0, por los headers (134 B contra 13 B), la respuesta obligatoria y los ACK adicionales, a los que se suman 40 ms por la espera de Nagle.
HTTPS con una conexión por request es el caso más costoso, sobre todo con TLS y latencia alta.	**Se confirma.** Fue el más caro en bytes y en latencia, y el único que con 200 ms de RTT pagó unos 5,5 RTT por lectura.
Con pérdida, MQTT QoS 1 mantiene la entrega a costa de duplicados, mientras que QoS 0 pierde lecturas.	**Se confirma a medias.** QoS 1 generó duplicados (5,4 % de las lecturas con 5 % de pérdida), pero QoS 0 no perdió ninguna lectura: TCP recuperó todo porque ninguna conexión se cortó. QoS 1 sí tuvo mejor cola de latencia.
Protobuf reduce el payload en ambos caminos, con efecto relativo mayor en MQTT, donde el payload es la mayor parte del mensaje.	**Se confirma, aunque no por la razón prevista.** El ahorro absoluto fue el mismo en todos los caminos (unos 54 B) y el relativo fue mayor en MQTT (33 %), pero porque su lectura es más chica, no porque el payload sea la mayor parte del mensaje: es el 11 %.

Limitaciones

- **Criptografía emulada.** Los handshakes TLS tardaron entre 3 y 3,8 s en QEMU, mucho más que en un ESP32 físico, por lo que los tiempos de conexión y la latencia de HTTP con una conexión por lectura deben leerse en términos relativos. Los bytes y los paquetes, en cambio, son los del firmware real.
- **Reloj.** Las latencias con RTT agregado se corrigieron restando RTT/2. Sin RTT agregado, la incertidumbre absoluta es de 1 ms, del orden de la latencia de MQTT, aunque no afecta las comparaciones entre transportes.
- **Estado compartido entre corridas.** Las métricas TCP que Linux guarda por destino afectaron las retransmisiones innecesarias, que solo pesan en HTTP con una conexión por lectura.
- **Un factor a la vez.** No se midieron interacciones entre factores, como pérdida y período o latencia y JSON, salvo la de Nagle y TLS.
- **Configuración concreta.** Otras configuraciones darían otros números, en especial para el costo fijo por conexión y la latencia de HTTP: Nagle activo en HTTP, Mosquitto enviando el certificado de la autoridad de certificación (411 B más por handshake), sin reanudación de sesión TLS, TLS 1.2 (TLS 1.3 ahorraría un RTT en el handshake y 7 B por registro), MQTT 3.1.1 y un cliente HTTP que abre la conexión en el primer envío.
- **Red emulada.** Se usó Ethernet emulado y pérdida aleatoria independiente, sin ráfagas. Con pérdidas en ráfaga o cortes de conexión, que no formaron parte de la matriz, las sesiones

BORRADOR

podrían caerse, y entonces QoS 0 sí perdería lecturas. Las corridas de 120 s tampoco permiten observar el keep-alive ni las reconexiones.
- **Un solo sentido.** Solo se midió la telemetría del nodo al servidor. El envío de comandos del servidor al nodo, donde HTTP necesitaría consultas periódicas, quedó fuera de la comparación.

Implicaciones

La Tabla 23 lleva los resultados a la escala de un despliegue. Supone una conexión que no se corta, por lo que usa el costo por lectura v de la Tabla 19, y una lectura cada 10 s por nodo (8640 por día). El keep-alive no suma tráfico, porque el período es menor que los 60 s configurados. Estos volúmenes son los que determinan el plan de datos necesario en un despliegue sobre una red celular, y sirven de dato de entrada para el mapeo tecnológico.

Tabla 23: Proyección del tráfico con una lectura cada 10 s.

Transporte (TLS)	**Por nodo y por día**	**1000 nodos, por mes**
MQTT QoS 0	1,2 MB	36 GB
MQTT QoS 1	1,7 MB	52 GB
HTTP, conexión reutilizada	4,5 MB	136 GB
HTTP, conexión por lectura	19,8 MB	595 GB

Para un sistema de telemetría como el estudiado, los resultados sugieren cuatro criterios:

- Lo que más importa es mantener la conexión abierta. La ventaja de MQTT depende de que la conexión dure más que unas pocas lecturas; para dispositivos que duermen y se reconectan en cada envío, pesan más la reanudación de sesión TLS o TLS 1.3 que la elección del protocolo de aplicación.
- Si se usa HTTP, conviene reutilizar la conexión y desactivar Nagle, o escribir cada solicitud con una sola llamada a `send()`. Con eso la latencia queda en el mismo orden que la de MQTT, aunque el costo en bytes sigue siendo unas tres veces mayor.
- Mientras la conexión TCP se mantiene, QoS 1 no mejora la entrega y agrega duplicados, por lo que la aplicación debe poder descartarlos, por ejemplo con el número de secuencia. Su valor aparece ante cortes de conexión, que este experimento no midió.
- Protocol Buffers ahorra lo mismo en todos los transportes, pero su efecto relativo es mayor justamente en MQTT, donde el resto de la lectura ya es chico.

|

5.3. Experimento 2: sincronización de osciladores acoplados

La sincronización colectiva aparece cuando un conjunto de osciladores, cada uno con su propio ritmo, termina funcionando a una frecuencia común sin que nadie la imponga. Las diferencias entre las frecuencias naturales no desaparecen, pero la interacción alcanza para compensarlas. En biología se observa en las células marcapasos del corazón, en las células del núcleo supraquiasmático que regulan los ritmos circadianos, en suspensiones de levaduras, en las luciérnagas que destellan al unísono y en los grillos que cantan a la vez. En física e ingeniería aparece en arreglos de láseres, en osciladores de microondas y en uniones de Josephson [69].

La idea del experimento es reproducir ese fenómeno con una red de nodos que se comportan como luciérnagas. Cada ESP32 ejecuta un único oscilador, con una frecuencia natural distinta a la de los demás: su fase avanza a su propio ritmo y cada vuelta completa equivale a un destello. Si los nodos no se comunican, cada uno destella a su frecuencia. Si cada uno conoce la fase

BORRADOR

de los otros y corrige la propia, los osciladores quedan acoplados. Cuando esa corrección es suficientemente intensa, el grupo se engancha: todos pasan a girar a una misma frecuencia, sin ningún nodo que actúe de coordinador.

Esa sincronía existe solo a través de los mensajes que intercambian los nodos, y por eso sirve para evaluar el stack. El modelo matemático predice cómo debería comportarse el grupo si la comunicación fuera ideal, de modo que la sincronización funciona como una carga de referencia con un comportamiento teórico conocido. Comparar esa predicción con lo que se mide en los nodos permite observar qué tan bien cumple el transporte con lo que la aplicación necesita. El experimento de la Sección 5.2 mide el costo de enviar lecturas desde muchos nodos hacia un único servidor. En este, en cambio, cada nodo produce un dato que les interesa a todos los demás y consume los de todos ellos, que es la forma de circulación de la información para la que se propone el modelo publicador/suscriptor (Sección 1).

La prueba se implementa sobre la plataforma común, con un oscilador por cada ESP32 emulada. Cada nodo es un cliente MQTT que abre una única conexión TCP hacia el broker, y por ella publica su fase y recibe las de los demás (Sección 2). Periódicamente integra su fase, la serializa con Protocol Buffers junto con su identificador y una marca de tiempo, y la envía como carga útil de un paquete PUBLISH (Sección 4) en un tópico propio, por ejemplo `kuramoto/phase/7`. Además, se suscribe al filtro `kuramoto/phase/+`, cuyo comodín coincide con el nivel del identificador (Sección 2), de modo que el broker le entrega las fases de los otros nodos. Con ellas calcula la corrección que aplica a su propia evolución, y el ciclo se repite. Todos los tópicos de esa jerarquía transportan el mismo tipo de mensaje, por lo que el tópico identifica también el esquema con el que se interpreta cada carga útil, y un único archivo `.proto` funciona como contrato entre todos los participantes (Sección 4).

Este esquema aprovecha dos de las tres dimensiones de desacoplamiento presentadas en la Sección 1. Con el desacoplamiento en el espacio, ningún nodo necesita conocer la dirección de los otros ni cuántos son: solo el broker tiene una dirección conocida, y sumar un nodo no requiere modificar a los que ya están. Con el desacoplamiento de sincronización, publicar no detiene la integración de la fase, y las fases de los demás llegan mientras el nodo sigue ejecutando su ciclo. El desacoplamiento en el tiempo, en cambio, no se utiliza. Las fases se publican con QoS 0 porque cada una queda reemplazada por la siguiente, igual que en los datos de alta frecuencia de los casos de uso (Sección 4.5), y un nodo que se desconecta no recibe lo publicado mientras estuvo ausente (Sección 2), algo que tampoco necesita, ya que una fase vieja no le sirve.

La aplicación también expone los costos del stack. El broker agrega un salto a cada mensaje y concentra la carga de todos los participantes (Sección 1): una fase recorre dos conexiones TCP antes de llegar a otro nodo, y cada publicación se entrega a los otros $N-1$, lo que da $N(N-1)$ entregas por cada ronda de publicaciones. Si el broker falla, además, la comunicación se interrumpe para todo el grupo (Sección 4.5). Como los mensajes son cortos y muy frecuentes, importan tanto la codificación binaria de Protocol Buffers (Sección 3) como el encabezado fijo de dos bytes de MQTT en los paquetes pequeños (Sección 2). El costo por mensaje que mide el experimento de la Sección 5.2 aquí se multiplica por la cantidad de entregas.

5.3.1. Planteo

El modelo. El experimento se basa en el modelo de osciladores acoplados que propuso Kuramoto [70]. Para presentarlo se sigue la revisión de Strogatz [69], y para el caso de una cantidad finita de osciladores, el trabajo de Verwoerd y Mason [71]. Se consideran N osciladores, cada uno descrito solo por su fase y acoplado a todos los demás con el mismo peso. El oscilador

BORRADOR

i tiene una frecuencia natural ω_i, en rad/s, que es fija, y una fase θ_i, en radianes, que evoluciona según

$$\dot{\theta}_i = \omega_i + \frac{K}{N}\sum_{j=1}^{N}\sin\big(\theta_j - \theta_i\big), \quad i = 1, ..., N \tag{2}$$

donde el punto indica la derivada respecto del tiempo. El acoplamiento es la influencia que los osciladores ejercen entre sí, representada por el término de la sumatoria, que depende de las diferencias de fase. Su intensidad es K, en rad/s: con $K = 0$, cada oscilador avanza a su frecuencia natural sin afectar a los demás. El acoplamiento cambia la velocidad instantánea $\dot{\theta}_i$: si el oscilador j va adelantado, $\sin\big(\theta_j - \theta_i\big) > 0$ y el oscilador i se acelera; si va atrasado, se frena.

Parámetro de orden y campo medio. El grado de sincronía se mide con el centroide de las fases sobre el círculo unitario:

$$re^{\mathrm{i}\psi} = \frac{1}{N}\sum_{j=1}^{N} e^{\mathrm{i}\theta_j} \tag{3}$$

donde i es la unidad imaginaria. El módulo r, entre 0 y 1, se denomina parámetro de orden: vale 0 si las fases están dispersas y 1 si coinciden. El ángulo ψ es la fase media del grupo [69]. La Figura 9 muestra ambas situaciones.

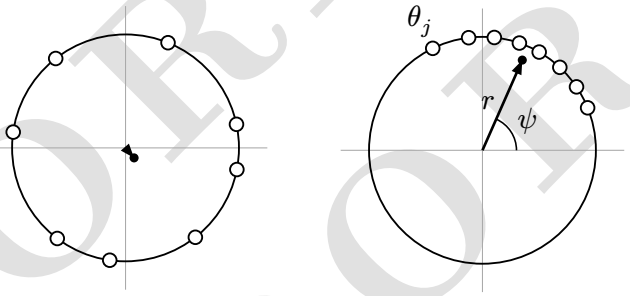


Figura 9: Fases de ocho osciladores sobre el círculo unitario (círculos vacíos) y su centroide $re^{\mathrm{i}\psi}$ (punto negro). A la izquierda, fases dispersas; a la derecha, fases agrupadas.

Si se multiplica la Ecuación 3 por $e^{-\mathrm{i}\theta_i}$ y se toma la parte imaginaria, resulta

$$r\sin(\psi - \theta_i) = \frac{1}{N}\sum_{j=1}^{N}\sin\big(\theta_j - \theta_i\big) \tag{4}$$

Al reemplazar esa suma en la Ecuación 2 se obtiene la forma de campo medio [69], que vale para cualquier N:

$$\dot{\theta}_i = \omega_i + Kr\sin(\psi - \theta_i) \tag{5}$$

Cada oscilador es atraído hacia la fase media con una fuerza Kr. Es la forma que se implementa en los nodos. El promedio de fases tiene que ser circular: se suman los vectores $\big(\cos\theta_j, \sin\theta_j\big)$ y se toman el módulo y el ángulo de la suma. Promediar los ángulos directamente falla en el corte entre 2π y 0.

Frecuencia común. Se dice que el grupo está enganchado cuando todos los osciladores giran a una misma frecuencia, que se denota Ω, y sus diferencias de fase permanecen constantes. Para hallar Ω, se suma la Ecuación 2 sobre todos los osciladores:

BORRADOR

$$\sum_{i=1}^{N} \dot{\theta}_i = \sum_{i=1}^{N} \omega_i + \frac{K}{N} \sum_{i=1}^{N} \sum_{j=1}^{N} \sin\bigl(\theta_j - \theta_i\bigr) \tag{6}$$

La suma doble vale cero. Como el seno es impar, $\sin\bigl(\theta_j - \theta_i\bigr) = -\sin\bigl(\theta_i - \theta_j\bigr)$, de modo que el término del par (i, j) se cancela con el del par (j, i), y los términos con $i = j$ ya son nulos. Queda entonces

$$\sum_{i=1}^{N} \dot{\theta}_i = \sum_{i=1}^{N} \omega_i \tag{7}$$

Esta relación vale siempre, haya o no enganche: el acoplamiento redistribuye las velocidades entre los osciladores, pero no cambia su suma. Si el grupo está enganchado, $\dot{\theta}_i = \Omega$ para todo i, y la Ecuación 7 da $N\Omega = \sum_i \omega_i$, es decir

$$\Omega = \overline{\omega} = \frac{1}{N} \sum_{i=1}^{N} \omega_i \tag{8}$$

Verwoerd y Mason llegan al mismo resultado: en un estado enganchado, cada oscilador avanza a una velocidad constante igual a la media de las frecuencias naturales [71]. A partir de aquí se trabaja con las desintonías $\Delta_i = \omega_i - \overline{\omega}$, que indican cuánto se aparta cada frecuencia natural de la media.

Límite de infinitos osciladores. Se denomina acoplamiento crítico K_c al valor de K a partir del cual aparece la sincronización. El primer resultado sobre este umbral es el análisis de Kuramoto para $N \to \infty$, que se resume siguiendo a Strogatz [69]. Las desintonías se distribuyen según una densidad de probabilidad $g(\Delta)$ simétrica y con un único máximo en $\Delta = 0$, y se buscan soluciones estacionarias, con r constante y ψ girando a la velocidad uniforme Ω. En lo que sigue, θ_i denota la fase medida desde ψ, con lo que la Ecuación 5 se reduce a

$$\dot{\theta}_i = \Delta_i - Kr \sin\theta_i \tag{9}$$

Según el valor de Δ_i, los osciladores se separan en dos grupos. Los que cumplen $|\Delta_i| \le Kr$ tienden a un punto fijo, en el que su fase relativa deja de cambiar, con $\Delta_i = Kr \sin\theta_i$ y $|\theta_i| \le \pi/2$, y quedan enganchados. Los que tienen $|\Delta_i| > Kr$ no alcanzan ningún punto fijo y derivan alrededor del círculo.

El valor de r tiene que ser coherente con estas dos poblaciones. La simetría de g anula la contribución de los osciladores a la deriva al centroide, y también la parte imaginaria de la contribución de los enganchados. Solo queda el promedio de $\cos\theta$ sobre los enganchados, donde $\theta(\Delta)$ es la fase que cumple $\Delta = Kr \sin\theta$, y con ese cambio de variable resulta

$$r = \int_{-Kr}^{Kr} \cos\theta(\Delta)\, g(\Delta)\, \mathrm{d}\Delta = Kr \int_{-\pi/2}^{\pi/2} \cos^2\theta\, g(Kr\sin\theta)\, \mathrm{d}\theta \tag{10}$$

Esta ecuación siempre admite la solución $r = 0$, que corresponde al estado incoherente, en el que las fases están repartidas sin un ritmo común. Las demás soluciones, que forman una rama de sincronización parcial, se obtienen dividiendo por r:

$$1 = K \int_{-\pi/2}^{\pi/2} \cos^2\theta\, g(Kr\sin\theta)\, \mathrm{d}\theta \tag{11}$$

BORRADOR

El acoplamiento crítico es el valor en el que esa rama nace desde $r = 0$. Al tomar $r \to 0^+$ en la Ecuación 11, g se evalúa en 0 y la integral de $\cos^2\theta$ vale $\pi/2$, de modo que $1 = K_c g(0)\pi/2$, es decir

$$K_c = \frac{2}{\pi g(0)} \tag{12}$$

Para una distribución de Lorentz de ancho γ, $g(\Delta) = \gamma/(\pi(\gamma^2 + \Delta^2))$ y $g(0) = 1/(\pi\gamma)$, por lo que $K_c = 2\gamma$ y, por encima del umbral, $r = \sqrt{1 - K_c/K}$ [69]. Estas expresiones valen en el límite de infinitos osciladores. Con N finito, r fluctúa con una amplitud del orden de $N^{-1/2}$ [69]. Por eso, en este trabajo la Ecuación 12 queda solo como contexto, y K_c se calcula como se muestra a continuación.

Umbral exacto con N finito. Verwoerd y Mason estudian el mismo modelo para una cantidad finita de osciladores [71], y definen el acoplamiento crítico como el menor K para el que existe un estado enganchado. No es el mismo umbral que el de la Ecuación 12: aquel marca el comienzo de la sincronización parcial, y este, el enganche de todos los osciladores.

En un estado enganchado, $\dot{\theta}_i = \overline{\omega}$ para todo i. Reemplazando en la Ecuación 5 se obtiene

$$\Delta_i = u\,\mathrm{sin}(\theta_i - \psi), \quad u = Kr \tag{13}$$

La magnitud u es la fuerza con la que el grupo atrae a cada oscilador, y como el seno no supera 1 en módulo, cada oscilador necesita $|\Delta_i| \leq u$. De la parte real de la Ecuación 3, multiplicada por $e^{-\mathrm{i}\psi}$, sale $r = 1/N \sum_i \cos(\theta_i - \psi)$, y cada coseno vale $\pm\sqrt{1 - (\Delta_i/u)^2}$. Verwoerd y Mason demuestran que existe un estado enganchado si y solo si la ecuación resultante tiene solución con todos los signos positivos [71], lo que da la condición de autoconsistencia:

$$r = \frac{u}{K} = f(u) = \frac{1}{N}\sum_{i=1}^{N}\sqrt{1 - \left(\frac{\Delta_i}{u}\right)^2}, \quad u \geq \max_i |\Delta_i| \tag{14}$$

Despejando, $K = u/f(u)$, y el menor acoplamiento para el que hay solución es

$$K_c = \min_{u \geq \max_i|\Delta_i|} \frac{u}{f(u)} \tag{15}$$

Al derivar $u/f(u)$ e igualar a cero se obtiene $f(u) = uf'(u)$, y al desarrollar $f'(u)$, el mínimo queda en el valor u^* que cumple

$$2 \cdot \frac{1}{N}\sum_{i=1}^{N}\sqrt{1 - \left(\frac{\Delta_i}{u^*}\right)^2} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\sqrt{1 - \left(\frac{\Delta_i}{u^*}\right)^2}} \tag{16}$$

Esa ecuación tiene una única solución [71], y entonces $K_c = u^*/f(u^*)$. El resultado garantiza que el estado enganchado existe, pero no que el sistema lo alcance desde cualquier condición inicial, porque el trabajo no aborda la estabilidad. Ese punto se comprueba mediante la simulación numérica del modelo.

Con esto se calculan, para un conjunto dado de frecuencias naturales, el umbral de enganche total, el valor de r y el desfasaje de cada oscilador respecto de la fase media, que es $\theta_i - \psi = \arcsin(\Delta_i/u)$. Con $N = 2$ y $\Delta\omega = \omega_2 - \omega_1$ se obtiene $K_c = |\Delta\omega|$.

Enganche parcial y normalización. Con $N \geq 3$ y frecuencias asimétricas, puede engancharse un subgrupo mientras otro oscilador deriva. Este caso no tiene fórmula simple y se estudia por

BORRADOR

simulación, comparando las frecuencias efectivas de cada oscilador, es decir, la velocidad media a la que avanza su fase. Como el acoplamiento se divide por N, al agregar osciladores con el mismo K cada interacción individual se debilita.

5.3.2. Implementación

5.3.3. Resultados

5.3.4. Análisis de resultados

6. Mapeo tecnológico

Los experimentos se realizaron sobre placas emuladas, una red simulada y un broker corriendo en la misma computadora que el emulador. Llevar el sistema a un despliegue real requiere reemplazar cada una de esas piezas por un equipo o un servicio que se pueda comprar o contratar. A continuación se analizan los costos que este paso implica para el escenario del Experimento 1 (Sección 5.2), con ocho nodos que envían una lectura cada 10 s, la misma frecuencia que se usó para proyectar el tráfico en la Tabla 23. El criterio no es buscar la alternativa de mayor prestación, sino una solución viable que equilibre la calidad técnica, el plazo de implementación y el presupuesto. Los precios, a fin de generalizar, se expresan en dólares estadounidenses, son aproximados y corresponden a noviembre de 2026, sin impuestos ni costos de importación.

6.1. Nodos

Cada nodo se construye sobre el mismo microcontrolador que se emuló en los experimentos, el ESP32 de Espressif. Al tratarse del mismo chip, el firmware se compila sin cambios, salvo por la interfaz de red. La emulación conectaba cada nodo por Ethernet, mientras que el chip físico trae integrada un módulo Wi-Fi. El ESP32 se comercializa montado en placas de distintos fabricantes, y una placa de referencia como la ESP32-DevKitC, del propio Espressif, cuesta alrededor de USD 10 [72].

Cuando el lugar de instalación no cuenta con una red Wi-Fi, el nodo necesita conectarse a la red celular. Para ese caso se considera la placa LilyGO T-A7670SA, que combina el mismo microcontrolador con un módem celular y cuesta alrededor de USD 28 [73]. El módem opera en LTE categoría 1, una variante de LTE pensada para dispositivos que transmiten a baja velocidad y que funciona sobre la red 4G común de los operadores. Existen variantes más eficientes en consumo, como LTE-M, pero su disponibilidad en Argentina no está confirmada para todos los operadores, por lo que la categoría 1 resulta la opción de menor riesgo.

6.2. Conectividad

Con Wi-Fi, el nodo aprovecha la red ya existente en el lugar y no agrega costo de conectividad. Como es el dispositivo el que abre la conexión hacia el broker, no necesita aceptar conexiones entrantes y funciona detrás del NAT y del firewall de esa red (Sección 2.1).

Con conectividad celular, en cambio, cada nodo requiere un plan de datos. Para aplicaciones industriales o de Internet de las Cosas no corresponden los planes de telefonía personal, sino los planes M2M (*machine to machine*), pensados para dispositivos. Los operadores locales no publican las tarifas de estos planes, pero los proveedores globales de tarjetas SIM para IoT sí lo hacen. Su precio depende principalmente del volumen de datos transferido, al que algunos suman un cargo fijo mensual por tarjeta.

BORRADOR

En este punto, las mediciones del Experimento 1 se traducen directamente en dinero. El operador factura los bytes IP que atraviesan su red, con los encabezados de TCP y TLS incluidos, que son los que se midieron. Con MQTT sobre TLS, una lectura ocupa unos 140 B (Tabla 19), lo que equivale a unos 36 MB por mes en cada nodo. Con las tarifas de estos proveedores, ese volumen cuesta alrededor de USD 2 por nodo y por mes [74], [75], [76]. Las decisiones analizadas en el experimento se reflejan entonces en la factura. Enviar el mismo contenido en JSON aumentaría ese volumen en alrededor de un 40 %, y usar HTTP con conexión reutilizada lo multiplicaría casi por cuatro. Este volumen es, de todos modos, un piso, porque en una red celular real se suman las reconexiones, cada una con un handshake TLS de unos 3 KB, las consultas DNS y los mensajes de keep-alive.

Las redes LoRa, presentadas entre los casos de uso (Sección 4.4), son otra alternativa para zonas sin cobertura celular, pero cambian la arquitectura del nodo y quedan fuera de este relevamiento.

6.3. Broker

Para el broker se comparan dos alternativas. La primera es un servidor propio que ejecute Mosquitto, el mismo broker usado en los experimentos. Un servidor virtual privado (VPS), es decir, una máquina virtual alquilada a un proveedor de nube, de la menor capacidad disponible alcanza holgadamente para ocho nodos y cuesta alrededor de USD 5 por mes [77], mientras que Mosquitto es software libre y no tiene costo de licencia. Conviene alojarlo en la región del proveedor más cercana a los dispositivos, que en este caso es São Paulo, porque así el trayecto atraviesa menos sistemas autónomos y queda menos expuesto a la convergencia de BGP (Sección 2.3).

La segunda alternativa es un servicio administrado, en el que el proveedor opera el broker y cobra según el uso. AWS IoT Core, por ejemplo, costaría alrededor de USD 2 por mes en este escenario, pero su forma de facturar contrasta con la del plan de datos. El servicio cobra por mensaje y mide cada uno en bloques de 5 KB [78], de modo que una lectura de 18 B en Protocol Buffers cuesta lo mismo que una de 72 B en JSON. Con este tipo de tarifa, achicar el mensaje no reduce el costo, y lo único que lo reduce es enviar menos mensajes.

Se elige el servidor propio. A esta escala la diferencia de precio es mínima, y la decisión la definen el plazo y la continuidad con lo ya probado, ya que se trata del mismo broker y la misma configuración del experimento, sin necesidad de adaptar el sistema a un servicio distinto.

6.4. Costo del sistema

La Tabla 24 resume los componentes relevados.

Tabla 24: Componentes del sistema y precios aproximados en dólares estadounidenses (noviembre de 2026, sin impuestos).

Componente	**Equipo o servicio**	**Precio aproximado**
Nodo con Wi-Fi	Placa con ESP32 (por ejemplo, ESP32-DevKitC)	USD 10
Nodo celular	Placa LilyGO T-A7670SA (ESP32 y módem LTE)	USD 28
Datos celulares	Plan IoT de un proveedor global de SIM	USD 2 por mes por nodo, más unos USD 3 por SIM
Broker	VPS en São Paulo con Mosquitto	USD 5 por mes

BORRADOR

Con conectividad Wi-Fi, poner en marcha los ocho nodos requiere unos USD 80 en placas y USD 5 por mes para el broker. Con conectividad celular, la inversión inicial sube a unos USD 250 entre placas y tarjetas SIM, y el costo mensual a unos USD 21, de los cuales 16 corresponden a los datos. En ambos casos, el sistema estudiado puede implementarse con equipos de bajo costo y de amplia disponibilidad. La principal incertidumbre no es el precio sino la cobertura. Ninguno de los proveedores globales con tarifas públicas declara operar sobre la red de Personal, por lo que antes de un despliegue habría que verificar qué operador tiene señal en el lugar de instalación.

6.5. Escalabilidad

El escenario de ocho nodos es solo un caso de uso entre muchos posibles, y cada componente de la Tabla 24 crece con una variable distinta.

- Las placas y las tarjetas SIM crecen con la cantidad de nodos y se pagan una sola vez.
- El cargo fijo mensual de las tarjetas crece con la cantidad de nodos.
- Los datos celulares crecen con la cantidad de nodos, la frecuencia de envío y el tamaño de cada lectura.
- El broker contratado crece con la cantidad de mensajes, sin importar su tamaño.
- El broker propio crece de a escalones, según el tamaño de servidor que exige la carga, y por transferencia solo cuando el tráfico supera el volumen incluido en el plan.

Según el caso de uso, el costo total queda dominado por uno u otro de estos componentes.

Cuando se supervisan variables que cambian lentamente, como una temperatura o el nivel de un tanque, basta con enviar una lectura por minuto o incluso menos. Un nodo celular con esa frecuencia transfiere unos 6 MB por mes y, con las tarifas relevadas, paga alrededor de USD 1,2, de los cuales USD 1 corresponde al cargo fijo de la tarjeta [74]. En este caso, reducir el tamaño de los mensajes casi no modifica la factura, y el costo lo definen el hardware y la cantidad de nodos.

La situación se invierte en una flota grande conectada por celular que envía datos con frecuencia, como la de mil nodos con una lectura cada 10 s considerada en la Tabla 23. Los datos pasan a ser el componente principal, del orden de USD 2000 por mes para toda la flota, y cada decisión del stack se multiplica por la cantidad de nodos. Enviar JSON en lugar de Protocol Buffers aumentaría el volumen de datos de toda la flota en alrededor de un 40 %, y usar HTTP en lugar de MQTT lo multiplicaría casi por cuatro, con el consiguiente aumento de la parte del costo que depende del volumen. También cambia la elección del broker. Con esa carga, un servicio que cobra por mensaje costaría alrededor de USD 260 por mes. Un servidor propio también tiene costos que crecen con el uso, por el cómputo y la transferencia de datos, pero lo hacen de a escalones. El tráfico de toda la flota, de algunas decenas de gigabytes por mes, queda muy por debajo del volumen que incluyen incluso los planes más chicos, y aun un servidor varias veces más grande que el de la Tabla 24 cuesta una fracción del servicio contratado. El servidor propio pasa a ser así la alternativa más económica, a pesar del trabajo que implica operarlo. A esta escala, de todos modos, los precios de lista pierden representatividad, porque los operadores negocian los planes M2M con descuentos por volumen, y las cifras deben leerse como órdenes de magnitud.

En un depósito con robots móviles (Sección 4.2), el tráfico también es intenso, porque cada robot publica su posición y su estado de forma continua, pero viaja por la red Wi-Fi de la instalación, que no cobra por byte. El costo de comunicación se reduce al broker y resulta

BORRADOR

despreciable frente al valor de los robots. En este caso, la ventaja del stack no se mide en dinero, sino en la latencia y en el uso de la red.

La mensajería (Sección 4.1) se ubica en el extremo opuesto. Los datos viajan por el plan de telefonía de cada usuario, que es quien paga el tramo celular, y el costo de la empresa se concentra en mantener conexiones persistentes con millones de dispositivos móviles. A esa escala, el broker deja de ser un producto que se contrata y pasa a desarrollarse a medida, como muestra la evolución de Messenger.

Estos casos muestran que el ahorro de bytes del stack se convierte en dinero solo cuando la conexión se cobra por volumen y la frecuencia de envío es alta. En los demás, la elección de MQTT y Protocol Buffers se justifica por la latencia y el uso de la red más que por el costo.

BORRADOR

Conclusiones

BORRADOR

Bibliografía

[1] OpenRobOps/oro, «oro: OpenRobOps open-source robot fleet management and operations platform». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/OpenRobOps/oro

[2] D. E. Comer, *Internetworking with TCP/IP, Volume One: Principles, Protocols, and Architecture*, 6.ª ed. Upper Saddle River, NJ: Pearson, 2014.

[3] J. F. Kurose y K. W. Ross, *Redes de computadoras. Un enfoque descendente*, 7.ª ed. Madrid: Pearson Educación, 2017.

[4] P. T. Eugster, P. A. Felber, R. Guerraoui, y A.-M. Kermarrec, «The many faces of publish/subscribe», *ACM Computing Surveys*, vol. 35, n.º 2, pp. 114-131, 2003, doi: 10.1145/857076.857078.

[5] W. Stallings, *Comunicaciones y redes de computadores*, 7.ª ed. Madrid: Pearson Educación, 2004.

[6] A. Banks, E. Briggs, K. Borgendale, y R. Gupta, Eds., «MQTT version 5.0», OASIS, OASIS Standard, 2019. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[7] Google, «Overview». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/overview/

[8] protocolbuffers, «protobuf: Protocol Buffers - Google's data interchange format». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://github.com/protocolbuffers/protobuf

[9] MQTTX Team, «How to Publish and Receive Protobuf Messages within MQTT?». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://www.emqx.com/en/blog/how-to-publish-and-receive-protobuf-messages-within-mqtt

[10] Google, «Encoding». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/encoding/

[11] Google, «Language Guide (proto 3)». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/proto3/

[12] Google, «Techniques». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://protobuf.dev/programming-guides/techniques/

[13] Amazon Web Services, «Working with binary payloads». Accedido: 28 de octubre de 2026. [En línea]. Disponible en: https://docs.aws.amazon.com/iot/latest/developerguide/binary-payloads.html

[14] Meshtastic, «MQTT: Integrations Overview». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://meshtastic.org/docs/software/integrations/mqtt/

[15] Eclipse Foundation, «The Eclipse Foundation Announces Sparkplug as an International Standard for a "Plug and Play" Industrial IoT». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://newsroom.eclipse.org/node/39754

BORRADOR

[16] Eclipse Sparkplug Contributors, «Sparkplug Specification, Version 3.0.0», Especificación, 2022. [En línea]. Disponible en: https://sparkplug.eclipse.org/specification/version/3.0/documents/sparkplug-specification-3.0.0.pdf

[17] inorbit-ai, «edge-sdk-python: InOrbit Python Edge SDK». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/edge-sdk-python

[18] R. T. Fielding, M. Nottingham, y J. Reschke, Eds., «HTTP Semantics», RFC 9110, 2022. doi: 10.17487/RFC9110.

[19] S. Loreto, P. Saint-Andre, S. Salsano, y G. Wilkins, «Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP», RFC 6202, 2011. doi: 10.17487/RFC6202.

[20] Z. Shelby, K. Hartke, y C. Bormann, «The Constrained Application Protocol (CoAP)», RFC 7252, 2014. doi: 10.17487/RFC7252.

[21] K. Hartke, «Observing Resources in the Constrained Application Protocol (CoAP)», RFC 7641, 2015. doi: 10.17487/RFC7641.

[22] N. Naik, «Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP», en *2017 IEEE International Systems Engineering Symposium (ISSE)*, Vienna, Austria: IEEE, 2017. doi: 10.1109/SysEng.2017.8088251.

[23] D. Thangavel, X. Ma, A. Valera, H.-X. Tan, y C. K.-Y. Tan, «Performance evaluation of MQTT and CoAP via a common middleware», en *2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, Singapore: IEEE, 2014. doi: 10.1109/ISSNIP.2014.6827678.

[24] D. P. Proos y N. Carlsson, «Performance comparison of messaging protocols and serialization formats for digital twins in IoV», en *2020 IFIP Networking Conference (Networking)*, Paris, France: IFIP, 2020.

[25] C. Bormann, S. Lemay, H. Tschofenig, K. Hartke, B. Silverajan, y B. Raymor, «CoAP (Constrained Application Protocol) over TCP, TLS, and WebSockets», RFC 8323, 2018. doi: 10.17487/RFC8323.

[26] A. Stanford-Clark y H. L. Truong, «MQTT For Sensor Networks (MQTT-SN) Protocol Specification, Version 1.2», Especificación técnica de IBM, 2013.

[27] T. Bray, Ed., «The JavaScript Object Notation (JSON) Data Interchange Format», RFC 8259, 2017. doi: 10.17487/RFC8259.

[28] J. C. Viotti y M. Kinderkhedia, «A Benchmark of JSON-compatible Binary Serialization Specifications», *arXiv preprint arXiv:2201.03051*, 2022, doi: 10.48550/arXiv.2201.03051.

[29] D. Friesel y O. Spinczyk, «Data Serialization Formats for the Internet of Things», *Electronic Communications of the EASST*, vol. 80, 2021, doi: 10.14279/tuj.eceasst.80.1134.

[30] S. Perreault, I. Yamagata, S. Miyakawa, A. Nakagawa, y H. Ashida, «Common Requirements for Carrier-Grade NATs (CGNs)», RFC 6888, 2013. doi: 10.17487/RFC6888.

[31] S. Guha, K. Biswas, B. Ford, S. Sivakumar, y P. Srisuresh, «NAT Behavioral Requirements for TCP», RFC 5382, 2008. doi: 10.17487/RFC5382.

[32] R. Braden, Ed., «Requirements for Internet Hosts – Communication Layers», RFC 1122, 1989. doi: 10.17487/RFC1122.

BORRADOR

[33] J. Moy, «OSPF Version 2», RFC 2328, 1998. doi: 10.17487/RFC2328.

[34] D. Katz y D. Ward, «Bidirectional Forwarding Detection (BFD)», RFC 5880, 2010. doi: 10.17487/RFC5880.

[35] G. Malkin, «RIP Version 2», RFC 2453, 1998. doi: 10.17487/RFC2453.

[36] D. Savage, J. Ng, S. Moore, D. Slice, P. Paluch, y R. White, «Cisco's Enhanced Interior Gateway Routing Protocol (EIGRP)», RFC 7868, 2016. doi: 10.17487/RFC7868.

[37] Y. Rekhter, T. Li, y S. Hares, Eds., «A Border Gateway Protocol 4 (BGP-4)», RFC 4271, 2006. doi: 10.17487/RFC4271.

[38] C. Labovitz, A. Ahuja, A. Bose, y F. Jahanian, «Delayed Internet Routing Convergence», en *Proceedings of the Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '00)*, Stockholm, Sweden: ACM, 2000.

[39] M. Shand y S. Bryant, «A Framework for Loop-Free Convergence», RFC 5715, 2010. doi: 10.17487/RFC5715.

[40] V. Paxson, M. Allman, J. Chu, y M. Sargent, «Computing TCP's Retransmission Timer», RFC 6298, 2011. doi: 10.17487/RFC6298.

[41] The Linux Kernel Developers, «IP Sysctl». Accedido: 6 de noviembre de 2026. [En línea]. Disponible en: https://docs.kernel.org/networking/ip-sysctl.html

[42] Zigbee2MQTT, «MQTT Topics and Messages». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://www.zigbee2mqtt.io/guide/usage/mqtt_topics_and_messages.html

[43] Signal, «WebSocketProtocol.proto (Signal-Server)». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/signalapp/Signal-Server/blob/main/websocket-resources/src/main/proto/WebSocketProtocol.proto

[44] L. Zhang, «Building Facebook Messenger». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://engineering.fb.com/2011/08/12/android/building-facebook-messenger/

[45] Apache Software Foundation, «About Apache Thrift». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://thrift.apache.org/about

[46] J. Fein, «Building Mobile-First Infrastructure for Messenger». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://engineering.fb.com/2014/10/09/production-engineering/building-mobile-first-infrastructure-for-messenger/

[47] J. Barber *et al.*, «Bladerunner: Stream Processing at Scale for a Live View of Backend Data Mutations at the Edge», en *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*, ACM, 2021, pp. 708-723. doi: 10.1145/3477132.3483572.

[48] VDA y VDMA, «VDA 5050: Interface for the Communication between Mobile Robots and a Fleet Control». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/VDA5050/VDA5050/blob/3.0.0/VDA5050_EN.md

[49] ISO, «Robotics – Industrial mobile robots – Communications and interoperability», International Standard ISO 21423:2026, oct. 2026. Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://www.iso.org/standard/86749.html

BORRADOR

[50] Association for Advancing Automation, «ISO 21423: Building Global Consensus for Mobile Robot Interoperability». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://www.automate.org/robotics/editorials/iso-21423-building-global-consensus-for-mobile-robot-interoperability

[51] Eclipse Sparkplug Contributors, «Sparkplug 3.0.0: Sparkplug Specification», Specification, 2022. Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://sparkplug.eclipse.org/specification/version/3.0/documents/sparkplug-specification-3.0.0.pdf

[52] Meshtastic, «Mesh Broadcast Algorithm». Accedido: 1 de noviembre de 2026. [En línea]. Disponible en: https://meshtastic.org/docs/overview/mesh-algo/

[53] Meshtastic, «Meshtastic Protobuf Definitions». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/meshtastic/protobufs

[54] Meshtastic, «MQTT Module Configuration». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://meshtastic.org/docs/configuration/module/mqtt/

[55] LoRa Alliance, «What is LoRaWAN». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://lora-alliance.org/about-lorawan/

[56] Semtech, «Basic communication protocol between Lora gateway and server». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/Lora-net/packet_forwarder/blob/master/PROTOCOL.TXT

[57] ChirpStack, «ChirpStack Gateway Bridge: Events». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://www.chirpstack.io/docs/chirpstack-gateway-bridge/payloads/events.html

[58] ChirpStack, «ChirpStack Gateway Bridge: Semtech UDP packet-forwarder». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://www.chirpstack.io/docs/chirpstack-gateway-bridge/backends/semtech-udp.html

[59] ChirpStack, «gw.proto». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/chirpstack/chirpstack/blob/master/api/proto/gw/gw.proto

[60] ChirpStack, «chirpstack-gateway-bridge: ChirpStack Gateway Bridge». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/chirpstack/chirpstack-gateway-bridge

[61] ChirpStack, «chirpstack: ChirpStack open-source LoRaWAN Network Server». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/chirpstack/chirpstack

[62] ChirpStack, «integration.proto». Accedido: 31 de octubre de 2026. [En línea]. Disponible en: https://github.com/chirpstack/chirpstack/blob/master/api/proto/integration/integration.proto

[63] J. Nagle, «Congestion Control in IP/TCP Internetworks», RFC 896, 1984. doi: 10.17487/RFC0896.

[64] W. M. Eddy, Ed., «Transmission Control Protocol (TCP)», RFC 9293, 2022. doi: 10.17487/RFC9293.

[65] D. Mills, J. Martin, J. Burbank, y W. Kasch, «Network Time Protocol Version 4: Protocol and Algorithms Specification», RFC 5905, 2010. doi: 10.17487/RFC5905.

BORRADOR

[66] T. Dierks y E. Rescorla, «The Transport Layer Security (TLS) Protocol Version 1.2», RFC 5246, 2008. doi: 10.17487/RFC5246.

[67] A. Banks y R. Gupta, Eds., «MQTT Version 3.1.1», OASIS, OASIS Standard, 2014. [En línea]. Disponible en: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[68] E. Rescorla, «The Transport Layer Security (TLS) Protocol Version 1.3», RFC 8446, 2018. doi: 10.17487/RFC8446.

[69] S. H. Strogatz, «From Kuramoto to Crawford: exploring the onset of synchronization in populations of coupled oscillators», *Physica D: Nonlinear Phenomena*, vol. 143, n.º 1–4, pp. 1-20, 2000, doi: 10.1016/S0167-2789(00)00094-4.

[70] Y. Kuramoto, «Self-entrainment of a population of coupled non-linear oscillators», en *International Symposium on Mathematical Problems in Theoretical Physics*, en Lecture Notes in Physics, vol. 39. Springer, 1975, pp. 420-422. doi: 10.1007/BFb0013365.

[71] M. Verwoerd y O. Mason, «Global phase-locking in finite populations of phase-coupled oscillators», *SIAM Journal on Applied Dynamical Systems*, vol. 7, n.º 1, pp. 134-160, 2008, doi: 10.1137/070686858.

[72] Espressif Systems, «ESP32-DevKitC-32E». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://www.mouser.com/ProductDetail/Espressif-Systems/ESP32-DevKitC-32E

[73] LilyGO, «T-A7670SA». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://wiki.lilygo.cc/products/t-sim-series/t-a7670/t-a7670sa/

[74] Hologram, «Pricing». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://www.hologram.io/pricing/

[75] Things Mobile, «Pay per use plan without subscription fee». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://www.thingsmobile.com/business/plans/pay-per-use-plan-without-subscription-fee

[76] 1NCE, «1NCE Connect Pricing». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://www.1nce.com/en-us/1nce-connect/pricing

[77] Amazon Web Services, «Amazon Lightsail Pricing». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://aws.amazon.com/lightsail/pricing/

[78] Amazon Web Services, «AWS IoT Core Pricing». Accedido: 7 de noviembre de 2026. [En línea]. Disponible en: https://aws.amazon.com/iot-core/pricing/

[79] InOrbit, «InOrbit.AI: We make robots work». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://www.inorbit.ai/

[80] OpenRobOps, «OpenRobOps: Open source robot fleet management». Accedido: 19 de octubre de 2026. [En línea]. Disponible en: https://openrobops.org/

[81] InOrbit, «inorbit-robot-connectors: InOrbit Robot Connectors». Accedido: 1 de noviembre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/inorbit-robot-connectors

[82] InOrbit, «Interoperability Standards (InOrbit Docs)». Accedido: 1 de noviembre de 2026. [En línea]. Disponible en: https://github.com/inorbit-ai/docs/blob/main/ground-control/robot-integration/interoperability.mdx

BORRADOR

[83] InOrbit, «InOrbit.AI Demonstrates the Future of Multi-Vendor Robot Orchestration and Physical AI at Automate 2026». Accedido: 1 de noviembre de 2026. [En línea]. Disponible en: https://www.inorbit.ai/press-release-inorbit.ai-automate-2026

BORRADOR

Anexos

A. Mensajes del experimento de sincronización

BORRADOR